МЕТОДИЧЕСКОЕ ИЗДАНИЕ В ПОМОЩЬ УЧАСТНИКАМ АКСЕЛЕРАТОРА МИР-ЛЭТИ.

# Часть А. Первичное знакомство с SDR и подготовка к hard-треку

**Авторы:**

Маркелов О. А. и Искаков Б. А.

Рабочая версия 0.7-dev

Санкт-Петербург, 2026

# Оглавление

**Использованные сокращения** 6

**Термины и определения** 10

**Предисловие** 13

**Введение** 18

Congratulations! . . . 18
Как пользоваться этим методическим указанием . . . 18
Акцентуация внимания . . . 18
Порог вхождения в SDR . . . 19
К концу части А читатель должен уметь: . . . 20
Основной и дополнительный маршруты . . . 20
ИИ-ассистенты и среда разработки . . . 20
Чего нет в части А: . . . 21
Краткое описание материала глав . . . 22

**1 Стенд, безопасность и комплект** 23

1.1 Состав комплекта . . . 24
1.2 Путь сигнала . . . 24
1.3 Уровни и защита входа . . . 27
1.3.1 Разные шкалы уровня . . . 27
1.3.2 Бюджет закрытого тракта . . . 29
1.3.3 Проверка до подачи сигнала . . . 31
1.4 Кабельный и эфирный сценарии . . . 32
1.4.1 Кабельный тракт . . . 32
1.4.2 Эфирный тракт в контролируемом месте . . . 33
1.4.3 Выбор сценария . . . 34
1.5 S11 и согласование антенны . . . 34
1.6 Диаграмма направленности . . . 36
1.7 Паспорт стенда . . . 38

**2 Воспроизводимая программная среда, Git и сборка** 41

2.1 Как из исходного кода получается программа . . . 42

2.2 Каноническая и альтернативные среды . . . 44
2.3 Основы файловой системы Linux и работа в терминале . . . 45
2.3.1 Переменные окружения . . . 47
2.3.2 Как программа находит разделяемые библиотеки . . . 48
2.4 Git как фиксация эксперимента . . . 50
2.5 Репозиторий и навигация по коду . . . 51
2.6 Сборка программ в каталоге проекта . . . 53
2.7 libiio, подключение QPSK SDR Lab и каналы 2RX2TX . . . 55
2.7.1 Локальная сборка libiio и libad9361-iio . . . 55
2.7.2 Подключение и сетевые настройки . . . 58
2.7.3 Проверка каналов без запуска потока . . . 60
2.8 Автоматические тесты . . . 61
2.8.1 Где описаны тесты . . . 61
2.8.2 Запуск и чтение результата . . . 62
2.8.3 XML-описание платы вместо платы . . . 63
2.9 Паспорт программной среды . . . 65

**3 SDR-платформа изнутри: AD9361, Zynq и libiio 67**
3.1 Трансивер AD9361 . . . 67
3.2 Цифровая цепочка и частота дискретизации . . . 71
3.3 Плата: AD9361 и Zynq-7010 . . . 73
3.4 Как устроена libiio . . . 75
3.5 Что показывает iio_info . . . 77
3.6 Порядок настройки трансивера . . . 79
3.7 Предел потока отсчётов . . . 80
3.8 Проверка платы перед работой . . . 82

**4 I/Q-отсчёты и минимальный инструментарий ЦОС 85**
4.1 От времени к номеру отсчёта . . . 86
4.1.1 Блок как структурная единица сигнала . . . 86
4.2 Комплексная базовая полоса . . . 87
4.2.1 Почему одного смесителя недостаточно . . . 88
4.2.2 Квадратурный приём и передача . . . 90
4.2.3 Комплексная огибающая и аналитический сигнал . . . 94
4.2.4 Что означает нулевая частота . . . 98
4.2.5 Комплексный тон, NCO и цифровая настройка . . . 98

4.3 Дискретизация и алиасинг . . . 101
4.3.1 Почему одной настройки $f_s$ недостаточно . . . 102
4.4 Децимация и интерполяция . . . 106
4.4.1 Децимация . . . 106
4.4.2 Интерполяция . . . 107
4.4.3 Ловушка нормированной частоты . . . 108
4.5 Дискретное преобразование Фурье . . . 110
4.5.1 Утечка и оконные функции . . . 111
4.5.2 FFTW3 . . . 114
4.5.3 Нормировка FFTW3 . . . 115
4.6 Амплитуда, мощность и децибелы . . . 116
4.6.1 Относительные уровни в децибелах . . . 116
4.6.2 Абсолютные уровни мощности: дБВт и дБм . . . 116
4.6.3 Цифровой уровень и калибровка SDR . . . 118
4.7 Форматы IQ . . . 119
4.7.1 Соглашение этого методического пособия для файлов . . . 120
4.7.2 Минимальные метаданные . . . 121
4.7.3 Контрольный IQ-файл . . . 121
4.7.4 Порядок проверки . . . 123

**5 Первый приём и анализ спектра 125**
5.1 От супергетеродина к SDR . . . 125
5.2 Подготовка: библиотеки и сборка . . . 127
5.3 Минимальный C++ RX-тракт . . . 128
5.4 Четыре частотных параметра . . . 130
5.5 Усиление и АРУ . . . 131
5.6 Время, спектр и водопад . . . 132
5.7 Запись с метаданными . . . 135
5.8 Демодуляция FM . . . 135
5.9 Программный тракт без платы . . . 137

**6 Аппаратный тракт TX→RX и AM/FM как диагностические сигналы 138**
6.1 От классического передатчика к SDR . . . 139
6.2 Схема опыта и допуск . . . 139
6.3 Тестовые сигналы и циклический буфер . . . 140
6.4 Программа передачи . . . 142

6.5 Комплексный тон . . . . . 143
6.6 AM как тест амплитудного тракта . . . . . 144
6.7 FM как тест фазочастотного тракта . . . . . 145
6.8 Лестница уровней . . . . . 146
6.9 Сравнение с программным эталоном . . . . . 146

**Список использованных источников** **149**

# Использованные сокращения

| | |
|---|---|
| **ADC** | Analog-to-Digital Converter, (рус. АЦП) — аналого-цифровой преобразователь. |
| **AGC** | Automatic Gain Control, (рус. АРУ) — автоматическая регулировка усиления. |
| **AM** | Amplitude Modulation, (рус. АМ) — амплитудная модуляция. |
| **API** | Application Programming Interface — программный интерфейс библиотеки: набор функций и типов, через которые к ней обращается программа. |
| **AUXADC, AUXDAC** | Auxiliary ADC, Auxiliary DAC — вспомогательные АЦП и ЦАП трансивера для контроля и управления внешними узлами. |
| **AXI** | Advanced eXtensible Interface — внутренняя шина Zynq между процессорной частью и ПЛИС. |
| **BBPLL** | Baseband PLL — синтезатор тактовых частот АЦП, ЦАП и цифрового интерфейса трансивера. |
| **DAC** | Digital-to-Analog Converter, (рус. ЦАП) — цифро-аналоговый преобразователь. |
| **dBFS** | Decibels relative to Full Scale — уровень в децибелах относительно полной шкалы АЦП или ЦАП. |
| **DC** | Direct Current — постоянная составляющая; DC-блокер — элемент тракта, не пропускающий постоянный ток. |
| **DDR3** | Double Data Rate 3 — тип динамической оперативной памяти. |
| **DDS** | Direct Digital Synthesis — прямой цифровой синтез: генератор тона, вычисляющий отсчёты синусоиды в ПЛИС. |
| **DFT** | Discrete Fourier Transform, (рус. ДПФ) — дискретное преобразование Фурье. |
| **DMA** | Direct Memory Access, (рус. ПДП) — прямой доступ к памяти: передача данных между устройством и памятью без участия процессора. |
| **DSP** | Digital Signal Processing, (рус. ЦОС) — цифровая обработка сигналов; Digital Signal Processor, (рус. ЦСП) — цифровой сигнальный процессор, в зависимости от контекста. |
| **ENSM** | Enable State Machine — конечный автомат режимов транси- |

вера AD9361 (сон, ожидание, приём, передача).

**FFT** Fast Fourier Transform, (рус. БПФ) — быстрое преобразование Фурье.

**FIR** Finite Impulse Response, (рус. КИХ) — конечная импульсная характеристика.

**FM** Frequency Modulation, (рус. ЧМ) — частотная модуляция.

**FPGA** Field-Programmable Gate Array, (рус. ПЛИС) — программируемая логическая интегральная схема.

**GPO** General-Purpose Output — выход общего назначения.

**HB** Half-Band filter — полуполосный фильтр с фиксированными коэффициентами в цепочке децимации и интерполяции AD9361.

**I/Q** In-phase/Quadrature — синфазная и квадратурная составляющие комплексного отсчёта.

**IEEE** Institute of Electrical and Electronics Engineers — Институт инженеров электротехники и электроники, разработчик стандартов (IEEE 754, IEEE 802.11).

**IIO** Industrial Input/Output — подсистема ядра Linux для устройств ввода-вывода (АЦП, ЦАП, трансиверов); libiio — библиотека доступа к ним, в том числе по USB и сети.

**IP** Internet Protocol — межсетевой протокол; IP-адрес — адрес узла в сети.

**JSON** JavaScript Object Notation — текстовый формат данных; в курсе в нём хранятся метаданные IQ-файлов.

**JTRS** Joint Tactical Radio System — программа единых тактических систем радиосвязи Минобороны США.

**LNA** Low Noise Amplifier, (рус. МШУ) — малошумящий усилитель.

**LO** Local Oscillator — гетеродин: генератор, сигнал которого переносит спектр по частоте.

**LPF** Low Pass Filter, (рус. ФНЧ) — фильтр нижних частот.

**LVDS** Low-Voltage Differential Signaling — дифференциальная передача сигналов малыми напряжениями; цифровой интерфейс между AD9361 и ПЛИС.

**NCO** Numerically Controlled Oscillator, (рус. ЦУГ) — численно управляемый генератор.

**OTG** On-The-Go — режим USB, в котором устройство может быть

| | |
|---|---|
| | и ведущим, и ведомым. |
| **PL, PS** | Programmable Logic, Processing System — программируемая логика (ПЛИС) и процессорная часть системы на кристалле Zynq. |
| **PLL** | Phase-Locked Loop, (рус. ФАПЧ) — система фазовой автоподстройки частоты; синтезатор частоты на её основе. |
| **QAM** | Quadrature Amplitude Modulation, (рус. КАМ) — квадратурная амплитудная модуляция. |
| **QPSK** | Quadrature Phase Shift Keying, (рус. КФМн) — квадратурная фазовая манипуляция. |
| **QSPI** | Quad SPI — четырёхпроводный вариант SPI для быстрой флеш-памяти; на плате в ней хранится прошивка. |
| **RF** | Radio Frequency, (рус. РЧ) — радиочастота или радиочастотный тракт. |
| **RFSoC** | Radio Frequency System-on-Chip — радиочастотная система на кристалле: интегральная платформа, объединяющая процессорную систему, программируемую логику и высокоскоростные АЦП и ЦАП. |
| **RL** | Return Loss — возвратные потери: отношение падающей мощности к отражённой в децибелах, $RL = -20\log_{10}\lvert\Gamma\rvert$. |
| **RMS** | Root Mean Square, (рус. СКЗ) — среднеквадратическое значение. |
| **RX, TX** | Receiver, Transmitter — приёмник и передатчик, а также их каналы и разъёмы. |
| **SCA** | Software Communications Architecture — архитектура программного обеспечения радиосредств, разработанная в программе JTRS. |
| **SDR** | Software Defined Radio, (рус. ПКР) — программно-конфигурируемое радио. |
| **SHA-256** | Secure Hash Algorithm — алгоритм контрольной суммы; совпадение суммы подтверждает неизменность файла. |
| **SMA** | SubMiniature version A — коаксиальный радиочастотный разъём с резьбовым соединением. |
| **SPI** | Serial Peripheral Interface — последовательный периферийный интерфейс; по нему процессор записывает регистры AD9361. |
| **TCP** | Transmission Control Protocol — протокол надёжной передачи |

| | |
|---|---|
| | данных в IP-сетях; по нему libiio связывается с сервером iiod. |
| **TU** | Time Unit — единица времени стандарта IEEE 802.11 (Wi-Fi), 1 TU = 1,024 мс. |
| **URI** | Uniform Resource Identifier — строка, задающая способ подключения и адрес устройства, например `ip:192.168.2.1`. |
| **USB** | Universal Serial Bus — универсальная последовательная шина. |
| **VCTCXO** | Voltage-Controlled Temperature-Compensated Crystal Oscillator — кварцевый генератор с термокомпенсацией и подстройкой напряжением; опорный генератор платы. |
| **VSWR** | Voltage Standing Wave Ratio, (рус. КСВН) — коэффициент стоячей волны по напряжению. |
| **WAV** | Waveform Audio File Format — формат звуковых файлов. |
| **XML** | eXtensible Markup Language — расширяемый язык разметки; в нём libiio описывает устройства платы. |
| **ZIF** | Zero Intermediate Frequency — приёмник прямого преобразования, нулевая промежуточная частота. |
| **АЧХ** | Амплитудно-частотная характеристика. |
| **ВЧ, СВЧ** | Высокая частота, сверхвысокая частота. |
| **ГУН** | Генератор, управляемый напряжением (англ. VCO). |
| **ДН** | Диаграмма направленности антенны. |
| **ИИ** | Искусственный интеллект. |
| **КПД** | Коэффициент полезного действия. |
| **ООП** | Объектно-ориентированное программирование. |
| **ОС** | Операционная система. |
| **ПК** | Персональный компьютер. |
| **ПО** | Программное обеспечение. |
| **ПЧ** | Промежуточная частота. |
| **УМ** | Усилитель мощности. |
| **УНЧ** | Усилитель низкой частоты. |
| **УПЧ** | Усилитель промежуточной частоты. |
| **УРЧ** | Усилитель радиочастоты. |

# Термины и определения

**Отсчёты, частота и спектр**

$f_s$ (или $F_s$) — Частота дискретизации, [Гц].

$\Omega_s = 2\pi f_s$ — Циклическая частота дискретизации, [рад/с].

$T_s$ — Период дискретизации: $T_s = 1/f_s$, [с].

$n$ — Номер отсчёта сигнала.

$N$ — Количество отсчётов сигнала (длина ДПФ или БПФ).

$x[n]$ — $n$-й комплексный отсчёт сигнала.

$I[n]$ — Синфазная составляющая $n$-го отсчёта.

$Q[n]$ — Квадратурная составляющая $n$-го отсчёта.

$A$ — Амплитуда сигнала; в главе 1 с индексом — ослабление элемента тракта, [дБ].

$\varphi$ — Фаза сигнала, [рад].

$f_0$ — Центральная (несущая) частота, [Гц].

$\Omega_0 = 2\pi f_0$ — Круговая несущая частота, [рад/с].

$f_c$ — Частота сигнала (смещение относительно $f_0$), [Гц].

$f_{\text{RF}}$ — Радиочастота компоненты сигнала: $f_{\text{RF}} = f_0 + f_c$, [Гц].

$X[k]$ — $k$-й комплексный отсчёт спектра сигнала.

$k$ — Номер частотного отсчёта (FFT-бина).

$\Delta f$ — Частотный шаг: $\Delta f = f_s/N$, [Гц].

$W$ — Ширина полосы частот; в главе 5 — аналоговая полоса приёмника, [Гц].

$B$ — Занимаемая полоса сигнала, [Гц].

$t_0$ — Момент времени, соответствующий первому отсчёту, [с].

$T_{\text{obs}}$ — Длительность наблюдаемого блока сигнала, [с].

$T_{\text{blk}}$ — Длительность блока ДПФ: $T_{\text{blk}} = N/F_s$, [с].

$N_{\text{avg}}$ — Число усредняемых спектров.

$f_{\text{NCO}}$ — Частота численно управляемого генератора, [Гц].

$w[n]$ — $n$-й коэффициент оконной функции.

$G_c$ — Когерентный коэффициент усиления окна.

$h[n]$ — $n$-й коэффициент импульсной характеристики FIR-фильтра.

$M_h$ — Число коэффициентов FIR-фильтра.

$M$ — Целочисленный коэффициент децимации в записи рационального преобразования частоты $L/M$.

$D$ — Целочисленный коэффициент децимации.

$L$ — Целочисленный коэффициент интерполяции.

$\ell$ — Дискретная задержка в корреляционной функции, [отсчёты].

$\tau$ — Постоянная времени цепи коррекции предыскажений FM, [с].

### Уровни и бюджет тракта

$P$, $P_{\text{dBm}}$ — Мощность, [мВт], и её уровень относительно 1 мВт, [дБм].

$G$ — Усиление, [дБ].

$A_{\text{att}}$ — Ослабление аттенюаторов, [дБ]; индексы nom и min здесь и далее — номинальное и минимальное с учётом допуска значения.

$A_{\text{cable}}$ — Потери кабелей, [дБ].

$A_{\text{other}}$ — Потери прочих пассивных элементов, [дБ].

$P_{\text{TX,max}}$ — Верхняя граница мощности передатчика, [дБм].

$P_{\text{RX,worst}}$ — Наихудший (наибольший) уровень на входе приёмника, [дБм].

$P_{\text{limit}}$ — Паспортный предел входной мощности приёмника, [дБм].

$M_{\text{safe}}$, $M_{\text{req}}$ — Запас до паспортного предела и требуемый рабочий запас, [дБ].

$Z_0$, $Z_{\text{L}}$ — Опорный импеданс линии и импеданс нагрузки, [Ом].

$\Gamma$ — Комплексный коэффициент отражения.

$S_{11}$ — Коэффициент отражения однопортового измерения; $S_{11,\mathrm{dB}} = 20\log_{10}|\Gamma|$, [дБ].

$L_{\mathrm{mismatch}}$ — Потери рассогласования, [дБ].

$L_{\mathrm{FS}}$ — Потери распространения в свободном пространстве, [дБ].

$G_{\mathrm{TX}}$, $G_{\mathrm{RX}}$ — Коэффициенты усиления передающей и приёмной антенн, [дБ].

$\lambda$ — Длина волны: $\lambda = c/f_0$, [м].

$R$ — Расстояние между антеннами, [м].

**Платформа и поток данных**

$N_{\mathrm{RX}}$ — Число одновременно принимаемых каналов.

$R_{\mathrm{IQ}}$ — Поток IQ-данных от платы к ПК: $R_{\mathrm{IQ}} = 4N_{\mathrm{RX}}F_s$, [Б/с].

*«Что не осознаётся, возвращается к нам в виде судьбы».*

(Карл Густав Юнг)

# Предисловие

В технологиях беспроводной связи происходит революция, затрагивающая все сферы — от разработки аппаратного обеспечения до информационной безопасности и от аэрокосмической инженерии до любительской связи. Профессиональные разработчики электроники, исследователи в области кибербезопасности, хакеры, работающие с аппаратным обеспечением, и операторы любительских радиостанций — каждый может воспользоваться преимуществами нового прекрасного сочетания: программного обеспечения и радио.

Термин «программное радио» был введён Дж. Митолой (J. Mitola) в начале 1990-х годов. Основная идея оригинальной статьи заключается в том, чтобы ограничить аналоговую часть радиоприёмника, расположенную рядом с антенной, ВЧ-усилителем и фильтрами, после чего использовать высокоскоростной АЦП для преобразования сигнала основной полосы частот в цифровую форму. После этого все обычные операции (демодуляция, фильтрация и т. д.) выполняются в цифровом виде с помощью ЦСП. Это было бы идеальным решением для SDR, которое достижимо сегодня, хотя и остаётся довольно дорогим, таким, например, как радиосистема AMD-Xilinx Zynq Ultrascale+ RFSoC с FPGA.

С момента публикации статьи Дж. Митолы прошло более 30 лет, когда он работал на Министерство обороны (*Министерство войны*) США. В связи со сложившимися историческими обстоятельствами регламентация в области SDR тесно связана с понятием программно-определяемой архитектуры связи (SCA, Software Communications Architecture), спроектированной вышеупомянутым министерством в рамках работ по единым тактическим системам радиосвязи (JTRS, Joint Tactical Radio System), которые проводились с 1997 г. с целью разработки стандарта SDR. Архитектура SCA была призвана обеспечить работу систем военной радиосвязи за счёт стандартизированных интерфейсов прикладного программирования. В целях унификации оборудование, производимое для нужд МВ США, должно было отвечать требованиям SCA. В Европе концепция SCA также получила развитие для оборонных нужд под названием ESSOR (European Secure Software-defined Radio — проект програм-

мы Европейского Союза по созданию SDR).

Дадим несколько различных определений SDR, которые встречаются в международных стандартах и профильной литературе:

- SDR согласно WInnF (Wireless Innovation Forum — SDR-форум, по сути послуживший инициатором по международной регламентации и стандартизации в области SDR) — это радиооборудование, в котором все или большинство функций физического уровня выполняются в *программном виде*, а функции, выполняемые аппаратно, должны оперативно модифицироваться по требованиям рабочего стандарта связи.
- SDR — направление развития радиосвязи, призванное объединить на единой аппаратной платформе работу радиостанций различных типов и стандартов. Предполагается, что SDR-радио реализует функции радиоприёмника и радиопередатчика в цифровой форме и с помощью программно управляемых аппаратных компонентов, которые в силу своей физической природы не могут быть реализованы программно: усилитель мощности или антенна.
- SDR согласно IEEE — это программно-конфигурируемое оборудование, в котором радиочастотные параметры могут быть установлены или изменены при помощи программного обеспечения и/или оборудования, с помощью которого это достигается. Программное изменение касается диапазона частот, типа модуляции, выходной мощности, но не ограничивается этими параметрами.
- SDR согласно Федеральной комиссии связи США — это радиооборудование, в котором такие параметры, как диапазон частот, тип модуляции и выходная мощность могут быть изменены при помощи программного обеспечения без изменений в аппаратных компонентах, используемых для излучения и приёма радиочастот.

Имеющееся обилие определений термина SDR у неискушённого в этой теме новичка может вызвать некоторый вакуум понимания:

«Так что же всё-таки такое этот ваш SDR, о котором в последние годы из каждого утюга слышно?»

Так вот, уважаемый читатель, предлагаю вместе со мной провести вполне реальный вживую, но пока, при текущих вводных, следующий мысленный эксперимент:

«Неделя жизни пользователя SDR».

Понедельник.

*Промозглым утром в сером году*
*Я на работу иду*
*Средь сотен сонных людей...*

*А там в светлом просторном бюро*
*Сидят, прищурясь хитро,*
*Творцы несвежих идей...*

*И вот приходит липкая лень...*
*И вот приходит липкая лень,*
*На мысли падает тень,*
*И в голове дребедень...*

— Сергей Минаев

Придя на работу, вы вдруг обнаружили, что после введения новых санкций в адрес вашей страны вам заблокировали доступ к облачному сервису управления системой «умных чайников, кофеварок, ламп», а утро без кофе обречено на провал.

Вы, как истинный SDR-щик, подходите к решению проблемы оригинально: вы знаете, что управление в системе «умных» устройств организовано по радиоканалу, поверх которого лежит протокол Zigbee. Вы достаёте из широкого ящика SDR отечественного образца и начинаете писать стек взаимодействия с устройствами, «выжившими из ума». Накопленный опыт работы с SDR и ранее реализованные проекты помогают вам за 15 минут написать софт для управления системой и привести все устройства в чувство.

Далее, вы понимаете, что управлять через консоль всеми устройствами не очень удобно и, уже налив себе кофе, с помощью ИИ вайбкодите графическую оболочку и деплоите все на корпоративный сервер.

Voilà! Вы только что создали собственный хаб и ПО управления «умными» устройствами.

Вторник.

Вы обнаруживаете, что в сети Wi-Fi возникли какие-то неполадки. Подозревая проблемы с безопасностью, вы перенастраиваете SDR для одновременного сканирования всех 14 доступных каналов Wi-Fi и определения типа

трафика. Вы сохраняете все необработанные радиочастотные данные в файл для дальнейшей обработки на досуге.

Среда.

В лаборатории внезапно перестаёт отвечать беспроводной датчик температуры. Вы настраиваете SDR на частоту 433,92 МГц, сравниваете несколько принятых посылок и выясняете, что датчик исправен, а его сигнал заглушает помеха от нового импульсного блока питания.

Четверг.

Над зданием подозрительно долго кружит самолёт, а привычный интернет-сервис недоступен. Приняв пакеты ADS-B на частоте 1090 МГц, вы самостоятельно определяете борт, его высоту и траекторию, а заодно собираете небольшую локальную карту воздушного движения.

Пятница.

Прогноз погоды обещает солнце, хотя за окном собираются тучи. Во время ближайшего пролёта метеорологического спутника вы принимаете сигнал в диапазоне 137–138 МГц и из записанных отсчётов восстанавливаете изображение облачности над своим регионом.

Суббота.

Знакомый приносит старый радиоприёмник и жалуется, что его шкала настройки безбожно врёт. Просканировав вещательный диапазон 88–108 МГц и сопоставив найденные станции, вы строите таблицу поправок и возвращаете приёмнику способность попадать точно в любимую волну.

Воскресенье.

В эфире непривычно тихо, и возникает вопрос: сломалась антенна или изменилось прохождение? Вы оставляете SDR наблюдать цифровые сигналы около 14,074 МГц и по принятым позывным оцениваете, в какие стороны сегодня открылась ионосферная трасса.

Начинаете понимать, насколько мощные эти устройства? Вы можете не только легко внедрять новые схемы радиосвязи, но и переключаться между

ними, используя всего несколько нажатий клавиш или вызывая функцию в коде. И это лишь примеры связных приложений, что уж говорить о радиолокации, медицине и других сферах деятельности...

Для полноты картины следует упомянуть, что во всём этом есть одна сложность: несмотря на то что радиостанции стали программируемыми, инженеры до сих пор не придумали, как сделать доступные программируемые антенны. По крайней мере, на данный момент при значительных изменениях несущей частоты вам, вероятно, придётся менять антенны.

Подведём итог предисловию и выделим ключевые особенности программно-конфигурируемого радио:

- перенос значительной части обработки сигналов в программную область позволяет изменять рабочую частоту, полосу, вид модуляции и способ доступа к каналу без полной переработки устройства;
- единая аппаратная платформа может поддерживать различные стандарты связи и экспериментальные радиоинтерфейсы в пределах возможностей её ВЧ-тракта, АЦП, ЦАП, вычислительных ресурсов и антенны;
- программно-управляемое оборудование позволяет создавать действующие прототипы, реализующие функции физического уровня, и проверять новые алгоритмы непосредственно в радиоэфире;
- повторное использование одной платформы и набора программных компонентов сокращает время и стоимость перехода от модели к экспериментальному образцу по сравнению с изготовлением отдельного аппаратного макета для каждой новой идеи.

Таким образом, SDR — это не просто радиостанция с программными настройками, а гибкая инженерная среда, объединяющая математическую модель, алгоритм обработки сигналов и действующий радиопрототип. Её главная ценность заключается в возможности быстро превратить идею в проверяемую систему, адаптировать её к новым требованиям и использовать одну аппаратную основу для решения широкого круга задач.

*«Наука должна быть весёлая, увлекательная и простая. Таковыми же должны быть и учёные».*

(Пётр Леонидович Капица)

# Введение

## Congratulations!

Хочу тебя поздравить, уважаемый читатель, с тем, что ты решился познавать радио по ещё неистоптанному пути: через освоение SDR!

При написании этого методического указания мы как авторы ставили перед собой следующие цели:

- помочь читателю сделать первые шаги к серьёзной разработке на основе программно-конфигурируемого радио (SDR) и подготовить его к hard-треку;
- сформировать у читателя базовое видение комплексности системы на основе SDR;
- предложить читателю одну из парадигм (*концептуальная рамка*) разработки систем на основе SDR;
- оставить у читателя программную и документальную базу, которая позволит ему самостоятельно повторять эксперименты и развивать их в дальнейшем.

## Как пользоваться этим методическим указанием

### Акцентуация внимания

В методическом указании используются блоки акцентуации внимания, такие как *Важно*, *Внимание* и *Контрольная точка*.

**Важно**

В данном блоке приводится материал, который авторы считают особенно важным для понимания и усвоения. Читателю рекомендуется внимательно изучить этот материал и убедиться, что он его понял.

**Внимание**

Этот блок содержит предупреждения о возможных ошибках, неисправностях и повреждениях, которые могут возникнуть при выполнении экспериментов, а также в нём содержатся настоятельные рекомендации по их предотвращению.

**Контрольная точка**

Контрольная точка — это блок, в котором авторы предлагают сверить внутренние часы читателя с их собственными часами, чтобы убедиться, что читатель усвоил материал и готов двигаться дальше.
В данном блоке зачастую будут присутствовать вопросы, на которые читателю предлагается ответить самостоятельно, и задачи для самостоятельного решения. Символом * помечены задачи и вопросы повышенной сложности, которые авторы рекомендуют решать только после того, как читатель освоит весь материал главы.

## Порог вхождения в SDR

Поскольку разработка на основе SDR требует владения широкими компетенциями, авторы методического указания рекомендуют читателю освежить свои знания в следующих областях:

- базовая электроника и схемотехника (аналоговые/цифровые схемы и их компоненты; фильтры; усилители; АЦП/ЦАП);
- основы ЦОС (дискретизация и квантование; спектральный анализ; фильтрация; преобразование Фурье, Гильберта; оконные функции; алгоритмы БПФ);
- основы статистической радиотехники и её математического аппарата (теория вероятности, кумулянты, распределения; спектральный анализ, корреляция; линейная алгебра, матрицы, векторы, комплексные числа; теория различения сигналов и оценки случайных параметров последних);
- основы СВЧ-техники (фидеры, антенны; аттенюаторы; согласование; отражения);
- основы программирования (C++ — контейнеры, классы, ООП).

## К концу части A читатель должен уметь:

- подключить плату QPSK SDR Lab через libiio, проверить её каналы и поток отсчётов;
- собирать C++-код в RED OS (или иной Linux-системе по выбору читателя) и запускать его;
- создать и прочитать IQ-файл, проверить его метаданные;
- построить временное представление, спектр и водопад сигнала;
- принять реальный радиосигнал, записать его и демодулировать FM-станцию;
- провести один и тот же тест на программном эталоне в файле и через безопасный аппаратный тракт;
- локализовать ошибку по слоям и доказать исправление контрольным измерением;
- завершать практическую работу по главе паспортом эксперимента, конфигурацией, контрольными изображениями и короткими/тезисными причинными выводами.

## Основной и дополнительный маршруты

Основной маршрут написан на C++ и использует libiio, FFTW3 и OpenCV. MATLAB можно применять для независимой проверки формул и результатов. GNU Radio не запрещён, но не является предметом этого методического пособия: студент, которому удобен этот инструмент, может самостоятельно повторить в нём тот же эксперимент.

**Внимание**

В целом авторы не настаивают на строгом следовании маршрута: читатель волен самостоятельно выбирать инструменты и языки программирования, которые ему удобны, при этом осознавая риски того, что используемые им инструменты могут не поддерживать все программно-архитектурные возможности, требуемые для выполнения заданий как и по Lite, так и по Hard-треку.

## ИИ-ассистенты и среда разработки

Использовать ИИ-ассистенты при выполнении работы разрешено. Удобная связка — редактор VS Code с одним из расширений: Claude Code, Codex или GitHub Copilot. Другой вариант — открытый агент Goose, подключённый к

API выбранной языковой модели, например DeepSeek. Ассистент видит файлы проекта, может объяснить код, предложить правку и запустить сборку или тесты.

Полезные сценарии:

- попросить объяснить незнакомый фрагмент кода, сообщение компилятора или ошибку компоновки;
- написать небольшой блок — функцию, тест, скрипт построения графика — и затем разобрать его построчно;
- обсудить гипотезы при отладке: приложить команду, полный текст ошибки и версии инструментов;
- перепроверить вывод формулы или пересчитать контрольный пример.

Правила работы с ассистентом:

- просите маленькие изменения и читайте каждое из них перед тем, как принять; после правки собирайте проект и запускайте тесты;
- делайте commit до и после работы агента, чтобы любую его правку можно было просмотреть командой `git diff` и откатить;
- числа — допустимые мощности, диапазоны, параметры API — сверяйте с документацией: языковые модели уверенно ошибаются именно в них;
- не передавайте в облачные сервисы пароли, токены и закрытые данные;
- отмечайте в отчёте, в каких частях работы использовался ИИ.

**Внимание**

Отдать ассистенту задание целиком и сдать результат, не разобравшись в нём, нельзя. Автор работы должен уметь объяснить каждую строку своего кода и каждое число в отчёте. Код, который вы не понимаете, нельзя ни отладить на стенде, ни защитить.

## Чего нет в части A:

- полного описания архитектуры и программных интерфейсов всех популярных SDR устройств;
- готовой инструкции по решению задач Hard-трека, поскольку Hard-трек — это самостоятельная исследовательская работа читателя;
- глубокого погружения в теорию физического и аппаратного уровней SDR, поскольку это предмет отдельного курса.

## Краткое описание материала глав

**Глава 1. Стенд, безопасность и комплект.**

Состав комплекта QPSK SDR Lab, путь сигнала от антенны до программы, расчёт бюджета уровней и решение *GO/NO-GO* до первой передачи, $S_{11}$, КСВН и диаграмма направленности антенны, паспорт стенда.

**Глава 2. Воспроизводимая программная среда, Git и сборка.**

Основы Linux и Git, сборка учебных программ и локальных libiio и libad9361-iio, подключение платы и проверка каналов 2RX2TX, паспорт программной среды.

**Глава 3. SDR-платформа изнутри: AD9361, Zynq и libiio.**

Узлы трансивера AD9361 и их атрибуты, цепочка децимации и пределы $F_s$, архитектура платы, устройство libiio и разбор вывода `iio_info`, порядок настройки трансивера, измерение предела потока отсчётов и проверка платы перед работой.

**Глава 4. I/Q-отсчёты и минимальный инструментарий ЦОС.**

Комплексная базовая полоса, дискретизация и алиасинг, NCO, децимация и интерполяция, ДПФ и FFTW3, уровни в децибелах и формат учебных IQ-файлов.

**Глава 5. Первый приём и анализ спектра.**

Приём FM-диапазона на C++ и libiio, частотные параметры приёмника, усиление и перегрузка АЦП, спектр и водопад, запись с метаданными и демодуляция FM-станции.

**Глава 6. Аппаратный тракт TX→RX.**

Передача тона, AM- и FM-сигналов через безопасный кабельный тракт и сравнение с программным эталоном.

# 1 Стенд, безопасность и комплект

Во введении был задан общий маршрут части A: от воспроизводимой программной модели к реальному радиочастотному тракту. До установки программ и первого запуска платы нужно понять, из чего состоит стенд, где в нём проходит физический сигнал, как он преобразуется по пути и какие уровни допустимы в каждой точке. Иначе даже корректная программа может перегрузить или повредить вход приёмника.

После изучения главы читатель должен уметь:

- перечислить состав комплекта и объяснить назначение каждого элемента;
- проследить пути RX и TX от антенны до C++-программы и обратно;
- рассчитать номинальный и наихудший уровни на входе RX до соединения портов и обосновать решение *GO* или *NO-GO*;
- собрать кабельный или эфирный тракт так, чтобы опыт можно было повторить;
- связать $S_{11}$, коэффициент отражения, возвратные потери и КСВН и читать диаграмму направленности антенны;
- заполнить паспорт стенда.

**Важно**

Надпись на кабеле или аттенюаторе и значение gain в программе сами по себе не доказывают безопасность. Доказательство — это расчёт: верхняя граница мощности передатчика, минимальное ослабление тракта с учётом допусков, допустимая мощность каждого элемента и полученный уровень на входе RX.

**Внимание**

Прямое соединение TX→RX без рассчитанного внешнего ослабления запрещено! В этой главе передатчик не включают: сначала составляют схему, рассчитывают уровни и собирают пассивную цепь. Положительный результат расчёта не является командой на передачу — аппаратный опыт выполняется только в предусмотренной для него главе.

## 1.1 Состав комплекта

Аппаратная платформа методического пособия называется **QPSK SDR Lab**. Её комплект, который поставляется участникам акселератора, приведён в таблице 1.1. Даже у двух внешне одинаковых стендов могут различаться ревизия платы, кабели, переходники и рабочий диапазон антенн, а вместе с ними — схема подключения и бюджет уровней. Поэтому состав своего комплекта нужно сверить с таблицей и зафиксировать отличия.

**Таблица 1.1:** Состав комплекта QPSK SDR Lab

| Элемент | Основные характеристики |
|---|---|
| Плата QPSK SDR Lab | 2 канала RX и 2 канала TX; диапазон 70 МГц – 6 ГГц; $F_s$ до 61,44 Мвыб/с (в режиме 1RX1TX); аналоговая полоса $W$ от 200 кГц до 56 МГц. |
| Антенны | 4 шт., двухдиапазонные: 1800 МГц (полоса 240 МГц) и 4680 МГц (полоса 120 МГц); разъём SMA. |
| Коаксиальные кабельные сборки | 50 Ом, длина 0,3 м, кабель RG402, разъёмы SMA. |
| Фиксированные аттенюаторы | 6, 10, 20 и 30 дБ, по одному; до 8 ГГц; SMA. |
| Пассивные элементы | DC-блокер до 6 ГГц, SMA. |

## 1.2 Путь сигнала

Одна из частых ошибок при чтении блок-схем SDR — смешать радиосигнал, поток IQ-данных и управляющие команды. На разных участках тракта это разные физические величины. Настройка центральной частоты, переданная через драйвер, не является радиосигналом: она лишь меняет состояние синтезатора и аналогового тракта.

Укрупнённо приёмный и передающий пути от безразмерного комплексного отсчёта до электромагнитной волны и обратно изображены на рис. 1.1, а обозначения его блоков расшифрованы в таблице 1.2.

На приёме до АЦП сигнал — непрерывное физическое напряжение в фидерном и аналоговом трактах. АЦП формирует цифровые отсчёты двух квадратурных ветвей, после чего ПЛИС выполняет заложенную в её конфигурацию цифровую обработку и формирует поток. Через USB или Ethernet передаются уже цифровые данные, а libiio отдаёт приложению буферы отсчётов $I$ и $Q$; преобразование их в комплексный формат выполняет программа.

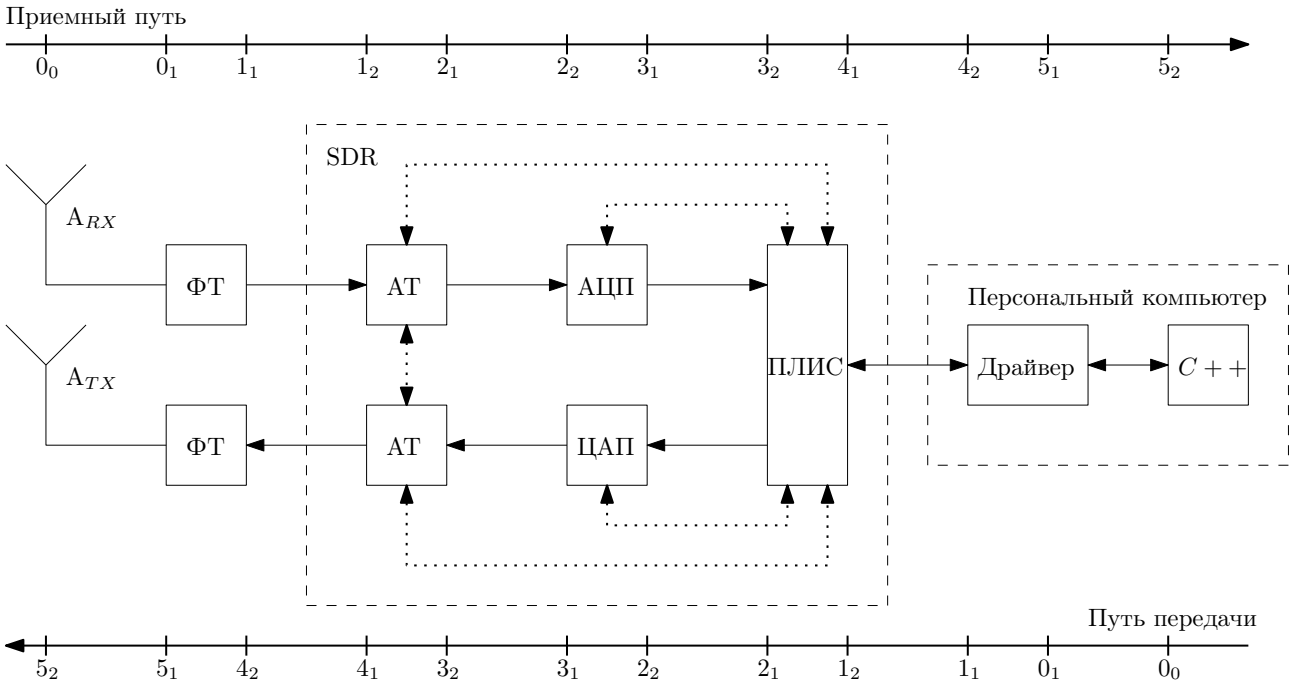



**Рис. 1.1:** Укрупнённая структурная схема приёмного и передающего путей сигнала.

**Таблица 1.2:** Блоки структурной схемы рис. 1.1

| Блок | Расшифровка | Назначение |
|---|---|---|
| $A_{RX}$ | Приёмная антенна | Преобразует падающие на неё свободные волны в направляемые волны фидера, подводящие принятую энергию ко входу приёмника. |
| $A_{TX}$ | Передающая антенна | Преобразует направляемые волны фидера в свободные волны, излучаемые в пространство. |
| ФТ | Фидерный тракт | Передаёт сигнал между антенной и приёмником или передатчиком, обеспечивает согласование и экранирование от помех. |
| АТ | Аналоговый тракт | Фильтрация, усиление (в том числе АРУ), перенос частоты и разделение на квадратурные составляющие. |
| АЦП | Аналого-цифровой преобразователь | Преобразует аналоговый сигнал в цифровые отсчёты. |
| ЦАП | Цифро-аналоговый преобразователь | Преобразует цифровые отсчёты в аналоговый сигнал. |
| ПЛИС | Программируемая логическая интегральная схема | Цифровая обработка сигнала и формирование потока IQ-данных. |
| Драйвер | Библиотека libiio | Программный интерфейс между прикладной программой и SDR: чтение и запись параметров устройства, обмен буферами отсчётов. |
| C++ | Прикладная программа | Вторичная обработка сигнала. |

Путь передачи проходит в обратном направлении: программа создаёт безразмерные комплексные отсчёты в установленном масштабе, после цифровой обработки они поступают в ЦАП, а полученный аналоговый сигнал переносится на центральную частоту и проходит через передающий тракт. Поэтому на рис. 1.1 участки передающего пути пронумерованы от программы к антенне, и одному и тому же физическому участку соответствуют номера $k$ на приёме и $5 - k$ на передаче (таблица 1.3).

**Таблица 1.3:** Участки приёмного и передающего путей и проверяемые параметры

| RX | TX | Что физически передаётся | Что задают или проверяют |
|---|---|---|---|
| $0_0 \to 0_1$ | $5_1 \to 5_2$ | Электромагнитная волна | Частота, полоса, мощность, поляризация, диаграмма направленности, усиление и КПД антенны. |
| $1_1 \to 1_2$ | $4_1 \to 4_2$ | Напряжение и ток в линии | Потери, согласование (КСВН), комплексный импеданс, допустимая мощность. |
| $2_1 \to 2_2$ | $3_1 \to 3_2$ | Аналоговые квадратурные сигналы | Центральная частота $f_0$, аналоговая полоса, усиление (МШУ и АРУ на приёме, ослабление на передаче); фактические значения читают обратно из устройства. |
| $3_1 \to 3_2$ | $2_1 \to 2_2$ | Цифровые отсчёты квадратурных компонент | Частота дискретизации, цифровые фильтры, формат внутренних данных. |
| $4_1 \to 4_2$ | $1_1 \to 1_2$ | Пакеты данных и управляющие сообщения | Пропускная способность интерфейса (USB, Ethernet), формат потока, переполнения на приёме и опустошения буфера на передаче. |
| $5_1 \to 5_2$ | $0_0 \to 0_1$ | Буферы IQ-отсчётов в памяти ПК | Формат отсчётов, размер буфера, обработка ошибок потока. |

Центральную частоту задаёт программа через драйвер, но физически её реализует аналоговый синтезатор платы. Частоту дискретизации программа тоже только запрашивает, а устройство выбирает достижимую конфигурацию тактирования и цифровых преобразований. Поэтому после каждой настройки приложение должно прочитать фактически установленное значение. Архитектура платы и последовательность команд драйвера рассматриваются в главе 3; здесь достаточно понимать границы участков.

**Контрольная точка**

Пусть программа принимает линию на смещении +200 кГц относительно центральной частоты. Изменение отображения FFT не перестраивает аналоговый гетеродин; изменение центральной частоты через драйвер — перестраивает. Если же поменять только подпись оси, ни физический

тракт, ни IQ-поток не изменятся. Для каждой настройки назовите блок, который она изменяет, и наблюдение, которым это подтверждается.

## 1.3 Уровни и защита входа

Расчёт начинается не со значения gain, а с мощности и точки тракта, к которой она относится. Одна и та же цифра без единицы может означать настройку усилителя, амплитуду комплексного отсчёта или мощность на разъёме; если их смешать, вывод о безопасности становится недоказуемым.

### 1.3.1 Разные шкалы уровня

Уровень в дБм — абсолютная мощность относительно 1 мВт:

$$P_{\text{dBm}} = 10\log_{10}\left(\frac{P}{1\ \text{мВт}}\right), \qquad P\,[\text{мВт}] = 10^{P_{\text{dBm}}/10}. \tag{1.1}$$

Например, 0 дБм соответствует 1 мВт, −10 дБм — 0,1 мВт, а −20 дБм — 0,01 мВт. Децибел без дополнительной буквы выражает отношение мощностей: ослабление $A = 20$ дБ уменьшает мощность в 100 раз и переводит 0 дБм в −20 дБм. Подробнее децибелы и цифровые уровни разобраны в разделе 4.6. Величины, которые нельзя подменять друг другом, сведены в таблицу 1.4.[1]

Паспортные границы для QPSK SDR Lab задаёт её радиочастотный трансивер семейства AD936x. По документации изготовителя [3][2]:

- максимальная выходная мощность передатчика — около +8 дБм на 800 МГц и +7,5 дБм на 2,4 ГГц при нулевом ослаблении; с ростом частоты она снижается;
- абсолютный максимум мощности на входе приёмника — +2,5 дБм, причём это *пиковое* значение.

Выходная мощность в документации — типовое значение, а не гарантированный максимум конкретного экземпляра. Поэтому в защитном бюджете берут верхнюю границу с запасом, $P_{\text{TX,max}} = +10$ дБм, если нет калиброванного измерения для данной платы и частоты. Предел приёмника задан для пиковой мощности: у модулированного сигнала с пик-фактором в несколько децибел средняя мощность должна быть ниже предела ещё на эту величину.

[1] Диапазоны gain в SoapySDR — по исходному коду модуля [1]; атрибуты `hardwaregain` и режимы АРУ — по документации драйвера AD9361 для Linux [2].

[2] О пиковом характере предела входа: [4].

**Таблица 1.4:** Величины, которые нельзя подменять друг другом

| Величина | Типичная запись | Что она означает |
|---|---|---|
| Мощность в точке RF-тракта | $P = -30$ дБм | Абсолютная средняя или пиковая мощность при явно указанных способе измерения и точке отсчёта. |
| Усиление или ослабление | $G = 12$ дБ, $A = 30$ дБ | Отношение уровней в двух точках. В этой главе ослабление $A$ записывается положительным числом и вычитается из бюджета тракта. |
| Параметр gain драйвера | например, `gain=20` | Управляющая настройка аналоговых каскадов, а не мощность на разъёме. Её шкала и даже знак зависят от драйвера. Так, модуль SoapySDR для плат на трансиверах AD936x задаёт TX gain от 0 до 89 дБ (чем больше, тем больше выходная мощность), а RX gain — от 0 до 73 дБ. В libiio тот же TX-параметр называется `hardwaregain` и принимает значения от $-89{,}75$ до 0 дБ, то есть по смыслу это ослабление: максимум мощности достигается при `hardwaregain=0`. RX-усиление в libiio задаётся вручную только при выключенной АРУ, а его диапазон зависит от частоты. |
| Цифровая амплитуда IQ | например, $\lvert x[n]\rvert = 0{,}2$ ($\approx -14$ dBFS) | Безразмерный масштаб отсчётов относительно принятой полной шкалы. Без калибровки не переводится однозначно в дБм. |
| Предел и рабочий уровень | $P_{\text{limit}} = +2{,}5$ дБм | Паспортный предел входной мощности — граница повреждения, а не уровень для нормальной работы. Порог компрессии и рабочий уровень лежат заметно ниже. |

Из этих двух чисел следует главное правило кабельного тракта. Передатчик на максимальной мощности, соединённый с приёмником кабелем напрямую, превышает абсолютный максимум входа примерно на 5–8 дБ. Значит, между TX и RX всегда стоит внешний аттенюатор, а установка ослабления программой защитой не считается: ошибка в коде, перезапуск или сброс настроек платы вернут передатчик к другому уровню.

Следует различать три уровня ограничения:

- *предел повреждения* задаёт область, в которую заходить нельзя;
- *насыщение или компрессия* начинается при существенно меньшем входном уровне и вызывает нелинейные и интермодуляционные искажения;
- *рабочий уровень* выбирают ещё ниже, с инженерным запасом на неопределённости и переходные процессы.

Предел +2,5 дБм защищает от повреждения, но не от перегрузки: линейность тракта проверяется отдельно в аппаратном опыте.

### 1.3.2 Бюджет закрытого тракта

Для закрытого (кабельного) тракта, в котором TX и RX соединены через пассивные элементы, все ослабления запишем положительными числами. Почти все слагаемые зависят от частоты, поэтому номинальный уровень на входе RX на центральной частоте $f_0$ равен

$$\begin{aligned} P_{\mathrm{RX,nom}}(f_0) = P_{\mathrm{TX,nom}}(f_0) - A_{\mathrm{att,nom}}(f_0) - A_{\mathrm{cable,nom}}(f_0) \\ - A_{\mathrm{other,nom}}(f_0) + G_{\mathrm{ext,nom}}(f_0). \end{aligned} \tag{1.2}$$

Здесь $A_{\mathrm{att}}$ — суммарное ослабление аттенюаторов, $A_{\mathrm{cable}}$ — потери кабелей, $A_{\mathrm{other}}$ — потери остальных пассивных элементов (переходников, DC-блокера), а $G_{\mathrm{ext}}$ — усиление внешних активных узлов, если они есть. Настройка RX gain в эту формулу не входит: $P_{\mathrm{RX}}$ определён на внешнем разъёме, до внутренних усилителей приёмника.

Номинальный расчёт описывает ожидаемый режим, но безопасность определяет наихудший случай, в котором все неопределённости выбраны в сторону увеличения мощности на RX:

$$\begin{aligned} P_{\mathrm{RX,worst}}(f_0) = P_{\mathrm{TX,max}}(f_0) - A_{\mathrm{att,min}}(f_0) - A_{\mathrm{cable,min}}(f_0) \\ - A_{\mathrm{other,min}}(f_0) + G_{\mathrm{ext,max}}(f_0). \end{aligned} \tag{1.3}$$

Минимальное ослабление находят с худшим знаком допуска: $A_{\mathrm{att,min}} =$

$A_{\text{att,nom}} - \Delta A^{-}_{\text{att}}$, где $\Delta A^{-}_{\text{att}} > 0$ — возможное уменьшение ослабления в гарантированном производителем диапазоне частот и температур. Неизвестному пассивному элементу назначают 0 дБ ослабления. Рассогласование также не считается защитой входа.

Запас до паспортного предела равен

$$M_{\text{safe}} = P_{\text{limit}} - P_{\text{RX,worst}}, \qquad P_{\text{limit}} = +2{,}5 \text{ дБм}. \tag{1.4}$$

При $M_{\text{safe}} \le 0$ подключение запрещено. Положительный результат показывает лишь расстояние до границы повреждения. Для решения *GO* заранее выбирают рабочий запас $M_{\text{req}} > 0$, учитывающий компрессию приёмника, и проверяют

$$P_{\text{RX,worst}} \le P_{\text{limit}} - M_{\text{req}}. \tag{1.5}$$

Число $M_{\text{req}}$ выбирают до сборки тракта, а не подгоняют под уже собранный.

**Таблица 1.5:** Проверка уровней перед физическим подключением

| Величина | Расчёт | Критерий |
|---|---|---|
| Номинальный уровень на RX | формула (1.2) | Для планирования измерения, но не для допуска. |
| Наихудший уровень на RX | формула (1.3) | Основа решения о допуске. |
| Запас до предела | формула (1.4) | $M_{\text{safe}} > 0$. |
| Запас до рабочего критерия | $M_{\text{safe}} - M_{\text{req}}$ | Не меньше нуля, иначе *NO-GO*. |
| Мощность на каждом аттенюаторе | Уровень перед этим элементом | Не выше его допустимой мощности. |

**Учебный пример.** Пусть на $f_0 = 2{,}4$ ГГц передатчик работает на $P_{\text{TX,nom}} = -10$ дБм, а его верхняя граница $P_{\text{TX,max}} = +10$ дБм. В тракт включены комплектные аттенюаторы 30 и 10 дБ ($A_{\text{att,nom}} = 40$ дБ, с учётом допусков $A_{\text{att,min}} = 39$ дБ), кабель ($A_{\text{cable,nom}} = 0{,}3$ дБ, $A_{\text{cable,min}} = 0{,}2$ дБ) и DC-блокер ($A_{\text{other,nom}} = 0{,}2$ дБ, подтверждённого минимума нет, $A_{\text{other,min}} = 0$). Внешних усилителей нет. Тогда

$$P_{\mathrm{RX,nom}} = -10 - 40 - 0{,}3 - 0{,}2 = -50{,}5 \text{ дБм}, \tag{1.6}$$

$$P_{\mathrm{RX,worst}} = 10 - 39 - 0{,}2 - 0 = -29{,}2 \text{ дБм}, \tag{1.7}$$

$$M_{\mathrm{safe}} = 2{,}5 - (-29{,}2) = 31{,}7 \text{ дБ}. \tag{1.8}$$

При выбранном $M_{\mathrm{req}} = 20$ дБ условие (1.5) выполнено с остатком 11,7 дБ. Первый аттенюатор со стороны передатчика принимает до +10 дБм (10 мВт); это значение сравнивают с его допустимой мощностью. Решение станет *GO* только после того, как все исходные числа подтверждены паспортами элементов или измерением.

**Контрольная точка**

Используйте $P_{\mathrm{TX,max}} = +10$ дБм, $P_{\mathrm{limit}} = +2{,}5$ дБм, $M_{\mathrm{req}} = 20$ дБ. Потерями кабеля и DC-блокера пренебрегите, допуск каждого аттенюатора примите равным $-1$ дБ.

1. Найдите $P_{\mathrm{RX,worst}}$ и $M_{\mathrm{safe}}$ для прямого кабеля без аттенюатора и для одного аттенюатора 6 дБ.
2. Какие комплектные аттенюаторы (один или два последовательно) с наименьшим суммарным номиналом удовлетворяют условию (1.5)?
3. Почему нельзя добиться того же, установив в программе ослабление TX 40 дБ?

Ответы: +10 дБм и −7,5 дБ; +5 дБм и −2,5 дБ — в обоих случаях *NO-GO*, вход может быть повреждён. Нужно $A_{\mathrm{att,min}} \geq 27{,}5$ дБ, то есть номинал не меньше 30 дБ. Подходят аттенюатор 30 дБ ($A_{\mathrm{att,min}} = 29$ дБ, остаток 1,5 дБ) и пара 20 + 10 дБ ($A_{\mathrm{att,min}} = 28$ дБ, остаток 0,5 дБ): допуски последовательных элементов складываются. Пара 20 + 6 дБ с минимумом 24 дБ не подходит. Программная настройка не является гарантированной верхней границей: при ошибке или сбросе передатчик вернётся к максимальной мощности.

### 1.3.3 Проверка до подачи сигнала

До любого разрешения на передачу выполняют следующий порядок:

1. зафиксировать частоту, полосу, выбранные каналы и порты по реальной плате;
2. получить для $P_{\mathrm{TX,max}}$ проверяемую верхнюю границу, а не пересчитывать

её из TX gain или амплитуды IQ;
3. проверить импеданс, механическую совместимость, диапазон частот и допустимую мощность каждого кабеля, аттенюатора и переходника;
4. вычислить номинальный и наихудший бюджеты, считая неизвестные пассивные потери равными нулю;
5. рассчитать мощность не только на входе RX, но и на входе каждого пассивного элемента;
6. при выключенном TX собрать пассивную цепь по схеме и проверить, что порты не перепутаны, а соединители не повреждены;
7. записать решение *GO/NO-GO* и его основание в паспорт стенда.

**Внимание**

Сначала собирают и проверяют пассивную цепь при неактивном передатчике. Пробная передача «на минимальном gain» не является способом измерить неизвестный бюджет и не используется для получения допуска.

## 1.4 Кабельный и эфирный сценарии

Оба сценария решают одну задачу — передать известный сигнал от TX к RX, — но проверяют разные части системы. Кабельный тракт лучше воспроизводит ослабление между разъёмами и отделяет радиотракт от геометрии антенн и внешних помех. Эфирный тракт включает антенны, поляризацию и распространение волны, поэтому требует больше параметров и, кроме того, учитывает влияние внешних источников.

### 1.4.1 Кабельный тракт

Обобщённая топология имеет вид

выбранный TX-порт → DC-блокер → аттенюатор(ы) → коаксиальный кабель → выбранный RX-порт.

Аттенюатор устанавливают в пассивную цепь до любого разрешения на передачу. Первый элемент со стороны передатчика должен выдерживать максимально возможную мощность в этой точке; следующие проверяют по уже

уменьшенному уровню. Разнесение суммарного ослабления между несколькими элементами не освобождает от проверки каждого из них. Разъёмы должны сопрягаться без механического усилия.

Паспорт кабельного сценария должен однозначно отвечать на вопросы:

1. какие TX- и RX-порты выбраны;
2. все ли элементы рассчитаны на 50 Ом и нужный диапазон частот;
3. чему равны $A_{\text{att,min}}$, $A_{\text{cable,min}}(f_0)$ и $A_{\text{other,min}}$;
4. выдерживает ли каждый элемент мощность в своей точке;
5. выполнено ли условие (1.5).

Кабель осматривают по всей длине: повреждённая оболочка, перегиб, люфт центрального контакта или загрязнение соединителя — основание для замены кабеля, а не для добавления предполагаемых потерь в расчёт.

### 1.4.2 Эфирный тракт в контролируемом месте

Эфирный сценарий допускается только в безэховой камере либо в другом явно разрешённом и контролируемом месте по утверждённому плану. Бытовое помещение и открытый эфир подходящей заменой не считаются. До опыта фиксируют разрешённые центральную частоту, полосу, верхнюю границу излучаемого уровня, длительность, расписание, контроль доступа и способ немедленного прекращения передачи. Если хотя бы одно из этих полей не утверждено, решение — *NO-GO*.

Передаваемый уровень выбирают минимально необходимым для поставленной задачи. Настройку TX gain и цифровую амплитуду записывают, но разрешённую мощность определяют не они, а проверенная верхняя граница на RF-разъёме и бюджет излучающего тракта.

Даже в идеальной модели потери распространения зависят от расстояния $R$ и длины волны $\lambda$. Для свободного пространства

$$L_{\text{FS}} = 20\log_{10}\left(\frac{4\pi R}{\lambda}\right), \qquad \lambda = \frac{c}{f_0}, \tag{1.9}$$

а уровень на выходе приёмной антенны укрупнённо равен

$$P_{\text{RX}} = P_{\text{TX}} + G_{\text{TX}} + G_{\text{RX}} - L_{\text{FS}} - L_{\text{pol}} - L_{\text{other}}, \tag{1.10}$$

где $G_{\text{TX}}$, $G_{\text{RX}}$ — коэффициенты усиления антенн, $L_{\text{pol}}$ — поляризационные потери, а $L_{\text{other}}$ — прочие потери. Формулы (1.9) и (1.10) описывают иде-

ализированную дальнюю зону и не заменяют калибровку камеры: реальные отражения, рассогласование и ориентация антенн меняют результат. Изменение RX gain при этом не меняет мощность, принятую антенной.

Для воспроизводимости фотографируют расположение оборудования и измеряют расстояние между антеннами, их высоты, азимут и угол места. Формулировки «антенны рядом» или «направлены друг на друга» недостаточны.

### 1.4.3 Выбор сценария

Сравнение сценариев приведено в таблице 1.6.

**Таблица 1.6:** Кабельный и эфирный сценарии

| Критерий | Кабельный тракт | Эфирный тракт |
|---|---|---|
| Назначение | Проверка TX/RX-тракта с контролируемым ослаблением | Проверка тракта вместе с антеннами и распространением |
| Топология | TX — DC-блокер — аттенюатор(ы) — кабель — RX | TX — антенна — пространство — антенна — RX |
| От чего зависит результат | Номиналы и допуски элементов, частота, состояние соединений | Дополнительно расстояние, высоты, углы, поляризация, отражения и внешние источники |
| Основные риски | Перегрузка RX; превышение мощности аттенюатора; повреждение разъёмов | Неразрешённое излучение; неверная геометрия; неучтённые отражения и помехи |
| Что записать в паспорт | Порты, элементы тракта и их ослабления, $P_{\mathrm{RX,worst}}$ и запас | Разрешённые частота, полоса, мощность и время; антенны и геометрия |

Если подготовлены оба сценария, для первой диагностики обычно выбирают кабельный: в нём меньше неизвестных. Эфирный сценарий не используют для обхода неготового расчёта: его бюджет и разрешение оформляют так же строго.

## 1.5 S11 и согласование антенны

Линия передачи рассчитана на опорный импеданс $Z_0 = 50$ Ом. Если входной импеданс нагрузки $Z_{\mathrm{L}}$ отличается от $Z_0$, часть падающей волны отражается. Комплексный коэффициент отражения в выбранной плоскости отсчёта равен

$$\Gamma = \frac{Z_{\mathrm{L}} - Z_0}{Z_{\mathrm{L}} + Z_0}. \tag{1.11}$$

В однопортовом измерении, откалиброванном в этой плоскости, $S_{11} = \Gamma$. Модуль $|\Gamma|$ равен отношению амплитуд отражённой и падающей волн, а аргумент — сдвигу фазы отражённой волны относительно падающей. По одному модулю восстановить комплексный импеданс нельзя.

В логарифмическом масштабе модуль $S_{11}$ записывают как

$$S_{11,\mathrm{dB}} = 20 \log_{10} |\Gamma|. \tag{1.12}$$

Для пассивной нагрузки $0 \le |\Gamma| \le 1$, поэтому $S_{11,\mathrm{dB}} \le 0$: чем более отрицательно число, тем меньше отражение. Возвратные потери (return loss) — та же величина с противоположным знаком:

$$RL = -20 \log_{10} |\Gamma| = -S_{11,\mathrm{dB}}. \tag{1.13}$$

Здесь $RL \ge 0$, и большее значение лучше. Записи «$S_{11} = -10$ дБ» и «возвратные потери 10 дБ» описывают одно и то же отражение, но смешивать знаки без определения нельзя.

Коэффициент стоячей волны по напряжению (КСВН, VSWR) выражается через тот же модуль:

$$\mathrm{VSWR} = \frac{1 + |\Gamma|}{1 - |\Gamma|}, \qquad 0 \le |\Gamma| < 1. \tag{1.14}$$

При идеальном согласовании $|\Gamma| = 0$, $S_{11,\mathrm{dB}} \to -\infty$, $RL \to +\infty$ и $\mathrm{VSWR} = 1$. При полном отражении $|\Gamma| \to 1$ и КСВН неограниченно возрастает.

Отношение отражённой мощности к падающей равно

$$\frac{P_{\mathrm{refl}}}{P_{\mathrm{inc}}} = |\Gamma|^2, \tag{1.15}$$

а доля мощности, поступившей в нагрузку, равна $1 - |\Gamma|^2$. Соответствующие потери рассогласования составляют

$$L_{\mathrm{mismatch}} = -10 \log_{10}\!\left(1 - |\Gamma|^2\right). \tag{1.16}$$

Мощность, поступившая в антенну, не обязана целиком превращаться в излучение: часть теряется в проводниках, диэлектрике и окружающих предметах. Поэтому хорошее $S_{11}$ доказывает только малое отражение в заданной плоскости и на заданной частоте. Оно не доказывает высокий КПД, известный коэффициент усиления, правильную поляризацию или подходящую диа-

грамму направленности.

**Контрольная точка**

1. КСВН антенны на рабочей частоте равен 2. Найдите $|\Gamma|$, долю отражённой мощности, $RL$ и $L_{\text{mismatch}}$.
2. Эту антенну подключили через кабель с потерями 3 дБ и измерили КСВН на входе кабеля. Какое значение получится? Почему кабель с потерями «улучшает» КСВН и почему радоваться этому не стоит?
3. * Подберите два разных активных сопротивления нагрузки с одинаковым $|\Gamma|$, чтобы показать, что по $|S_{11}|$ нельзя восстановить $Z_{\text{L}}$.

Ответы: $|\Gamma| = 1/3$, отражается 11,1% мощности, $RL \approx 9{,}5$ дБ, $L_{\text{mismatch}} \approx 0{,}51$ дБ. Отражённая волна проходит кабель дважды и ослабляется на 6 дБ: $RL \approx 15{,}5$ дБ, $|\Gamma_{\text{in}}| \approx 0{,}167$, КСВН $\approx 1{,}4$. Антенна при этом не изменилась, а до неё доходит вдвое меньше мощности. В задаче * подходят, например, 25 и 100 Ом: $\Gamma = \mp 1/3$.

**Внимание**

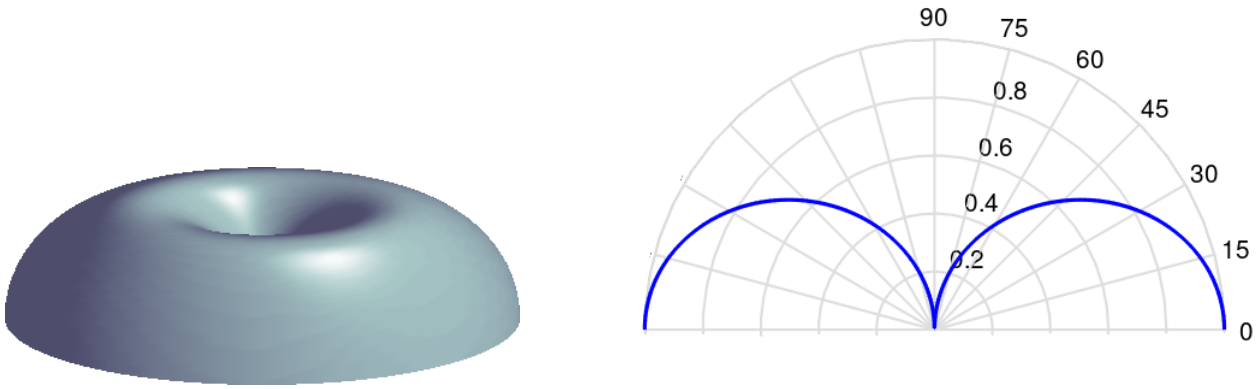
Потери рассогласования из формулы (1.16) не используют как защитное ослабление в бюджете раздела 1.3. Значение $S_{11}$ может измениться при перемещении кабеля, антенны и окружающих предметов; защита RX должна опираться на гарантированные элементы тракта.

## 1.6 Диаграмма направленности

*Диаграмма направленности* (ДН) антенны — графическое представление зависимости коэффициента усиления антенны или её коэффициента направленного действия от направления в заданной плоскости [5]. В общем случае это функция двух углов, то есть трёхмерная поверхность (рис. 1.2, а). Привычный полярный график показывает лишь одно её сечение, например при фиксированном азимуте или угле места (рис. 1.2, б); по одному сечению нельзя восстановить всё пространственное распределение.

Такая антенна излучает одинаково во всех азимутальных направлениях, но не излучает вдоль своей оси. Направленная антенна, наоборот, концентрирует энергию в узком секторе (рис. 1.3). Для чтения диаграммы достаточно различать следующие элементы:

- *главный лепесток* содержит направление максимума излучения;




а) б)

**Рис. 1.2:** ДН четвертьволнового штыря над идеально проводящей плоскостью: а) трёхмерная; б) сечение вертикальной плоскостью, по радиусу — относительная напряжённость поля, по окружности — угол места. Источник: Wikimedia Commons, автор Chetvorno, CC0: `https://commons.wikimedia.org/wiki/File:Monopole_3d_radiation_pattern_0.25_wavelength.png`, `https://commons.wikimedia.org/wiki/File:Monopole_radiation_pattern_0.25_wavelength.png`.

- *боковые* и *тыльный лепестки* соответствуют другим локальным максимумам;
- *нули* — направления сильного подавления между лепестками;
- *ширина луча* задаётся вместе с критерием, например между точками уровня $-3$ дБ относительно максимума.

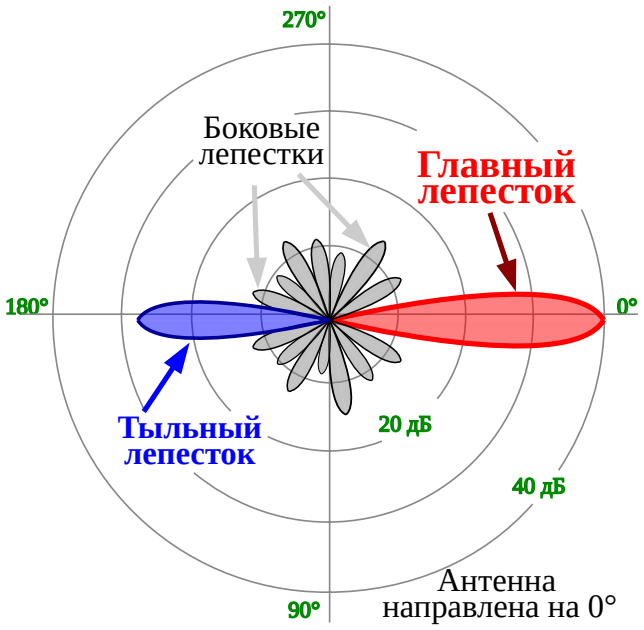



**Рис. 1.3:** Азимутальная ДН типичной направленной антенны, уровень в децибелах. Источник: Wikimedia Commons, авторы T. Truckle и Д. Ильин, CC0: `https://commons.wikimedia.org/wiki/File:Sidelobes-ru.svg`.

*Поляризация волны* — характеристика поперечных волн, описывающая поведение вектора колеблющейся величины в плоскости, перпендикулярной

направлению распространения [6]. Для радиоволн это вектор напряжённости электрического поля. При линейной поляризации он колеблется в одной плоскости, при круговой или эллиптической — его конец описывает окружность или эллипс. Даже если две антенны направлены максимумами друг на друга, несовпадение их поляризаций уменьшает принятый уровень.

Согласование и диаграмма направленности отвечают на разные вопросы. $S_{11}$ показывает долю отражённой мощности в порту, а ДН — угловое распределение излучаемой или принимаемой энергии. Антенна может быть хорошо согласована и при этом иметь неподходящее направление максимума, и наоборот: ни одну из характеристик нельзя вывести из другой.

ДН комплектных антенн в этой работе не измеряется. Для повторяемости эфирного опыта достаточно зафиксировать геометрию антенн стенда:

- точки, между которыми измерено расстояние;
- высоты и координаты креплений;
- азимут и угол места каждой антенны;
- тип и ориентацию поляризации каждой антенны, например «штырь вертикально»;
- положение кабелей, корпуса платы и крупных окружающих объектов;
- фотографию общего вида с видимой угловой разметкой.

**Важно**

Снижение принятого уровня после поворота антенны не доказывает, что найден нуль диаграммы. Причиной могут быть изменение поляризации, изгиб кабеля, смещение антенны или отражения. Корректный вывод: «уровень изменился при зафиксированном изменении геометрии».

## 1.7 Паспорт стенда

Паспорт связывает расчёт, физическую сборку и программную конфигурацию. Его заполняют один раз: при сборке нового стенда или перед крупным опытом — первой передачей по кабелю или эфирным сценарием. При отладке программ и повторных запусках на уже описанном стенде паспорт не переписывают: достаточно сослаться на него и записать, что изменилось. Форма (таблица 1.7) подходит для обоих сценариев.

**Таблица 1.7:** Паспорт стенда

| Поле | Значение и основание |
|---|---|
| **1. Идентификация** | |
| Дата, место, исполнитель | ..... |
| QPSK SDR Lab: серийный номер, версия прошивки | ..... |
| Выбранные TX/RX-каналы | ..... |
| ОС, версия libiio, commit репозитория | ..... |
| **2. Сценарий** | |
| Сценарий | ☐ кабельный ☐ эфирный |
| Фотография и структурная схема стенда | ..... |
| **3. Радиопараметры и бюджет** | |
| $f_0$, $F_s$, аналоговая полоса | ..... |
| TX gain, RX gain и режим АРУ | ..... дБ |
| Цифровая амплитуда | ..... |
| $P_{\mathrm{TX,max}}$ и основание | ..... дБм |
| $A_{\mathrm{att,min}}$, $A_{\mathrm{cable,min}}(f_0)$, $A_{\mathrm{other,min}}$ | ..... дБ |
| $P_{\mathrm{RX,nom}}$, $P_{\mathrm{RX,worst}}$ | ..... дБм |
| $M_{\mathrm{safe}}$ и выбранный $M_{\mathrm{req}}$ | ..... дБ |
| **4. Разрешение эфирного сценария** | |
| Разрешённые частота, полоса, мощность | ..... |
| Разрешённые время и длительность | ..... |
| Контроль доступа и условие остановки | ..... |
| **5. Результаты** | |
| Фотографии стенда | ..... |
| Команды, журналы и конфигурационные файлы | ..... |
| IQ-файлы | ..... |
| **6. Решение** | |
| До подачи сигнала | ☐ **GO** ☐ **NO-GO**; основание: ..... |

**Контрольная точка**

**Готовность стенда к первой передаче.**

- ☐ Состав комплекта и порты сверены с реальной платой.
- ☐ Пассивная цепь собрана по схеме при выключенном TX.
- ☐ $P_{\mathrm{RX,worst}}$ рассчитан, условие (1.5) выполнено, допустимые мощности элементов проверены.
- ☐ Для эфирного сценария получено разрешение.
- ☐ Паспорт заполнен, решение *GO* записано.

Таким образом, результат главы — не включённый передатчик, а описанный стенд: каждый элемент известен, путь сигнала понятен, наихудший уровень на входе RX рассчитан, и решение о подключении может проверить любой другой участник.

# 2 Воспроизводимая программная среда, Git и сборка

В предыдущей главе был подготовлен физически безопасный стенд. Теперь нужно подготовить второй компонент эксперимента — программную среду. Её нельзя считать готовой только потому, что компилятор однажды выдал работающий исполняемый файл: для повторения результата должны быть известны версия исходного кода, состав инструментов, параметры сборки и состояние подключённого оборудования.

Сначала без SDR-платы собираются и запускаются минимальные программы на C++17. Затем устанавливается соединение с платой QPSK SDR Lab и проверяется наличие двух каналов приёма и двух каналов передачи. Запуск потоков IQ и радиочастотные испытания относятся к следующей главе. Такое разделение позволяет отличить ошибки сборки от проблем подключения и прошивки.

После выполнения главы читатель должен уметь:

- объяснить, как из исходного текста получается программа: компиляция, компоновка, библиотеки и роль CMake;
- ориентироваться в каталогах Linux, выполнять команды в терминале и пользоваться переменными окружения;
- получить учебный репозиторий и определить версию его исходного кода;
- вручную подобрать пакеты для другого Linux-дистрибутива, если таковой используется;
- собрать программы, libiio и libad9361-iio в локальном каталоге проекта;
- получить контрольный IQ-файл с известными параметрами;
- проверить подключение QPSK SDR Lab и наличие каналов 2RX2TX;
- запускать автоматические тесты и понимать, что именно они проверяют;
- оформить паспорт программной среды.

## 2.1 Как из исходного кода получается программа

Программа на C++ начинается с исходного текста — обычных текстовых файлов. Файлы `.cpp` содержат определения функций, а заголовочные файлы `.hpp` и `.h` — объявления, которые подключаются директивой `#include`. Процессор такой текст не выполняет: его нужно перевести в машинный код. Этот перевод выполняется в несколько шагов (рис. 2.1).

*Компилятор* (`g++`, `clang++`) переводит один файл `.cpp` вместе со всеми подключёнными заголовками в *объектный файл* `.o`. В объектном файле лежит машинный код функций этого файла и список имён (символов), которые файл использует, но сам не определяет. Например, `qpsk_channels.cpp` вызывает функцию `iio_create_context_from_uri()`, но её кода в этом файле нет: компилятору достаточно объявления из `iio.h`.

*Компоновщик* (линкер, `ld`; обычно его вызывает тот же `g++`) собирает объектные файлы и библиотеки в исполняемый файл и проверяет, что для каждого использованного имени нашлось определение. Сообщение `undefined reference to ...` выдаёт именно компоновщик: объявление было, а определения он не нашёл — чаще всего не подключена нужная библиотека.

*Библиотека* — заранее скомпилированный код для повторного использования. *Статическая* библиотека (`.a`) при компоновке копируется внутрь программы. *Разделяемая* (`.so`, shared object) не копируется: в программе записывается только её имя, а саму библиотеку при каждом запуске загружает загрузчик операционной системы (раздел 2.3.2). libiio и libad9361-iio в этом проекте используются как разделяемые.

Таким образом, библиотека нужна дважды: её заголовки — при компиляции, а сам файл библиотеки — при компоновке и затем при каждом запуске. Поэтому в разделе 2.2 устанавливаются пакеты с суффиксом `-devel` (RED OS) или `-dev` (Debian): в них лежат заголовки и файлы для компоновки, которых нет в обычных пакетах, нужных только для запуска готовых программ.

Для проекта из десятков файлов набирать команды компилятора вручную неудобно, а перекомпилировать всё после каждой правки долго. Эту работу выполняют сборщики. *Make* читает файл правил `Makefile`: какой файл из каких получается и какой командой; при повторном запуске он выполняет только те команды, чьи исходные файлы изменились. *Ninja* делает то же, только быстрее.

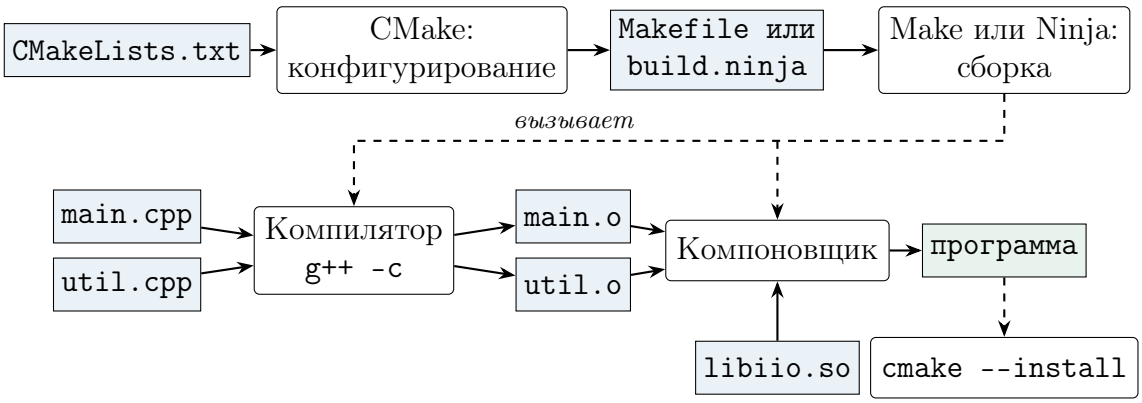



**Рис. 2.1:** Путь от исходного текста к программе. Сплошные стрелки — движение файлов, пунктир — управление: Make или Ninja запускают компилятор и компоновщик по правилам, которые сгенерировал CMake; `cmake --install` копирует готовые файлы в каталог установки.

*CMake* — не компилятор и не сборщик, а генератор правил сборки. В файле `CMakeLists.txt` описано, какие программы из каких исходников собрать и с какими библиотеками. По этому описанию CMake находит компилятор, заголовки и библиотеки на конкретном компьютере и записывает правила для Make или Ninja. Поэтому один и тот же `CMakeLists.txt` работает и в RED OS, и в Debian. Работа с CMake состоит из трёх шагов:

1. *конфигурирование* `cmake -S исходники -B каталог_сборки`: проверка среды и генерация правил. Параметры вида `-ДИМЯ=значение` сохраняются в файле `CMakeCache.txt` каталога сборки;
2. *сборка* `cmake --build каталог_сборки`: CMake запускает Make или Ninja, а те — компилятор и компоновщик;
3. *установка* `cmake --install каталог_сборки`, необязательная: копирование готовых программ, библиотек и заголовков в каталог установки (раздел 2.7).

**Контрольная точка**

1. Что делает компилятор, а что — компоновщик? Какой из них выдаст ошибку `undefined reference to 'iio_create_context_from_uri'` и что, скорее всего, забыли?
2. Почему CMake нельзя назвать компилятором?
3. Зачем устанавливать пакет `libusb1-devel`, если библиотека libusb в системе уже есть?

Ответы для сверки: компилятор переводит один `.cpp` в объектный файл, компоновщик собирает объектные файлы и библиотеки в программу.

Ошибку выдаёт компоновщик; не подключена библиотека libiio. CMake не переводит код в машинные команды: он только находит инструменты и библиотеки и пишет правила, по которым компилятор и компоновщик запускает сборщик. В пакете `-devel` лежат заголовки и файлы для компоновки, без которых libusb нельзя использовать в собственной сборке.

## 2.2 Каноническая и альтернативные среды

Канонической средой методического пособия является 64-разрядная RED OS 8. В ней авторы проверяют названия пакетов, команды и работу с оборудованием. Это не означает, что практикум привязан только к RED OS: Debian, Ubuntu и другие современные Linux-дистрибутивы допустимы, если участник самостоятельно устанавливает эквивалентные пакеты и обязательно фиксирует отличия в паспорте среды.

Автоматический скрипт установки намеренно не используется. Имена пакетов, состав репозиториев и политика установки различаются между дистрибутивами и их версиями. Системные инструменты устанавливаются штатным менеджером пакетов, а libiio собирается из исходников, включённых в учебный репозиторий.

Сначала нужно определить фактическую систему:

```
cat /etc/os-release
uname -m
uname -r
```

Канонические примеры проверяются на архитектуре `x86_64`. Если вывод отличается, это не обязательно делает работу невозможной, но полученная конфигурация должна быть проверена отдельно.

Базовый набор для этой главы:

- Git — получение исходного кода и контроль версий;
- компиляторы C и C++ с поддержкой C++17;
- CMake не ниже 3.20, Make и pkg-config — настройка и выполнение сборки;
- libusb и libxml2 с заголовочными файлами — зависимости libiio;
- утилиты USB — для диагностики подключения платы.

На RED OS перед установкой полезно проверить наличие и описание каждого пакета. Основные команды управления пакетами приведены в документации RED OS [7].

```
dnf info git cmake gcc gcc-c++ make pkgconf-pkg-config
```

```
dnf info libusb1-devel libxml2-devel usbutils
sudo dnf install git cmake gcc gcc-c++ make pkgconf-pkg-config
sudo dnf install libusb1-devel libxml2-devel usbutils
```

В Debian/Ubuntu тем же ролям обычно соответствуют следующие пакеты:

```
sudo apt update
sudo apt install git cmake build-essential pkg-config
sudo apt install libusb-1.0-0-dev libxml2-dev usbutils
```

Это пример соответствия, а не универсальная команда для любого дистрибутива. Перед установкой следует проверить кандидата командой `apt show` или `dnf info`. Если пакет отсутствует, найдите его эквивалент в репозиториях своей ОС. FFTW3, OpenCV и ZeroMQ устанавливаются по мере перехода к использующим их программам; для примеров этой главы они не требуются.

**Контрольная точка**

1. Запишите название и версию своей ОС, архитектуру и версию ядра. Совпадают ли они с канонической средой?
2. Как называется пакет с заголовочными файлами libusb в RED OS и в Debian? Как найти его эквивалент в другом дистрибутиве?
3. Почему в методическом пособии нет общего скрипта, устанавливающего все пакеты одной командой?

Ответы для сверки: `libusb1-devel` и `libusb-1.0-0-dev`; эквивалент ищут поиском менеджера пакетов (`dnf search`, `apt search`) и проверяют командой `dnf info` или `apt show`. Имена пакетов и состав репозиториев различаются между дистрибутивами и их версиями, поэтому общий скрипт скрывал бы эти различия и ломался бы на непроверенных системах.

## 2.3 Основы файловой системы Linux и работа в терминале

В Linux файлы организованы в дерево каталогов. Путь, начинающийся с символа `/`, отсчитывается от корня этого дерева; это абсолютный путь. Относительный путь отсчитывается от текущего каталога терминала. Обозначение `.` означает текущий каталог, `..` — родительский, а `~` — домашний каталог

пользователя. Регистр важен: `Data` и `data` — разные имена.

Терминал — окно для ввода команд, а командная оболочка, например Bash, разбирает и выполняет их. Команда обычно состоит из имени программы, параметров и аргументов. В `ls -l build` имя программы — `ls`, параметр `-l` включает подробный вывод, а `build` указывает каталог. Приглашение вида `[user@host project]$` копировать в команду не нужно.

```
pwd
ls
ls -la
mkdir -p ~/projects
cd ~/projects
cd ..
cd ~/projects
```

Команда `pwd` печатает текущий путь, `ls` показывает содержимое, `mkdir -p` создаёт каталог и недостающие родительские каталоги, `cd` меняет текущий каталог. Флаг `-a` показывает скрытые имена, начинающиеся с точки, а `-l` — подробные сведения о файлах. Пути с пробелами заключают в кавычки. Клавиша Tab дополняет имя команды или файла, стрелка вверх возвращает предыдущую команду, Ctrl+C прерывает текущую программу. В справке `man ls` выход выполняется клавишей `q`.

Программа по полному пути запускается непосредственно. Запись `./build/ch02/env_report` означает запуск файла относительно текущего каталога. Для запуска из произвольного места оболочка ищет имя программы в каталогах переменной `PATH`. Расширение `.cpp` обозначает исходный текст, а не готовую программу: перед запуском его нужно скомпилировать.

`sudo` запускает команду с административными правами; при вводе пароля символы в терминале обычно не отображаются. Здесь эти права нужны для установки системных пакетов, но не для сборки C++-проекта.

Версии инструментов проверяют непосредственно:

```
git --version
cmake --version
c++ --version
```

Чтобы сохранить вывод уже знакомой команды, используют `tee`:

```
cmake --version | tee cmake-version.log
```

Символ `|` передаёт вывод предыдущей программы следующей, а `tee` одновременно показывает его и записывает в файл. Простой запуск выше перезаписывает одноимённый журнал; ключ `-a` дописывает в конец. Для полноценного журнала сборки, включая сообщения об ошибках, пригодится:

```
set -o pipefail
cmake --build build/ch02 --parallel 2>&1 | tee build/ch02/build.log
```

Эту команду выполняют после конфигурирования проекта в разделе 2.6. Запись `2>&1` объединяет поток ошибок с обычным выводом, а `pipefail` не позволяет успешному `tee` скрыть ошибку сборки. Обратная косая черта в конце строки длинной команды означает продолжение на следующей строке; пробел после неё ставить нельзя.

**Внимание**

Не публикуйте полный вывод `env`, историю терминала и файлы с паролями, токенами или приватными ключами. Для отчёта достаточно версий инструментов, команд сборки и относящихся к эксперименту сообщений.

**Контрольная точка**

1. Текущий каталог — `~/projects/LETI_MIR_accelerator/build`. Где вы окажетесь после `cd ../..`? Абсолютный или относительный путь `../examples`?
2. Почему собранную программу запускают как `./build/ch02/env_report`, а Git — просто командой `git`?
3. Сборка завершилась ошибкой. Что окажется в файле журнала после `cmake --build build/ch02 | tee build.log` и какой код возврата увидит оболочка? Что изменят `2>&1` и `set -o pipefail`?

Ответы для сверки: в `~/projects`; путь относительный. Каталог `/usr/bin` с программой `git` входит в `PATH`, а каталог сборки — нет. Без `2>&1` сообщения об ошибках идут в поток ошибок и в журнал не попадают; без `pipefail` оболочка получит код возврата `tee`, равный нулю, и ошибка сборки останется незамеченной.

### 2.3.1 Переменные окружения

Каждая запущенная программа получает от запустившей её оболочки набор *переменных окружения* — пар «имя = значение». Через них программам передают общие настройки: домашний каталог `HOME`, текущий каталог `PWD`, список каталогов для поиска команд `PATH`, язык интерфейса `LANG`. В команде значение переменной подставляется знаком `$`:

```
echo $HOME
```

```
echo $PATH
printenv LD_LIBRARY_PATH
```

`PATH` — список каталогов через двоеточие; команда `type git` покажет, в каком из них оболочка нашла программу. Если переменная не задана, `printenv` ничего не выведет. Запись `"$PWD/build/iio-install"` превращается в абсолютный путь к каталогу внутри проекта; кавычки защищают путь, если в нём есть пробелы.

Задать переменную можно тремя способами, и они различаются тем, на кого действует новое значение (таблица 2.1).

**Таблица 2.1:** Способы задать переменную окружения

| Запись | На кого действует |
|---|---|
| `ИМЯ=значение команда` | Только на один запуск этой команды. Оболочка и следующие команды значения не видят. Так в пособии задаётся `LD_LIBRARY_PATH`. |
| `export ИМЯ=значение` | На текущее окно терминала и на все программы, запущенные из него после этой команды. После закрытия окна значение пропадает; удалить его раньше можно командой `unset ИМЯ`. |
| Строка `export` в файле `~/.bashrc` | На все новые окна терминала этого пользователя. В проекте так не делают: настройка начинает влиять на все программы, в том числе на чужие проекты, и о ней легко забыть. |

Разницу легко проверить. Команда `sh -c '...'` запускает новую оболочку, то есть дочернюю программу:

```
MYVAR=hello sh -c 'echo "child: $MYVAR"'   # child: hello
echo "shell: $MYVAR"                        # shell: (пусто)
export MYVAR=hello
sh -c 'echo "child: $MYVAR"'               # child: hello
unset MYVAR
```

Дочерняя программа получает *копию* окружения родителя. Она может изменить свою копию, но не окружение оболочки, которая её запустила.

### 2.3.2 Как программа находит разделяемые библиотеки

Программа, собранная с разделяемыми библиотеками, не содержит их кода. При запуске сначала работает *загрузчик* операционной системы (в Linux —

`ld-linux`). Он читает из исполняемого файла список нужных библиотек (записи `NEEDED`) и ищет каждую библиотеку в каталогах в строго определённом порядке (рис. 2.2):

1. каталоги из переменной окружения `LD_LIBRARY_PATH`;
2. каталоги `RUNPATH`, записанные в саму программу при компоновке;
3. системный кэш, который строит команда `ldconfig` по файлам `/etc/ld.so.conf` и `/etc/ld.so.conf.d/`;
4. стандартные каталоги системы, например `/lib64` и `/usr/lib64`.

Используется первая найденная библиотека с подходящим именем.[1] Если на компьютере несколько версий libiio — например, пакет операционной системы в `/lib64` и наша сборка в `build/iio-install/lib`, — программа загрузит ту, что встретится раньше. Ошибиться здесь легко: программа собрана с одной версией, а запускается с другой.

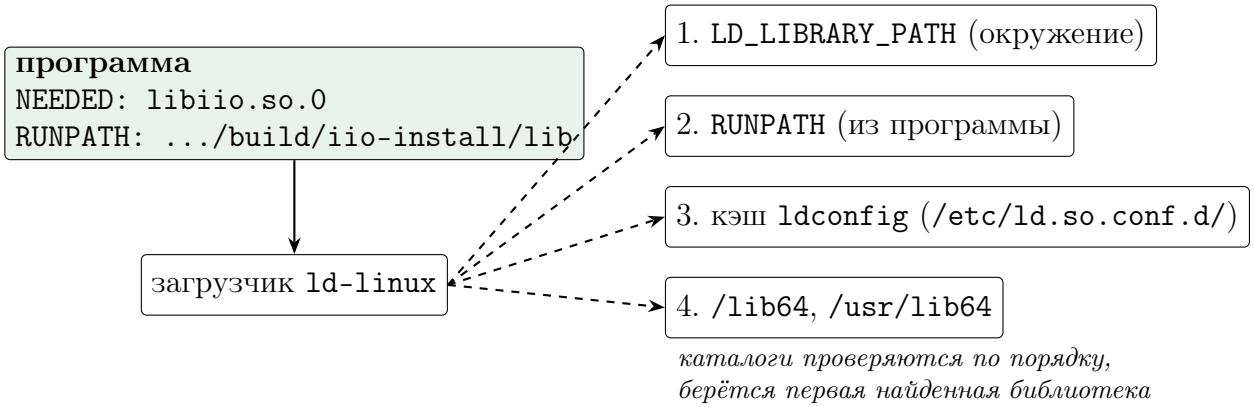



**Рис. 2.2:** Порядок поиска разделяемой библиотеки при запуске программы.

Посмотреть, что записано в программе и какие библиотеки она фактически загрузит, позволяют три команды. Они понадобятся в разделе 2.7, когда появится программа, использующая libiio:

```
readelf -d build/ch02-iio/qpsk_channels | grep -E 'NEEDED|RUNPATH'
ldd build/ch02-iio/qpsk_channels
ldconfig -p | grep libiio
```

`readelf` показывает записи внутри файла, `ldd` — результат поиска для текущего окружения, а `ldconfig -p` — содержимое системного кэша, то есть все libiio, известные системе. В этом проекте путь к локальной libiio записывается в программы как `RUNPATH`, поэтому при пустой `LD_LIBRARY_PATH` загружается именно она. Но `LD_LIBRARY_PATH` проверяется раньше: если в

[1] Устаревшая запись `RPATH` проверяется даже раньше `LD_LIBRARY_PATH`, но только если в файле нет `RUNPATH`. Современные компоновщики записывают `RUNPATH`. Подробности: `man ld.so`.

`~/.bashrc` она указывает на каталог с другой libiio, загрузится та. Поэтому в командах пособия переменная ещё и задаётся явно, на один запуск.

**Контрольная точка**

1. Выполните `export DEMO=1`, откройте новое окно терминала и проверьте `echo $DEMO`. Почему значение пропало?
2. В программе записан `RUNPATH` на `build/iio-install/lib`, `LD_LIBRARY_PATH` пуста, а в `/lib64` есть другая `libiio.so.0`. Какая библиотека будет загружена?
3. * В `~/.bashrc` есть строка `export LD_LIBRARY_PATH=/opt/old/lib`, и в этом каталоге лежит старая libiio. Что произойдёт при запуске программы без явной переменной и как это обнаружить?

Ответы для сверки: `export` действует только на текущее окно и запущенные из него программы. Загрузится библиотека из `RUNPATH`: этот каталог проверяется раньше системных. В последнем случае первой будет найдена старая libiio из `/opt/old/lib`; это видно в выводе `ldd`, где путь к `libiio.so.0` укажет на `/opt/old/lib`.

## 2.4 Git как фиксация эксперимента

Git хранит историю файлов проекта. *Репозиторий* — каталог проекта вместе с историей в скрытом каталоге `.git`. *Commit* (коммит) — зафиксированное состояние отслеживаемых файлов с комментарием; его хеш служит идентификатором версии. *Ветвь* (branch) — именованная линия разработки, например `main`. Имя ветви остаётся тем же, а её текущий commit меняется по мере работы.

GitHub хранит удалённую копию репозитория, но не заменяет Git на компьютере. Учебный код доступен по адресу `https://github.com/MCHelectronics/LETI_MIR_accelerator`. Чтобы впервые получить его:

```
cd ~/projects
git clone https://github.com/MCHelectronics/LETI_MIR_accelerator.git
cd LETI_MIR_accelerator
git status
git log -1 --oneline
git rev-parse HEAD
```

`clone` создаёт локальную копию и настраивает связь с удалённым репозиторием, обычно под именем `origin`. Для чтения публичного репозитория

учётная запись GitHub не требуется. `status` показывает состояние рабочей копии, `log` — историю, `rev-parse HEAD` — полный хеш текущего commit. Для повторения опыта фиксируют хеш, а не только название ветви.

Изменение файла в редакторе ещё не создаёт commit. Перед фиксацией просматривают изменения командой `git diff`, затем выбирают конкретные файлы командой `git add`. Например, после собственной правки `examples/ch02/iq_tone.cpp`:

```
git switch -c my-experiments
git diff
git add examples/ch02/iq_tone.cpp
git commit -m "Describe my tone experiment"
```

При первом commit Git может попросить имя и почту автора. Их можно задать командами `git config user.name` и `git config user.email`; без параметра `--global` настройка действует только в текущем репозитории. Используйте допустимые для публикации данные. Локальный commit не отправляет файлы в Интернет. Команда `push` отправляет их на сервер и требует права записи; у студента такие права в общем репозитории отсутствуют.

Для обновления `main` сначала сохраните свою работу и убедитесь, что `git status` не показывает несохранённых изменений:

```
git switch main
git pull --ff-only
```

Если Git отказал из-за расхождения истории или локальных правок, не применяйте принудительный сброс. Сначала сохраните полезную работу в отдельном commit или отдельно сохраните копию изменённых файлов. `.gitignore` исключает из обычного добавления результаты сборки: готовый бинарник и исходный код — разные результаты работы.

**Важно**

Одинаковый исходный commit не гарантирует одинаковую сборку при разных версиях библиотек и параметрах CMake. Поэтому вместе с хешем сохраняют команду конфигурирования. Если `git status --short` не пуст, опыт относится к commit вместе с локальными изменениями.

## 2.5 Репозиторий и навигация по коду

**Таблица 2.2:** Структура учебного проекта

| Путь | Назначение |
|---|---|
| `README.md` | Точка входа: порядок работы и команды по главам. |
| `CMakeLists.txt` | Описание сборки: программы, библиотеки и автоматические тесты. |
| `examples/ch02/` | Программы этой главы: отчёт о компиляторе, генератор IQ-тона и проверка каналов платы. |
| `examples/ch03/` | Программы главы 3: снимок состояния платы и измерение предела потока. |
| `examples/ch05/` | Программы главы 5: приём в файл, спектр и водопад, FM-демодулятор. |
| `examples/ch06/` | Программы главы 6: генератор тестовых сигналов и передача. |
| `include/lab/` | Заголовки общего кода: IQ-файлы с метаданными, ЦОС, построение графиков. |
| `src/` | Реализация общего кода из `include/lab/`. |
| `tests/` | Сценарии и данные автоматических тестов, в том числе XML-описания платы для проверок без оборудования (раздел 2.8). |
| `vendor/libiio/` | Зафиксированные исходники libiio с лицензиями. |
| `vendor/libad9361-iio/` | Зафиксированные исходники libad9361-iio с лицензией. |
| `vendor/README.md` | Источники, версии и commit зависимостей. |
| `docs/manual.pdf` | Методическое пособие. |
| `docs/*.md` | Инструкции сборки и подключения. |
| `docs/img/` | Рисунки и записи опытов в полном размере. |
| `build/` | Локальные сборки, библиотеки, IQ-файлы и журналы; не передаются через Git. |

Прошивка платы не скачивается и не обновляется при сборке проекта. Страницу репозитория и актуальную версию пособия удобно держать открытыми на телефоне, пока команды выполняются на компьютере:

**Учебный репозиторий: README и инструкции**
`https://github.com/MCHelectronics/LETI_MIR_accelerator`

**Методическое пособие в репозитории**
`https://github.com/MCHelectronics/LETI_MIR_accelerator/blob/main/docs/manual.pdf`

**Контрольная точка**

1. Опыт выполнен, а `git status --short` показывает изменённый `examples/ch02/iq_tone.cpp`. Достаточно ли записать в паспорт хеш commit?
2. Чем `git commit` отличается от `git push`?
3. Где лежат исходный текст генератора тона и его исполняемый файл после сборки? Почему каталог `build/` не отслеживается Git?

Ответы для сверки: нет — результат относится к commit вместе с локальными изменениями, поэтому их сначала фиксируют отдельным commit. `commit` сохраняет состояние в локальном репозитории, `push` отправляет его на сервер. Исходник — `examples/ch02/iq_tone.cpp`, программа — `build/ch02/iq_tone`; результаты сборки заново получаются из исходников и зависят от компьютера, поэтому в Git их не хранят.

## 2.6 Сборка программ в каталоге проекта

Прежде чем поручить сборку CMake, соберём самую маленькую программу вручную, чтобы увидеть оба шага из раздела 2.1. Команды выполняются из корня учебного репозитория:

```
mkdir -p build/manual
c++ -std=c++17 -c examples/ch02/env_report.cpp -o build/manual/env_report.o
nm -C build/manual/env_report.o | grep " U "
c++ build/manual/env_report.o -o build/manual/env_report
./build/manual/env_report
```

Ключ `-c` означает «только компилировать»: результат первой команды — объектный файл `env_report.o`. Команда `nm` перечисляет имена в нём; строки с буквой `U` (undefined) — это функции стандартной библиотеки C++, которые программа использует, но не содержит, например операторы вывода `std::ostream`. Вторая команда `c++` без ключа `-c` вызывает компоновщик: он

находит эти функции в стандартной библиотеке и создаёт исполняемый файл. Программа печатает версию стандарта C++ и компилятора.

Для программы с libiio пришлось бы ещё указать компилятору, где искать `iio.h` (`-I.../include`), а компоновщику — где искать библиотеку и какую именно (`-L.../lib -liio`). Писать такие команды для каждого файла незачем: это делает CMake по описанию из `CMakeLists.txt`. Исходники остаются на месте, результаты сборки складываются в каталог `build/`.

Первую сборку через CMake выполняют без платы и libiio:

```
cmake -S . -B build/ch02 -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build/ch02 --parallel
ctest --test-dir build/ch02 --output-on-failure
./build/ch02/env_report
```

Параметр `-S .` указывает каталог с исходниками (точка — текущий каталог), `-B` — каталог сборки, а `-D` задаёт параметр CMake. `CMAKE_BUILD_TYPE` выбирает режим: `Debug` — без оптимизации и с отладочной информацией, `Release` — с оптимизацией, `RelWithDebInfo` — с оптимизацией и отладочной информацией одновременно. Ключ `--parallel` разрешает компилировать несколько файлов одновременно. Третья команда запускает автоматические тесты проекта; ожидается шесть успешных. Как они устроены, разобрано в разделе 2.8. В отчёте `env_report` значение стандарта C++ должно быть не меньше `201703`.

Сформируем 10 мс комплексного тона с частотой дискретизации 1 МГц, частотой 100 кГц и амплитудой 0,5:

```
mkdir -p build/ch02/results
./build/ch02/iq_tone build/ch02/results/tone.cf32 \
  1000000 100000 0.01 0.5
wc -c build/ch02/results/tone.cf32
cat build/ch02/results/tone.cf32.json
```

Аргументы после имени файла задают $F_s$, частоту тона, длительность и амплитуду. Программа не перезаписывает существующий результат: для нового опыта укажите другое имя. Файл содержит $N = F_s T = 10\,000$ комплексных отсчётов. Каждый записан как два числа IEEE 754 float32, сначала $I$, затем $Q$, с порядком байтов little-endian. Его размер равен

$$L = N \cdot 2 \cdot 4 = 80\,000 \text{ байт}. \tag{2.1}$$

Соседний JSON-файл содержит формат, частоту дискретизации, частоту тона, амплитуду и число отсчётов. Для произвольной длительности число

отсчётов округляется до ближайшего целого; JSON сохраняет и запрошенную, и фактическую длительность $N/F_s$. Расширение файла само по себе этих сведений не задаёт.

**Контрольная точка**

Сравните расчёт размера с фактическими 80 000 байтами, проверьте JSON и результат CTest. Затем сформируйте тон с отрицательной частотой и объясните, какая из компонент $I$, $Q$ изменится относительно положительной частоты при той же начальной фазе.

* Создайте вторую сборку в `build/ch02-debug` с параметром `CMAKE_BUILD_TYPE=Debug`. Сравните размеры исполняемых файлов и результаты тестов, не изменяя первую сборку.

## 2.7 libiio, подключение QPSK SDR Lab и каналы 2RX2TX

На SDR-плате работают Linux, IIO-драйвер радиотракта и сервер iiod. На компьютере библиотека libiio предоставляет C API, доступный также из C++. Через него приложение читает и записывает параметры устройств и обменивается буферами отсчётов. В нашем проекте между приложением и libiio нет промежуточных обёрток вроде SoapySDR.

Вторая библиотека, libad9361-iio, — надстройка Analog Devices над libiio для трансиверов семейства AD936x. При смене частоты дискретизации она рассчитывает и загружает согласованные цифровые фильтры и тактирование (функция `ad9361_set_bb_rate()`). Без такого фильтра трансивер не работает на частотах дискретизации ниже примерно 2,08 Мвыб/с, а примеры следующих глав используют и меньшие значения [8, 9].

Обязательное требование к стенду — два RX и два TX. Оно относится не только к разъёмам платы: прошивка должна предоставлять оба канала программному интерфейсу. Установка libiio на ПК сама по себе режим 2RX2TX не включает.

### 2.7.1 Локальная сборка libiio и libad9361-iio

В каталог `vendor/` включены libiio v0.26 и libad9361-iio v0.3. Их исходники, каталоги сборки и общий каталог установки `build/iio-install` разделены.

Команды выполняются из корня учебного репозитория. Сначала собирается libiio:

```
cmake -S vendor/libiio -B build/iio \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$PWD/build/iio-install" \
  -DCMAKE_INSTALL_LIBDIR=lib \
  -DCMAKE_INSTALL_RPATH="$PWD/build/iio-install/lib" \
  -DWITH_LOCAL_BACKEND=OFF -DWITH_IIOD=OFF \
  -DWITH_NETWORK_BACKEND=ON -DWITH_USB_BACKEND=ON \
  -DWITH_XML_BACKEND=ON -DHAVE_DNS_SD=OFF \
  -DWITH_TESTS=ON -DWITH_EXAMPLES=OFF \
  -DWITH_DOC=OFF -DWITH_MAN=OFF \
  -DPYTHON_BINDINGS=OFF -DINSTALL_UDEV_RULE=OFF
cmake --build build/iio --parallel
cmake --install build/iio
```

Первая команда — конфигурирование с параметрами libiio: включены сетевой и USB-интерфейсы; локальный доступ к IIO-устройствам самого компьютера и сервер iiod на ПК не нужны; флаг `WITH_TESTS` в этой версии libiio включает сборку утилит `iio_info`, `iio_attr` и других; автоматический поиск сетевых устройств отключён, поэтому для сети указывается IP-адрес. Вторая команда собирает библиотеку и утилиты в каталоге `build/iio`.

Третья команда, `cmake --install`, *копирует* готовые файлы в каталог установки — *префикс*, заданный параметром `CMAKE_INSTALL_PREFIX`: заголовки — в `include/`, библиотеки — в `lib/`, программы — в `bin/`. Если префикс не указать, CMake использует `/usr/local`. Это общесистемный каталог, записывать в который может только администратор, поэтому в инструкциях в интернете часто встречается `sudo make install`. После такой установки библиотека видна всем программам системы, может незаметно подменить другую версию, нужную другому проекту, а удалить её аккуратно непросто: её файлы перемешаны с файлами других программ. Мы задаём префикс внутри проекта (таблица 2.3).

**Таблица 2.3:** Глобальная и локальная установка библиотеки

| | Глобальная (по умолчанию) | Локальная (в этом проекте) |
|---|---|---|
| Префикс | `/usr/local` | `build/iio-install` |
| Права | нужен `sudo` | обычный пользователь |
| Кто видит | все программы системы | только программы, которым путь указан явно |
| Риск | подмена другой версии libiio | нет: системные каталоги не меняются |
| Удаление | по списку установленных файлов | удалить каталог `build/` |

После установки в префиксе появляются три каталога:

```
build/iio-install/bin/      iio_info  iio_attr  iio_genxml  iio_readdev ...
build/iio-install/include/  iio.h
build/iio-install/lib/      libiio.so -> libiio.so.0 -> libiio.so.0.26
```

У библиотеки три имени. Настоящий файл — `libiio.so.0.26`, с полной версией. `libiio.so.0` — ссылка с номером совместимости: это имя записывается в программу и ищется загрузчиком при запуске. `libiio.so` — ссылка без номера, её использует компоновщик при сборке по ключу `-liio`. Параметр `CMAKE_INSTALL_RPATH` записывает в установленные утилиты `RUNPATH` на `build/iio-install/lib` (раздел 2.3.2), поэтому `iio_info` не подхватит другую libiio, установленную в системе.

Затем libad9361-iio собирается против только что установленной libiio. Пути к ней задаются явно, чтобы CMake не нашёл системную копию:

```
cmake -S vendor/libad9361-iio -B build/ad9361 \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$PWD/build/iio-install" \
  -DCMAKE_INSTALL_RPATH="$PWD/build/iio-install/lib" \
  -DLIBIIO_INCLUDEDIR="$PWD/build/iio-install/include" \
  -DLIBIIO_LIBRARIES="$PWD/build/iio-install/lib/libiio.so" \
  -DWITH_DOC=OFF -DENABLE_PACKAGING=OFF
cmake --build build/ad9361 --parallel
cmake --install build/ad9361
```

При CMake 4 к обеим командам конфигурирования добавляют параметр `-DCMAKE_POLICY_VERSION_MINIMUM=3.5`: исходники библиотек рассчитаны на более старые версии CMake.

Теперь соберём учебную программу с явным указанием локальных библиотек:

```
cmake -S . -B build/ch02-iio \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DLAB_WITH_IIO=ON \
  -DLAB_IIO_PREFIX="$PWD/build/iio-install"
cmake --build build/ch02-iio --parallel
ctest --test-dir build/ch02-iio --output-on-failure
```

CMake ищет libiio и libad9361-iio только в каталоге `LAB_IIO_PREFIX` и останавливается с ошибкой, если их там нет. Ожидается девять тестов. Дополнительные три используют искусственные XML-контексты: проверяют распознавание 2RX2TX, отказ для 1RX1TX и отсутствие TX. Это тесты программы, не испытания реальной платы.

На компьютере могут быть другие версии libiio. Проверим, какие библиотеки фактически загружаются:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ldd ./build/ch02-iio/qpsk_channels
ldd build/iio-install/lib/libad9361.so.0
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/iio-install/bin/iio_info --version
```

В выводе `ldd` библиотеки `libiio.so.0` и `libad9361.so.0` должны находиться в `build/iio-install/lib`. Переменная `LD_LIBRARY_PATH` задана здесь только на один запуск (раздел 2.3.1), а путь к локальным библиотекам записан и в саму программу. Это видно по записям `RUNPATH`:

```
readelf -d build/ch02-iio/qpsk_channels | grep -E 'NEEDED|RUNPATH'
 0x...0001 (NEEDED)   Shared library: [libiio.so.0]
 0x...001d (RUNPATH)  Library runpath: [/home/.../build/iio-install/lib]
```

Поэтому программа найдёт локальную libiio и без переменной, если только `LD_LIBRARY_PATH` не указывает на другую версию: она проверяется раньше `RUNPATH` (раздел 2.3.2).

**Контрольная точка**

1. Покажите вывод `ldd` для `qpsk_channels` и `libad9361.so.0`. Откуда загружается `libiio.so.0`?
2. Почему libad9361-iio собирают после libiio и с явными путями `LIBIIO_INCLUDEDIR` и `LIBIIO_LIBRARIES`? Что могло бы произойти без них на компьютере, где уже установлена другая libiio?
3. Чем отличается запуск `LD_LIBRARY_PATH=build/iio-install/lib ./программа` от добавления того же пути в `~/.bashrc`?

Ответы для сверки: обе библиотеки должны ссылаться на `build/iio-install/lib`. Без явных путей CMake libad9361-iio нашёл бы первую попавшуюся libiio, например системную, и библиотека была бы собрана с чужими заголовками и связана с чужой версией. Переменная перед командой действует на один запуск, а запись в `.bashrc` — на все программы во всех новых сеансах терминала, включая не относящиеся к проекту.

### 2.7.2 Подключение и сетевые настройки

Для обмена по USB используйте разъём DATA: разъём 5V-IN предназначен для питания. Сравните вывод `lsusb` до и после подключения. Команда `ip -br address` показывает сетевые интерфейсы и их адреса.

USB-подключение может предоставлять два пути к IIO: прямой USB с URI `usb:...` и сетевой интерфейс с URI вида `ip:192.168.2.1`. URI — строка, указывающая библиотеке способ подключения и адрес устройства. Для RJ45 используется тот же префикс `ip:`, но адрес Ethernet-интерфейса платы. Доступные прямые USB-подключения покажет:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/iio-install/bin/iio_info -s
```

Полученный USB URI может измениться при переподключении. Для IP-подключения задавайте фактический адрес, а не автоматически адрес из примера.

В поддерживающей этот механизм прошивке плата также появляется как USB-накопитель с `config.txt`. Сохраните копию файла, затем отредактируйте нужные поля существующих секций. Пример:

```
[NETWORK]
hostname = pluto
ipaddr = 192.168.2.1
ipaddr_host = 192.168.2.10
netmask = 255.255.255.0

[USB_ETHERNET]
ipaddr_eth = 192.168.70.150
netmask_eth = 255.255.255.0
```

Секция `NETWORK` относится к USB-сети; `USB_ETHERNET` в прошивке QPSK SDR Lab задаёт Ethernet. Адреса приведены как пример. Для Ethernet компьютер настраивается отдельно, например на `192.168.70.10/24`, если эта подсеть свободна. Адреса компьютера и платы должны различаться и принадлежать одной подсети. Не назначайте одну и ту же подсеть USB-сети и RJ45 без отдельной настройки маршрутизации.

Сохраните `config.txt` и выполните извлечение накопителя средствами ОС, не отключая питание платы. Это не то же самое, что просто выдернуть кабель. Дождитесь применения настроек и проверьте соединение по новому адресу [10]. Не меняйте поля `dfu`, `reset` и `calibrate` для настройки IP.

Для прямого USB обычному пользователю могут потребоваться права на узел устройства. Порядок создания точечного правила udev по фактическому USB-ID описан в `docs/connection.md`. Для IP-backend такое USB-правило не нужно. Не рекомендуется решать проблему назначением прав всем USB-устройствам или постоянным запуском программы через `sudo`.

**Контрольная точка**

1. Какой URI вы использовали для подключения и через какой интерфейс он работает: прямой USB, USB-сеть или Ethernet?
2. Компьютер получил по USB-сети адрес `192.168.2.10/24`, а плата по Ethernet — `192.168.2.150`. Почему такая настройка приведёт к проблемам и как её исправить?
3. Почему после правки `config.txt` накопитель нужно извлечь средствами ОС, а не просто отключить кабель?

Ответы для сверки: чтобы достучаться до платы по Ethernet, компьютеру нужен адрес в той же подсети `192.168.2.0/24`, и одна подсеть окажется на двух интерфейсах; маршрут к плате станет неоднозначным. Ethernet-адрес платы переносят в другую подсеть, например `192.168.70.150`. Без извлечения ОС может не дописать файл, а прошивка — не получить сигнал применить настройки.

### 2.7.3 Проверка каналов без запуска потока

Для USB-сетевого адреса из приведённого примера:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/iio-install/bin/iio_info -u ip:192.168.2.1
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/ch02-iio/qpsk_channels ip:192.168.2.1
```

Программа ищет управляющее устройство `ad9361-phy`, потоковое RX-устройство `cf-ad9361-lpc` и TX-устройство `cf-ad9361-dds-core-lpc`. В каждом потоковом устройстве ожидаются четыре scan-элемента: `voltage0`, `voltage1` — компоненты $I$, $Q$ первого радиоканала; `voltage2`, `voltage3` — второго. Названия относятся к стандартной разметке AD936x; каналы управляющего устройства `ad9361-phy` имеют другое назначение.

Программа только читает описание контекста. Она не создаёт потоковых буферов, не записывает атрибуты и не запускает передачу. Успех означает наличие ожидаемой разметки 2RX2TX, но не подтверждает пропускную способность или одновременную работу потоков. Эти свойства проверяются на следующем этапе.

**Внимание**

Если видна только одна пара I/Q, сначала сохраните описание контекста и версию прошивки. Замена libiio не создаст отсутствующие каналы FPGA. Не применяйте случайные команды `fw_setenv` и образы другой платы: нужны подтверждённая конфигурация QPSK SDR Lab 2RX2TX и порядок её восстановления.

**Контрольная точка**

Покажите путь к используемой libiio, URI платы и вывод `qpsk_channels`. Объясните, почему четыре scan-элемента RX соответствуют двум радиоканалам, а не четырём.

* Оцените суммарный поток полезных данных для одновременных двух RX и двух TX при $F_s = 1$ Мвыб/с, если каждая компонента $I$, $Q$ хранится в 16 битах. Какие расходы транспорта в эту оценку не вошли?

## 2.8 Автоматические тесты

*Автоматический тест* — команда, которая запускается без участия человека и сама решает, верен ли результат. Тест проверяет одну конкретную вещь, работает доли секунды и не требует платы, поэтому весь набор тестов прогоняют после каждой сборки и каждой правки кода. Если тест перестал проходить, значит, правка что-то сломала, и это видно сразу, а не на стенде.

### 2.8.1 Где описаны тесты

Тесты запускает CTest — программа из состава CMake. Отдельной программы-«тестера» в проекте нет: каждый тест — это строка `add_test` в `CMakeLists.txt`, которая даёт тесту имя и задаёт команду запуска. Тест пройден, если команда завершилась с кодом возврата 0. Фрагмент описания тестов:

```
add_test(NAME environment COMMAND env_report)

add_test(NAME reject_nyquist COMMAND iq_tone unused.cf32 1000 500 0.01 0.5)
set_tests_properties(reject_nyquist PROPERTIES WILL_FAIL TRUE)

add_test(NAME tone_file COMMAND ${CMAKE_COMMAND}
  -DPROGRAM=$<TARGET_FILE:iq_tone> -DOUT=${CMAKE_CURRENT_BINARY_DIR}/test-tone.cf32
  -P ${CMAKE_CURRENT_SOURCE_DIR}/tests/tone_file.cmake)
```

```
add_test(NAME channels_2r2t COMMAND qpsk_channels
  "xml:${CMAKE_CURRENT_SOURCE_DIR}/tests/2r2t.xml")
```

В проекте используются три вида тестов.

*Запуск программы.* Тест `environment` просто запускает `env_report`: программа должна собраться, запуститься и завершиться без ошибки.

*Обязательный отказ.* Свойство `WILL_FAIL TRUE` переворачивает условие: тест пройден, если программа завершилась *с ошибкой*. Так проверяется защита от неверных аргументов. Тон 500 Гц при $F_s = 1000$ Гц нарушает условие Котельникова–Найквиста, и генератор `iq_tone` обязан отказаться его создавать; если он вдруг согласится, тест провалится.

*Сценарий проверки.* Команда `cmake -P файл.cmake` выполняет сценарий на языке CMake. Сценарий `tests/tone_file.cmake` запускает генератор, а затем сравнивает результат с ожиданием:

```
execute_process(COMMAND "${PROGRAM}" "${OUT}" 1000000 100000 0.01 0.5
                RESULT_VARIABLE result)
file(SIZE "${OUT}" bytes)
file(READ "${OUT}" first LIMIT 8 HEX)
if(NOT bytes EQUAL 80000 OR NOT first STREQUAL "0000003f00000000" ...)
  message(FATAL_ERROR "Incorrect size/count/first IQ sample/format")
endif()
```

Ожидаемые числа получены не запуском программы, а расчётом: 80 000 байт — по формуле (2.1), а первые восемь байт — это первый отсчёт $I = 0{,}5$, $Q = 0$. Число 0,5 в формате float32 записывается как `0x3f000000`, а в порядке little-endian его байты идут как `00 00 00 3f`. Вызов `message(FATAL_ERROR ...)` завершает сценарий с ошибкой, и тест считается проваленным. Все девять тестов этой главы описаны в таблице 2.4.

## 2.8.2 Запуск и чтение результата

Основные ключи `ctest`:

```
ctest --test-dir build/ch02-iio -N              # только список тестов
ctest --test-dir build/ch02-iio                 # запустить все
ctest --test-dir build/ch02-iio -R channels     # только тесты с "channels"
ctest --test-dir build/ch02-iio -R channels_1r1t -V   # подробно
```

Ключ `-R` отбирает тесты по части имени, `-V` печатает команду запуска и весь вывод программы, а `--output-on-failure` печатает вывод только проваленных тестов. Подробный запуск `channels_1r1t` на стенде авторов:

**Таблица 2.4:** Автоматические тесты главы

| Тест | Вид | Что проверяет |
|---|---|---|
| `environment` | запуск | программа собирается и запускается |
| `tone_file` | сценарий | размер файла, первый отсчёт, поля JSON и отказ перезаписать готовый файл |
| `reject_rate_nan`, `_inf`, `_1000000junk` | отказ | генератор не принимает нечисловую частоту дискретизации |
| `reject_nyquist` | отказ | генератор не создаёт тон выше $F_s/2$ |
| `channels_2r2t` | запуск | `qpsk_channels` узнаёт разметку 2RX2TX |
| `channels_1r1t` | отказ | `qpsk_channels` сообщает об ошибке, если каналов только по одному |
| `channels_missing_tx` | отказ | `qpsk_channels` сообщает об ошибке, если нет передающего устройства |

```
8: Test command: .../build/ch02-iio/qpsk_channels "xml:.../tests/1r1t.xml"
8: RX: cf-ad9361-lpc
8:   voltage0 (RF1 I): index=0 bits=12 storage=16 shift=0 signed=1 big_endian=0
8:   voltage1 (RF1 Q): index=1 bits=12 storage=16 shift=0 signed=1 big_endian=0
8:   voltage2 (RF2 I): MISSING scan element
8:   voltage3 (RF2 Q): MISSING scan element
8: TX: cf-ad9361-dds-core-lpc
8:   ...
8: Expected AD936x 2RX2TX layout is NOT exposed. Check firmware/context.
1/1 Test #8: channels_1r1t ....................   Passed    0.00 sec
```

Программа нашла только первый радиоканал и завершилась с ошибкой, а тест отмечен как пройденный: для него задано `WILL_FAIL`, то есть ошибка и была ожидаемым результатом.

### 2.8.3 XML-описание платы вместо платы

При подключении к плате libiio сначала получает от неё *описание контекста*: какие устройства есть, какие у них каналы, атрибуты и форматы отсчётов. Это описание передаётся в формате XML. libiio умеет создать контекст и из XML-файла: для этого служит URI вида `xml:путь/к/файлу.xml`. Программа видит в таком контексте те же устройства `ad9361-phy`, `cf-ad9361-lpc` и `cf-ad9361-dds-core-lpc`, читает их каналы и форматы, но отсчётов не получает: потоковых буферов у XML-контекста нет.

Для проверки `qpsk_channels` этого достаточно: программа как раз читает только описание. Файлы `tests/2r2t.xml`, `tests/1r1t.xml` и `tests/missing_tx.xml` написаны вручную и содержат лишь то, что программа проверяет. Так можно испытать реакцию на неисправности, которые

на исправной плате не получить. Фрагмент `tests/2r2t.xml`:

```
<device id="iio:device1" name="cf-ad9361-lpc">
  <channel id="voltage0" type="input">
    <scan-element index="0" format="le:S12/16&gt;&gt;0"/></channel>
  ...
  <channel id="voltage3" type="input">
    <scan-element index="3" format="le:S12/16&gt;&gt;0"/></channel>
</device>
```

Последовательность `>>` — это записанные по правилам XML символы `>>`, то есть формат `le:S12/16>>0` (раздел 4.7). Полное описание своей платы можно сохранить утилитой `iio_genxml`. Она печатает до и после XML служебные строки, поэтому команда `grep` оставляет только строку с XML:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/iio-install/bin/iio_genxml -u ip:192.168.2.1 \
  | grep '^<?xml' > build/my_board.xml
./build/ch02-iio/qpsk_channels xml:build/my_board.xml
```

Такой файл удобно приложить к отчёту: он фиксирует конфигурацию платы на момент опыта. С его копией можно и экспериментировать, например удалить канал и посмотреть, как отреагирует программа.

**Контрольная точка**

1. Какие тесты главы используют XML-описание и какие из них должны завершиться ошибкой?
2. Сравните `tests/1r1t.xml` и `tests/2r2t.xml`. Чем они различаются?
3. Удалите в копии `build/my_board.xml` канал `voltage3` устройства `cf-ad9361-lpc` и запустите `qpsk_channels`. Что выведет программа и каков код возврата (`echo $?`)?
4. * В своей ветке добавьте в `CMakeLists.txt` тест, проверяющий, что `iq_tone` отказывается работать с амплитудой 1,5. Пересоберите проект. Сколько тестов покажет `ctest -N`?

Ответы для сверки: XML используют три теста `channels_*`; ошибкой должны завершиться `channels_1r1t` и `channels_missing_tx`. В `1r1t.xml` у потоковых устройств только каналы `voltage0` и `voltage1`. Программа напишет `MISSING scan element` для `voltage3` и завершится с кодом 1. После добавления, например, `add_test(NAME reject_amplitude COMMAND iq_tone unused.cf32 1000000 100000 0.01 1.5)` со свойством `WILL_FAIL TRUE` тестов станет десять.

## 2.9 Паспорт программной среды

Результаты главы сводятся в один паспорт. Его заполняют фактическими значениями, а не ожидаемыми версиями из методического пособия. Значение «неизвестно» лучше ложной точности; оно сразу показывает, какую проверку ещё нужно выполнить.

**Таблица 2.5:** Паспорт программной среды и первичной проверки

| Поле | Фактическое значение |
|---|---|
| Дата и участник | |
| ОС и версия | Поля `NAME`, `VERSION`, `VERSION_ID` |
| Архитектура и ядро | Вывод `uname -m` и `uname -r` |
| Git | Версия; полный хеш commit; ветвь; локальные изменения |
| Компилятор | Семейство и версия |
| CMake | Версия и тип сборки |
| libiio, libad9361-iio | Версии; источник; фактический путь загруженных библиотек |
| Команда конфигурирования | Команды CMake со всеми параметрами |
| Автоматические тесты | Число пройденных и непройденных проверок |
| Контрольный IQ-файл | $F_s$, частота тона, длительность, амплитуда, число отсчётов, формат и размер |
| USB или Ethernet | Способ подключения, URI, версия прошивки, результат проверки каналов 2RX2TX, предел непрерывного потока (глава 3) |

**Контрольная точка**

**Итоговые вопросы к главе.**

1. Перечислите сведения, без которых другой участник не сможет повторить результаты этой главы на своём компьютере.
2. Какой размер будет у файла `iq_tone` длительностью 25 мс при $F_s = 2$ Мвыб/с? Сколько в нём комплексных отсчётов?
3. На реальной плате `qpsk_channels` сообщает `MISSING scan element` для `voltage2` и `voltage3`. Что это означает и что делать, а чего

делать не следует?

4. Почему успешный вывод `qpsk_channels` ещё не доказывает, что плата сможет одновременно передавать на ПК два канала RX при $F_s = 30$ Мвыб/с? Оцените требуемый поток и сравните его с USB 2.0 (480 Мбит/с) и гигабитным Ethernet.
5. * Как без платы проверить, что программа правильно отличает разметку 2RX2TX от 1RX1TX?

Ответы для сверки: полный хеш commit и локальные изменения, версии ОС, компилятора, CMake, libiio и libad9361-iio, команды конфигурирования, путь загруженной библиотеки, URI и версия прошивки. $N = 50\,000$ отсчётов, 400 000 байт. Прошивка предоставляет только один радиоканал: сохраняют описание контекста и версию прошивки и обращаются к преподавателю, не меняя libiio и не прошивая случайные образы. Поток составляет $2 \cdot 30 \cdot 10^6 \cdot 4 = 240$ Мбайт/с, тогда как USB 2.0 даёт не более 60 Мбайт/с, а гигабитный Ethernet — не более 125 Мбайт/с; разметка каналов о пропускной способности ничего не говорит. Для проверки без платы служат синтетические XML-контексты из `tests/`, которые использует CTest.

# 3 SDR-платформа изнутри: AD9361, Zynq и libiio

В главе 2 плата оставалась «чёрным ящиком»: программа нашла её устройства и убедилась, что каналов два на приём и два на передачу. Эта глава открывает ящик и отвечает на три вопроса, без которых следующие главы превращаются в набор рецептов:

- что происходит с сигналом внутри трансивера AD9361 и какой узел меняет каждый параметр, который программа записывает через libiio;
- как устроены плата и путь от вызова функции на ПК до регистра микросхемы;
- какой поток отсчётов плата на самом деле передаёт в ПК.

Все команды главы только читают состояние платы или принимают сигнал; передатчик не включается. Две новые программы — `board_state` (снимок состояния платы) и `rx_benchmark` (измерение потока) — собираются так же, как программы главы 2 с libiio:

```
cmake -S . -B build/ch03 -DCMAKE_BUILD_TYPE=Release \
  -DLAB_WITH_IIO=ON -DLAB_IIO_PREFIX="$PWD/build/iio-install"
cmake --build build/ch03 --parallel
ctest --test-dir build/ch03 --output-on-failure
```

Тестов по-прежнему девять. Автоматических тестов у программ этой главы нет: их результат — свойство конкретной платы и линии связи, а не кода, поэтому они проверяются на стенде. Измерения в главе выполнены через гигабитный Ethernet, адрес платы в нашей сети — `192.168.70.152`; в командах подставьте свой URI (раздел 2.7).

## 3.1 Трансивер AD9361

Вся радиочасть QPSK SDR Lab — одна микросхема: трансивер AD9363, программно переведённый в режим AD9361 [11]. Драйвер работает с ней как с AD9361, поэтому дальше используется документация на AD9361 [12]. Функциональная схема из этой документации приведена на рис. 3.1.

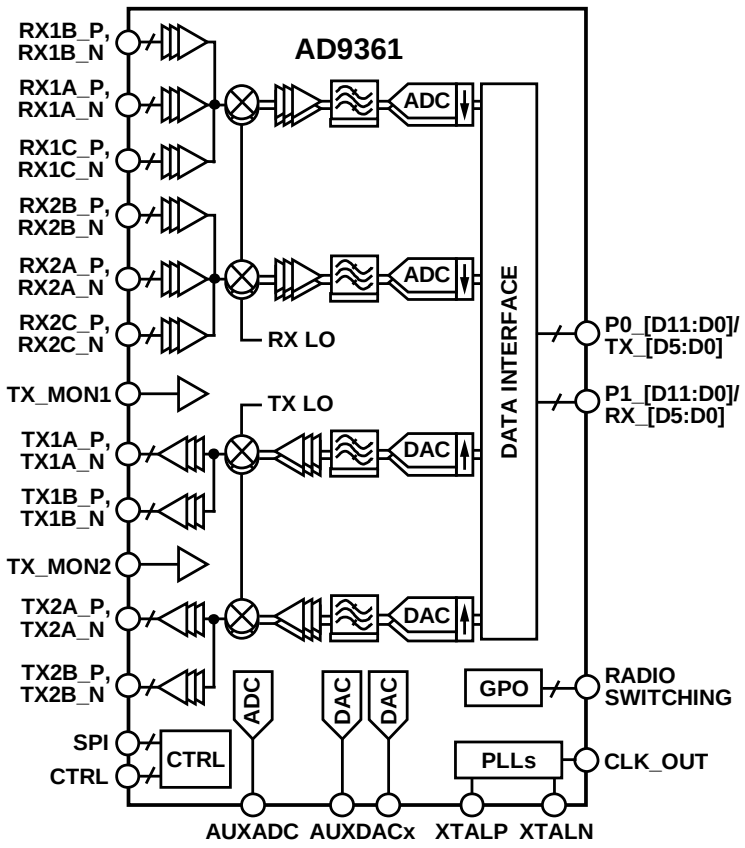



**Рис. 3.1:** Функциональная схема трансивера AD9361. Воспроизведена из документации Analog Devices [12] (рис. 1) без изменений.

**Приёмники.** Два одинаковых канала, RX1 и RX2. У каждого три дифференциальных входа: A, B и C (пары выводов `RX1A_P`, `RX1A_N` и т. д.). Переключатель подключает к тракту один из них; прошивка платы выбирает вход A (значение `A_BALANCED`), и менять его не нужно: при выборе входа, не соединённого с разъёмом, сигнал пропадает. Дальше по схеме слева направо: малошумящий усилитель (три треугольника), смеситель (круг с крестом), на который подаётся гетеродин RX LO, усилитель и аналоговый ФНЧ (значок фильтра), 12-разрядный АЦП и цифровые фильтры с понижением частоты отсчётов (стрелка вниз рядом с АЦП). На схеме смеситель изображён одним символом, но их два: для составляющей $I$ и для $Q$, с гетеродинами, сдвинутыми на 90°.

Это приёмник прямого преобразования: сигнал с частоты $f_0$ переносится сразу на нулевую частоту, промежуточной частоты нет. Отсюда две особенности, которые трансивер отслеживает и компенсирует сам: постоянная составляющая (просачивание гетеродина на вход смесителя) и неравенство ветвей $I$ и $Q$ (квадратурный дисбаланс). Остаток постоянной составляющей виден в спектрах главы 5 как линия на нулевой частоте.

Усиление приёмника распределено между МШУ, смесителем, усилителем и ФНЧ. Какой узел сколько даёт при заданном общем усилении, определяет таблица усиления в памяти трансивера; атрибут `hardwaregain` задаёт строку этой таблицы, а автоматическая регулировка (АРУ) выбирает её сама.

**Передатчики.** Два канала, TX1 и TX2, с обратным порядком узлов: цифровые фильтры с повышением частоты отсчётов, 12-разрядный ЦАП, ФНЧ, квадратурный модулятор с гетеродином TX LO и выходной каскад с ослаблением от 0 до 89,75 дБ с шагом 0,25 дБ. Выходов у каждого канала два (A и B), на плате используется A. Входы `TX_MON1`, `TX_MON2` служат для контроля выходной мощности и в курсе не используются.

**Синтезаторы и тактирование.** Блок PLLs на схеме содержит три синтезатора: RX LO (от 70 МГц до 6 ГГц) и TX LO (от 47 МГц до 6 ГГц) с шагом перестройки около 2,4 Гц, а также BBPLL — синтезатор тактовых частот АЦП, ЦАП и цифрового интерфейса (700–1400 МГц). Опорная частота подаётся на выводы `XTALP`, `XTALN`; на плате это термокомпенсированный генератор 40 МГц.

**Управление и вспомогательные узлы.** Все настройки — это регистры трансивера, которые записываются по последовательному интерфейсу SPI. Выводы CTRL позволяют управлять усилением в реальном времени, выходы GPO — переключать внешние усилители. Вспомогательный АЦП (AUXADC) измеряет напряжение на своём входе или встроенный датчик температуры, два вспомогательных ЦАП (AUXDAC) выдают управляющие напряжения. Режимами работы (сон, ожидание, приём, передача, одновременные приём и передача) управляет конечный автомат ENSM.

Связь узлов схемы с атрибутами, которые программа видит через libiio, сведена в таблицу 3.1. Её стоит держать перед глазами при чтении программ глав 5 и 6: каждая строка там, где вызывается `iio_channel_attr_write`, меняет один из этих узлов.

**Важно**

Гетеродинов два, а каналов четыре. Оба приёмника всегда настроены на одну частоту $f_0$, оба передатчика — на одну свою. Принять одновременно станции на 98 и 433 МГц двумя приёмниками нельзя: обе должны

**Таблица 3.1:** Узлы AD9361 и атрибуты устройства `ad9361-phy`

| Узел на рис. 3.1 | Канал и атрибут | Что меняется |
|---|---|---|
| Переключатель входов RX | `voltage0/1` (вход): `rf_port_select` | Какой из входов A, B, C подключён к тракту |
| МШУ, смеситель, усилитель, ФНЧ (усиление) | `voltage0/1` (вход): `gain_control_mode`, `hardwaregain` | Режим АРУ и строка таблицы усиления |
| ФНЧ приёмника и передатчика (полоса) | `voltage0/1`: `rf_bandwidth` | Частота среза аналогового фильтра |
| RX LO, TX LO | `altvoltage0` (`RX_LO`), `altvoltage1` (`TX_LO`): `frequency`, `powerdown` | Частота $f_0$ и включение синтезатора |
| BBPLL, АЦП, ЦАП, цифровые фильтры | `sampling_frequency`, `filter_fir_en`; атрибуты устройства `rx_path_rates`, `tx_path_rates` | Частота дискретизации $F_s$ и цепочка децимации (раздел 3.2) |
| Выходной каскад TX | `voltage0/1` (выход): `hardwaregain` | Ослабление от $-89{,}75$ до 0 дБ |
| Измеритель уровня RX | `voltage0/1` (вход): `rssi` | Только чтение: оценка уровня принятого сигнала |
| Компенсация DC и квадратур | `bb_dc_offset_tracking_en`, `rf_dc_offset_tracking_en`, `quadrature_tracking_en`; `calib_mode` | Непрерывное отслеживание постоянной составляющей и дисбаланса $I/Q$ |
| Опорная частота | атрибут устройства `xo_correction` | Значение опорной частоты, от которого драйвер считает все частоты |
| AUXADC и датчик температуры | `voltage2` (вход), `temp0` | Только чтение |
| AUXDAC1, AUXDAC2 | `voltage2`, `voltage3` (выход) | Вспомогательные напряжения; в курсе не используются |
| ENSM | атрибут устройства `ensm_mode` | Режим работы, на плате `fdd` |

попасть в полосу $f_0 \pm F_s/2$ одного гетеродина. Зато каналы когерентны: разность фаз между RX1 и RX2 не «плывёт» со временем, хотя и неизвестна заранее. На этом основаны разнесённый приём и пеленгация; фазу между каналами калибруют отдельно.

**Контрольная точка**

Какие узлы рис. 3.1 изменятся, если записать: а) `frequency` канала `altvoltage0`; б) `rf_bandwidth` канала `voltage0` (вход); в) `hardwaregain` канала `voltage0` (вход) в режиме `manual`; г) `hardwaregain` канала `voltage0` (выход)?

* Почему у функции `iio_device_find_channel` есть аргумент «выход», хотя имя канала уже задано?

Ответы: а) синтезатор RX LO, общий для обоих приёмников; б) аналоговые ФНЧ приёмника; в) распределение усиления между МШУ, смесителем, усилителем и ФНЧ приёмника RX1; г) ослабление выходного каскада TX1. Имя `voltage0` у `ad9361-phy` есть и у входного канала (RX1), и у выходного (TX1); без признака направления можно по ошибке изменить передатчик вместо приёмника.

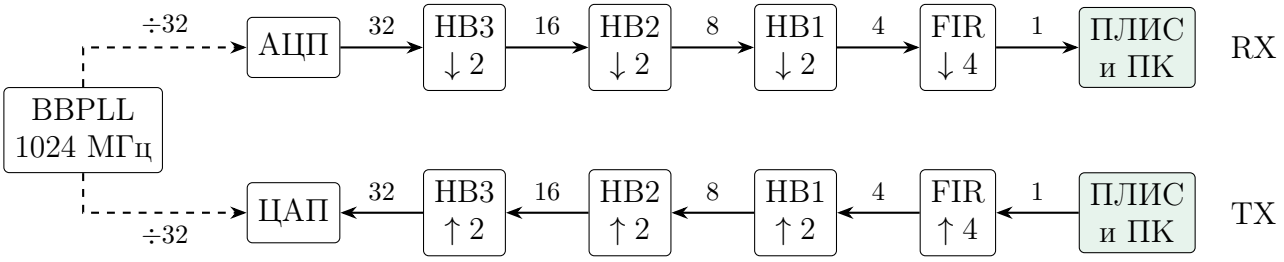
## 3.2 Цифровая цепочка и частота дискретизации

АЦП трансивера работает намного быстрее, чем частота $F_s$, которую получает программа. Лишние отсчёты отбрасывают цифровые фильтры с децимацией. Цепочку частот показывает атрибут `rx_path_rates`; при $F_s = 1$ Мвыб/с:

```
iio_attr -u ip:192.168.70.152 -d ad9361-phy rx_path_rates
BBPLL:1024000000 ADC:32000000 R2:16000000 R1:8000000 RF:4000000 RXSAMP:1000000
```

Расшифровка приведена на рис. 3.2. BBPLL работает на 1024 МГц, делитель на 32 даёт тактовую частоту АЦП 32 МГц. Три полуполосных фильтра (half-band, HB3, HB2 и HB1) понижают частоту отсчётов вдвое каждый, программируемый КИХ-фильтр (128 коэффициентов) — ещё в четыре раза. Полуполосный фильтр — ФНЧ с фиксированными коэффициентами и полосой пропускания до четверти частоты отсчётов на его входе; почти половина его коэффициентов равна нулю, поэтому он дёшев в аппаратуре. Передающая цепочка (`tx_path_rates`) зеркальна: те же ступени повышают частоту

отсчётов от $F_s$ до тактовой частоты ЦАП.




*числа над стрелками — частота отсчётов, МГц (Мвыб/с)*

**Рис. 3.2:** Цифровая цепочка AD9361 при $F_s = 1$ Мвыб/с по атрибутам `rx_path_rates` и `tx_path_rates`, прочитанным с платы.

Цепочку подбирает не программист, а функция `ad9361_set_bb_rate()` из libad9361-iio (раздел 2.7): она выбирает частоту BBPLL, коэффициенты децимации и рассчитывает КИХ-фильтр. Измеренные на плате варианты приведены в таблице 3.2.

**Таблица 3.2:** Цепочка децимации AD9361 при разных $F_s$ (измерено на плате)

| $F_s$, Мвыб/с | BBPLL, МГц | АЦП, МГц | HB3 | HB2 | HB1 | FIR | Всего |
|---|---|---|---|---|---|---|---|
| 0,521 | 800,26 | 25,01 | 3 | 2 | 2 | 4 | 48 |
| 1 | 1024 | 32 | 2 | 2 | 2 | 4 | 32 |
| 2,5 | 1280 | 80 | 2 | 2 | 2 | 4 | 32 |
| 10 | 1280 | 320 | 2 | 2 | 2 | 4 | 32 |
| 20 | 1280 | 640 | 2 | 2 | 2 | 4 | 32 |
| 30,72 | 983,04 | 491,52 | 2 | 2 | 2 | 2 | 16 |

Из таблицы следуют границы, которые раньше приходилось принимать на веру. Драйвер допускает тактовую частоту АЦП от 25 до 640 МГц, а ЦАП — не выше 320 МГц (константы `MIN_ADC_CLK`, `MAX_ADC_CLK` и `MAX_DAC_CLK` в исходном коде драйвера [13]). Без КИХ-фильтра полуполосные ступени понижают частоту не более чем в $3 \cdot 2 \cdot 2 = 12$ раз, поэтому наименьшая $F_s$ равна $25/12 \approx 2{,}08$ Мвыб/с — та самая граница из раздела 2.7, ниже которой нужен фильтр из libad9361-iio. С КИХ-фильтром ($\times 4$) коэффициент достигает 48, и наименьшая $F_s = 25/48 \approx 0{,}521$ Мвыб/с. Именно это значение стоит в начале атрибута `sampling_frequency_available`: `[520833 1 30720000]`. Верхняя граница 30,72 Мвыб/с в режиме 2RX2TX определяется пропускной способностью цифрового интерфейса между трансивером и ПЛИС; в режиме 1RX1TX она вдвое выше.

АЦП трансивера — сигма-дельта-типа: он работает с большим запасом по частоте (в таблице — в 16–48 раз выше $F_s$), а основную фильтрацию от

наложения спектров выполняют цифровые ступени. Поэтому $F_s$ и аналоговая полоса `rf_bandwidth` — разные параметры: первая задаёт цифровую цепочку, вторая — аналоговый ФНЧ перед АЦП. На 20 и 30,72 Мвыб/с частота ЦАП вдвое ниже частоты АЦП (атрибут `tx_path_rates` показывает `DAC:320000000` при `ADC:640000000`): иначе она превысила бы предельные для ЦАП 320 МГц.

Смена $F_s$ — медленная операция. Перестроить BBPLL, загрузить КИХ-фильтры и повторить калибровки занимает от 0,56 с (при 1 Мвыб/с) до 1,6 с (при 2,5 Мвыб/с), тогда как перестройка гетеродина — около 1 мс (таблица 3.3). Для обзора диапазона перестраивают гетеродин, а частоту дискретизации задают один раз.

**Контрольная точка**

Пусть при $F_s = 5$ Мвыб/с BBPLL работает на 1280 МГц, а цепочка та же, что в строке 2,5 Мвыб/с. Найдите частоту АЦП и частоты после каждой ступени. Почему нельзя установить $F_s = 0{,}4$ Мвыб/с?
Ответ: АЦП 160 МГц, затем 80, 40, 20 и 5 Мвыб/с (так и показывает плата). Для 0,4 Мвыб/с при наибольшем коэффициенте 48 АЦП пришлось бы тактировать частотой 19,2 МГц, что ниже допустимых 25 МГц; такие частоты получают дополнительной децимацией в программе (раздел 4.4).

## 3.3 Плата: AD9361 и Zynq-7010

Вокруг трансивера на плате три группы узлов (рис. 3.3): система на кристалле Zynq-7010, память и интерфейсы к ПК. Zynq объединяет в одной микросхеме процессорную часть (PS, два ядра ARM Cortex-A9) и программируемую логику (PL, ПЛИС) [11].

**ПЛИС.** В программируемой логике загружено ядро интерфейса AD9361 из открытого проекта Analog Devices [14]. Оно принимает от трансивера отсчёты $I$, $Q$ обоих приёмников по дифференциальной шине LVDS, разделяет их по каналам и через контроллеры DMA записывает блоками в DDR3. На передачу ядро читает блоки из памяти и выдаёт их трансиверу; в нём же находятся генераторы тона DDS. В libiio приёмная часть ядра видна как устройство `cf-ad9361-lpc`, передающая — как `cf-ad9361-dds-core-lpc`.

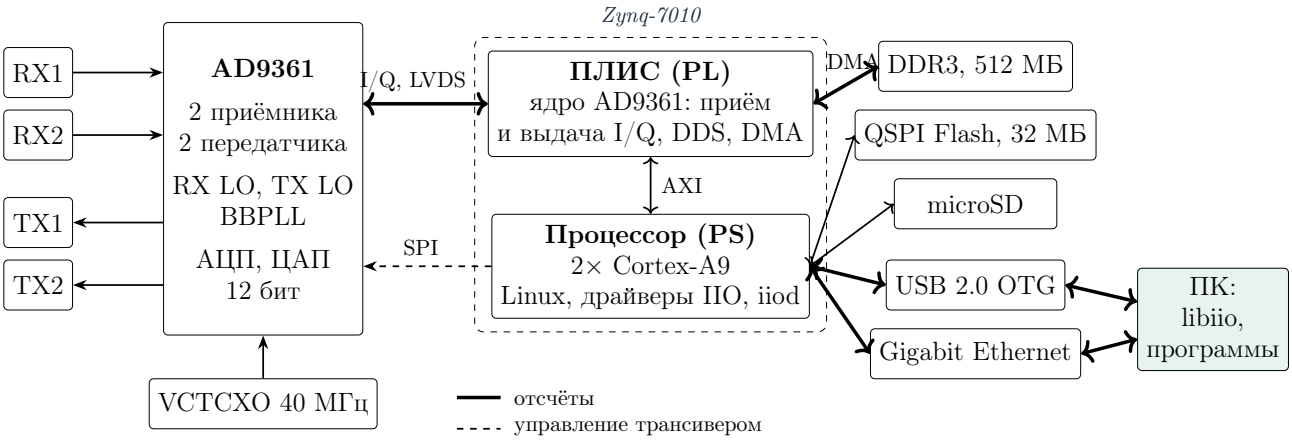



**Рис. 3.3:** Архитектура QPSK SDR Lab. Толстые линии — путь отсчётов, пунктир — управление трансивером.

**Процессор.** На ядрах ARM работает Linux. Его образ вместе с конфигурацией ПЛИС хранится во флеш-памяти QSPI (32 МБ) — это и есть прошивка, версию которой показывает атрибут `fw_version`. Оперативная память DDR3 (512 МБ) общая: в ней работают Linux и программы на плате, туда же DMA складывает блоки отсчётов. Драйверы IIO управляют трансивером по SPI и ядром ПЛИС, а сервер iiod даёт к ним доступ по сети или USB.

**Опорный генератор.** Все частоты платы — гетеродины, такты АЦП и ЦАП — получены от одного генератора 40 МГц с точностью ±0,5 ppm. На частоте 2,4 ГГц это ±1,2 кГц, а несущие двух плат могут разойтись до 2,4 кГц [11]. Для одной платы это незаметно, но приёмник, работающий с чужим передатчиком (часть Б курса), обязан оценивать и компенсировать сдвиг частоты. Атрибут `xo_correction` сообщает драйверу фактическую частоту опорного генератора, если она измерена; для общей опоры нескольких плат предусмотрен вход внешнего тактирования.

На рисунке видны два независимых пути, о которых шла речь в разделе 1.2. *Управляющий* путь (пунктир) несёт отдельные значения: процессор записывает регистры трансивера по SPI, каждая операция занимает доли миллисекунды. *Сигнальный* путь (толстые линии) несёт непрерывный поток: трансивер → LVDS → ПЛИС → DMA → DDR3 → Linux → Ethernet или USB → ПК. Его производительность ограничена самым медленным звеном (раздел 3.7).

## 3.4 Как устроена libiio

libiio [8] описывает любое устройство подсистемы IIO ядра Linux [15] одинаково, четырьмя уровнями объектов. Для нашей платы иерархия выглядит так:

```
контекст ip:192.168.70.152        атрибуты: fw_version, local,kernel, ...
|-- устройство ad9361-phy          атрибуты: ensm_mode, rx_path_rates, ...
|   |-- канал altvoltage0 (RX_LO), выход:  frequency, powerdown, ...
|   |-- канал voltage0, вход (RX1):        hardwaregain, rf_bandwidth, ...
|   |-- канал voltage0, выход (TX1):       hardwaregain, rf_bandwidth, ...
|   `-- ...                        отладочные: adi,2rx-2tx-mode-enable, ...
|-- устройство xadc                температура и питание Zynq
|-- устройство cf-ad9361-dds-core-lpc     буфер TX и генераторы DDS
`-- устройство cf-ad9361-lpc       буфер RX
    `-- каналы voltage0..3, вход: I1, Q1, I2, Q2, формат le:S12/16>>0
```

- *Контекст* — соединение с одной платой. `iio_create_context_from_uri()` создаёт его по URI.
- *Устройство* — один драйвер на плате. Устройство ищут по имени: номера `iio:deviceN` зависят от порядка загрузки драйверов и могут отличаться между прошивками.
- *Канал* — вход или выход устройства. Канал определяется идентификатором (`voltage0`) *и направлением*; у части каналов есть ещё имя (`RX_LO`). Каналы, которые попадают в буфер отсчётов, называются scan-элементами; у них есть номер и формат (раздел 2.7).
- *Атрибут* — именованное значение в виде текста. Числа передаются строкой в единицах драйвера: герцы, децибелы, тысячные доли градуса; функции `iio_channel_attr_read_longlong` и подобные лишь преобразуют строку. Атрибуты с суффиксом `_available` описывают допустимые значения: список или тройку `[минимум шаг максимум]`. Отладочные атрибуты (`iio_device_debug_attr_read`) показывают внутренние настройки драйвера.

Отдельно от этой иерархии существует *буфер*:

- `iio_device_create_buffer()` выделяет на плате несколько блоков памяти и запускает DMA;
- `iio_buffer_refill()` получает очередной заполненный блок;
- `iio_buffer_push()` отправляет блок на передачу.

Путь вызова от программы до микросхемы показан на рис. 3.4. На плате атрибуты — это файлы в `/sys/bus/iio/devices/` (а отладочные — в debugfs), буферы — символьные устройства `/dev/iio:deviceN`. Сервер iiod переводит

запросы libiio в чтение и запись этих файлов. Способ доставки запросов выбирает бэкенд по приставке URI: `ip:` — TCP-соединение с iiod, `usb:` — обмен по USB, `local:` — программа запущена на самой плате и обращается к файлам напрямую, `xml:` — только описание устройств без связи с платой (раздел 2.8).

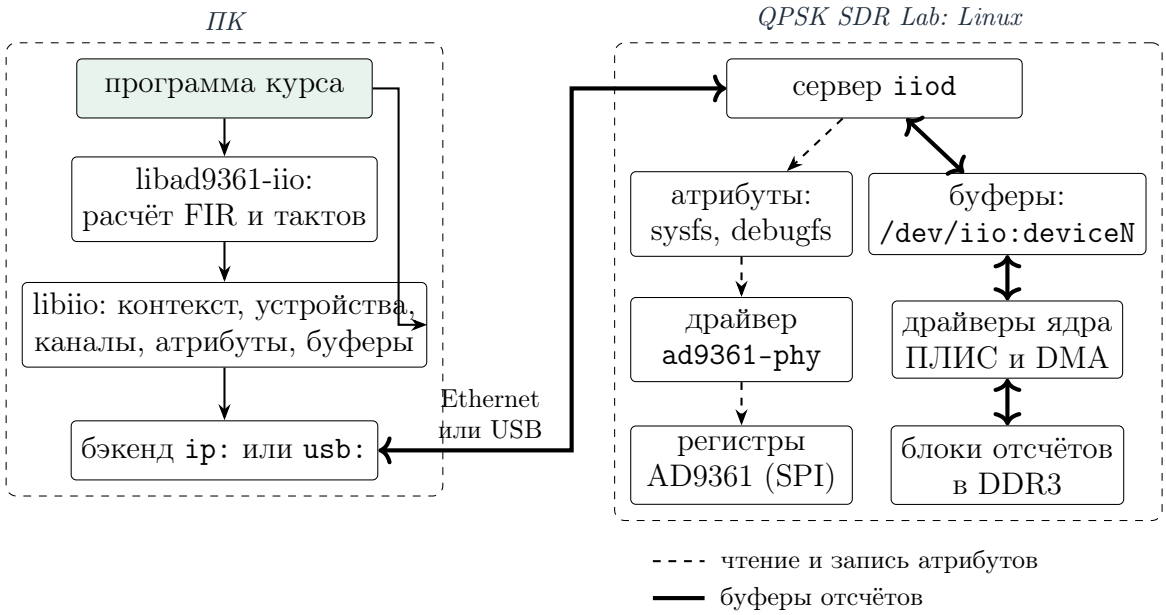



**Рис. 3.4:** Путь вызова libiio. Запись атрибута проходит через sysfs и драйвер до регистров трансивера; буферы отсчётов идут через DMA.

Каждая операция с атрибутом — это запрос и ответ по сети, а за ним работа драйвера. Измеренные затраты приведены в таблице 3.3.

**Таблица 3.3:** Время операций через гигабитный Ethernet (измерено на плате)

| Операция | Время |
|---|---|
| Создание контекста (`iio_create_context_from_uri`) | 2,4 мс |
| Чтение атрибута (`frequency` канала `RX_LO`) | 0,27 мс |
| Перестройка RX LO (запись `frequency`) | 1,1 мс |
| Смена $F_s$ (`ad9361_set_bb_rate`) | 0,56–1,6 с |

Для ручной проверки удобна утилита `iio_attr` из нашей сборки libiio. Её ключи соответствуют уровням иерархии:

```
B=./build/iio-install/bin; U=ip:192.168.70.152
$B/iio_attr -u $U -C fw_version                         # атрибут контекста
fw_version: v0.32-1-g7bdc-dirty
$B/iio_attr -u $U -d ad9361-phy ensm_mode               # атрибут устройства
fdd
$B/iio_attr -u $U -c ad9361-phy RX_LO frequency         # атрибут канала
98349998
$B/iio_attr -u $U -i -c ad9361-phy voltage0 hardwaregain   # -i: только входы
40.000000 dB
```

```
$B/iio_attr -u $U -D ad9361-phy adi,2rx-2tx-mode-enable    # отладочный
1
```

Если после имени атрибута указать значение, `iio_attr` его *запишет*. Для атрибутов приёмника это безопасно, для передатчика — только после решения GO главы 1. Ключ `-g файл.c` вдобавок сохраняет программу на C, которая выполняет то же действие через API libiio; это быстрый способ найти нужную функцию.

**Контрольная точка**

Программа в цикле перестраивает гетеродин по 100 частотам и на каждой читает по одному атрибуту. Оцените время цикла по таблице 3.3. Что изменится, если на каждой частоте ещё и менять $F_s$?
Ответ: около $100 \cdot (1{,}1 + 0{,}27) \approx 140$ мс без учёта самого приёма. Смена $F_s$ добавит $100 \cdot (0{,}56 \ldots 1{,}6)$ с, то есть от одной до трёх минут: частоту дискретизации задают один раз до цикла.

## 3.5 Что показывает iio_info

Полный вывод `iio_info` для нашей платы — около шестисот строк. В главе 2 из него понадобились только scan-элементы; здесь разобрано всё остальное. В начале выводятся атрибуты контекста: версия прошивки `fw_version`, ядро Linux `local,kernel`, модель трансивера, с которой работает драйвер (`ad9361-phy,model: ad9361`), опорная частота `ad9361-phy,xo_correction` и URI. Затем перечислены четыре устройства (таблица 3.4).

**Таблица 3.4:** Устройства IIO платы QPSK SDR Lab

| Устройство | Назначение |
|---|---|
| `ad9361-phy` | Драйвер трансивера: всё управление по SPI. Потока отсчётов не имеет. |
| `xadc` | Встроенный АЦП Zynq: температура кристалла и напряжения питания `vccint`, `vccaux`, `vccbram`, `vccpint`, `vccpaux`, `vccoddr`. |
| `cf-ad9361-dds-core-lpc` | Передающая часть ядра ПЛИС: буфер TX (`voltage0`–`voltage3`) и генераторы DDS (`altvoltage0`–`altvoltage7`). |
| `cf-ad9361-lpc` | Приёмная часть ядра ПЛИС: буфер RX (`voltage0`–`voltage3`). |

Каналы `ad9361-phy` и их атрибуты связаны с узлами трансивера в таб-

лице 3.1. Несколько деталей, которые сбивают с толку при первом чтении:

- Гетеродины — *выходные* каналы типа `altvoltage`: в терминах IIO синтезатор выдаёт переменное напряжение.
- Канал `out` — не отдельный выход, а общий для приёма и передачи переключатель КИХ-фильтров (`voltage_filter_fir_en`).
- `temp0` возвращает температуру трансивера в тысячных долях градуса: `49123` означает 49,1 °C.
- Атрибуты с суффиксом `_available` не меняются при работе и задают пределы: например, `frequency_available` канала `RX_LO` равно `[70000000 1 6000000000]`.
- Сообщения `ERROR` в выводе не означают неисправности. Атрибуты `dcxo_tune_coarse` и `dcxo_tune_fine` недоступны, потому что на плате внешний генератор, а не кварцевый резонатор с подстройкой; `multichip_sync` и `fastlock_recall` предназначены только для записи.

Атрибуты устройства `ad9361-phy` описывают трансивер целиком: `ensm_mode` (режим, у нас `fdd` — приём и передача одновременно), `calib_mode` (калибровки, `auto`), `rx_path_rates` и `tx_path_rates` (раздел 3.2), `filter_fir_config` (число коэффициентов и коэффициент децимации КИХ-фильтров, например `FIR Rx: 128,4 Tx: 128,4`), `gain_table_config` (загруженная таблица усиления; она своя для диапазонов до 1,3 ГГц, до 4 ГГц и выше, поэтому максимум `hardwaregain` зависит от $f_0$). Отладочные атрибуты подтверждают режим каналов: `adi,2rx-2tx-mode-enable` равен 1, `adi,frequency-division-duplex-mode-enable` — тоже 1.

У передающего устройства есть восемь каналов DDS: `TX1_I_F1`, `TX1_I_F2`, `TX1_Q_F1`, …, `TX2_Q_F2` — по два генератора тона ($F1$, $F2$) на каждую составляющую каждого передатчика. Атрибуты `frequency` и `phase` задают частоту и фазу тона (у $I$ фаза 90°, у $Q$ — 0°, вместе они дают комплексную экспоненту), `scale` — амплитуду, `raw` — включение. DDS формирует сигнал в ПЛИС без участия ПК: передатчик может излучать тон, хотя ни одна программа не отправила буфер. Когда программа отправляет буфер, данные DMA заменяют DDS. У буферных устройств есть ещё атрибуты буфера: `data_available`, `length_align_bytes` и `watermark`.

Сразу после включения плата находится в состоянии, заданном прошивкой: RX LO 2400 МГц, TX LO 2450 МГц (включён), $F_s = 30{,}72$ Мвыб/с, аналоговая полоса 18 МГц, приёмники в режиме АРУ `slow_attack`, ослабление передатчиков 10 дБ, амплитуда всех DDS равна нулю.

**Внимание**

Передатчик после включения не молчит: гетеродин TX работает, ослабление всего 10 дБ. Полезного сигнала нет, потому что DDS выключен и буфер не отправлен, но на выходе присутствует просачивание гетеродина и шум. Поэтому правило главы 1 — соединять TX и RX только через рассчитанный аттенюатор — действует с момента подачи питания, а не с запуска `tx_play`. Программа `board_state` (раздел 3.8) предупреждает, если амплитуда DDS отлична от нуля.

## 3.6 Порядок настройки трансивера

Параметры трансивера зависят друг от друга, поэтому порядок записи важен. Программы глав 5 и 6 настраивают плату в следующей последовательности.

**Приём.**

1. Найти устройства и каналы по именам и направлениям. Это быстро и ничего не меняет на плате.
2. Задать $F_s$ функцией `ad9361_set_bb_rate()`. Она перестраивает тактирование и фильтры всего трансивера и запускает калибровки, поэтому выполняется первой.
3. Задать аналоговую полосу `rf_bandwidth`.
4. Задать частоту гетеродина `frequency`.
5. Выбрать режим усиления `gain_control_mode` и только затем, в режиме `manual`, усиление `hardwaregain`: в режимах АРУ драйвер не примет ручное значение.
6. Прочитать фактические значения. Синтезатор и BBPLL выбирают ближайшие достижимые частоты: в главе 5 на запрос 98,35 МГц плата установила `98349998` Гц.
7. Включить нужные scan-элементы, создать буфер и отбросить первый блок: в нём могут оказаться отсчёты, принятые до окончания настройки.

**Передача.** Сначала передатчику задают наибольшее ослабление, −89,75 дБ, затем выполняют шаги 2–4 и создают буфер. Рабочее ослабление устанавливают только после отправки буфера, а при любом выходе из программы возвращают наибольшее ослабление и выключают TX LO. По этому порядку написана программа `tx_play` из раздела 6.4.

## 3.7 Предел потока отсчётов

Каждый комплексный отсчёт одного приёмника занимает в буфере 4 байта (по 16 бит на $I$ и $Q$). Поток данных от платы к ПК

$$R_{\mathrm{IQ}} = 4\, N_{\mathrm{RX}}\, F_s, \tag{3.1}$$

где $N_{\mathrm{RX}}$ — число одновременно принимаемых каналов. Один приёмник на 1 Мвыб/с даёт 4 МБ/с, два приёмника на 30,72 Мвыб/с — 246 МБ/с. Для сравнения: USB 2.0 передаёт не более 60 МБ/с, гигабитный Ethernet — не более 125 МБ/с. Широкая полоса в ПК непрерывно не проходит; вопрос в том, где именно проходит граница.

Программа `rx_benchmark` отвечает на него измерением. Для каждой $F_s$ она в течение нескольких секунд принимает поток, считает отсчёты, дошедшие до ПК за секунду, и читает флаг переполнения ядра ПЛИС. Флаг выставляется, когда у DMA не осталось свободного блока памяти и отсчёты пришлось выбросить. Это бит 2 регистра состояния `0x80000088` приёмного устройства; так же его проверяет пример `iio_adi_xflow_check.c` из libiio [16]:

```
// AXI ADC core status register: bit 2 = RX DMA overflow, samples lost.
constexpr std::uint32_t kStatusReg = 0x80000088;
constexpr std::uint32_t kOverflow = 0x4;
...
// The first block only starts the flow; the clock starts after it.
check(iio_buffer_refill(buf.get()), "RX refill");
check(iio_device_reg_write(rx, kStatusReg, kOverflow), "clear overflow flag");
...  // refill for the requested number of seconds, count bytes
check(iio_device_reg_read(rx, kStatusReg, &status), "read overflow flag");
const bool overflow = (status & kOverflow) != 0;
```

Программа меняет только частоту дискретизации и в конце возвращает исходную. Измерение через гигабитный Ethernet, буфер 262 144 отсчёта, 3 с на точку:

```
./build/ch03/rx_benchmark --uri ip:192.168.70.152
link ip:192.168.70.152, buffer 262144 samples, 3.0 s per point
RX  Fs, MS/s  received, MS/s  payload, MB/s  overflow  result
1        1.000          1.000           4.0  no        continuous
1        2.500          2.502          10.0  no        continuous
1        5.000          5.003          20.0  no        continuous
1        7.500          7.505          30.0  no        continuous
1       10.000         10.132          40.5  no        continuous
```

```
1       15.000          13.721          54.9  yes       SAMPLES LOST
1       20.000          13.478          53.9  yes       SAMPLES LOST
1       30.720          13.549          54.2  yes       SAMPLES LOST
2        1.000           1.001           8.0  no        continuous
2        2.500           2.541          20.3  no        continuous
2        5.000           5.005          40.0  no        continuous
2        7.500           6.914          55.3  yes       SAMPLES LOST
2       10.000           7.123          57.0  yes       SAMPLES LOST
2       15.000           6.846          54.8  yes       SAMPLES LOST
2       20.000           6.827          54.6  yes       SAMPLES LOST
2       30.720           6.838          54.7  yes       SAMPLES LOST
Sample rate restored to 1.000000 MS/s
```

Выводы из измерения:

- Потолок — около 55 МБ/с независимо от числа каналов: один приёмник получает до 13,7 Мвыб/с, два — до 6,9 Мвыб/с каждый. Это меньше половины возможностей гигабитного Ethernet: ограничивает не кабель, а программный путь на плате — сетевой стек и сервер iiod на процессоре ARM.
- Между 40 и 55 МБ/с результат меняется от запуска к запуску. В повторных измерениях по 10 с один приёмник на 12 Мвыб/с (48 МБ/с) трижды прошёл без потерь, а два приёмника на 5 Мвыб/с (40 МБ/с) один раз из трёх потеряли отсчёты. Все повторы при 32 МБ/с прошли без потерь. Для непрерывного приёма закладывайте не более 30 МБ/с: один приёмник до 7,5 Мвыб/с, два — до 3,75 Мвыб/с каждый.
- Размер буфера тоже влияет: каждый блок — отдельный запрос по сети. В наших пробах с буферами 16 384 и 65 536 отсчётов один приёмник терял отсчёты уже на 10–12 Мвыб/с, а с буфером 1 048 576 без потерь прошли и 12 Мвыб/с.
- Даже ниже предела поток может прерваться из-за нагрузки на ПК или сеть, поэтому программа, которой важна непрерывность, проверяет флаг переполнения, а не надеется на запас.

Широкую полосу всё же можно записать — *отрезками*. Внутри одного блока DMA отсчёты непрерывны при любой $F_s$ до 30,72 Мвыб/с: ПЛИС пишет их в DDR3 платы с полной скоростью, а передача в ПК идёт уже после. Пропуски возможны только между блоками. Поэтому программа `rx_capture` (глава 5) записывает всю запись одним буфером: длина такой записи ограничена памятью платы, а не линией связи. Узкополосный сигнал можно принимать непрерывно сколь угодно долго, широкополосный — отрезками с паузами между ними.

**Контрольная точка**

Повторите измерение на своём стенде. Через USB достаточно команды с URI `ip:192.168.2.1`; сравните пределы для USB и Ethernet. Затем повторите две точки вблизи предела с ключом `--seconds 10` три раза и запишите, сколько запусков прошли без потерь.

* Задача части Б: модем QPSK с символьной скоростью 1 Мсимв/с и четырьмя отсчётами на символ должен непрерывно принимать по двум каналам. Хватит ли пропускной способности? Ответ: $F_s = 4$ Мвыб/с, по формуле (3.1) $R_{\text{IQ}} = 4 \cdot 2 \cdot 4 = 32$ МБ/с — на границе рекомендованного запаса; надёжнее принимать один канал или снизить число отсчётов на символ.

## 3.8 Проверка платы перед работой

Программа `board_state` собирает в одну сводку всё, что нужно знать о плате перед опытом. Она только читает атрибуты: режим каналов, гетеродины, цепочку частот, настройки каналов, состояние DDS, температуры и напряжения питания. Ключ `--json` сохраняет ту же сводку в файл, который удобно приложить к записи опыта.

```
mkdir -p build/ch03/results
./build/ch03/board_state ip:192.168.70.152 --json build/ch03/results/board.json
uri                     ip:192.168.70.152
firmware                v0.32-1-g7bdc-dirty
transceiver             ad9361
channel_mode            2RX2TX
duplex                  FDD
rx_lo_mhz               98.349998
tx_lo_mhz               2450.000000 (on)
rx_path_rates           BBPLL:1024000000 ADC:32000000 R2:16000000 ... RXSAMP:1000000
rx1_gain                manual, 40.000000 dB
rx1_bw_mhz              1.000
tx1_gain                -10.000000 dB
...
fs_msps                 1.000000
dds_tones               off (scale 0)
ad9361_temp_c           49.1
zynq_temp_c             56.5
vccint_v                1.021
...
vccoddr_v               1.505
No warnings
```

Программа выводит предупреждение (`WARNING`), если прошивка предоставляет только один канал приёма и передачи, если включён хотя бы один генератор DDS и если температура Zynq выше 80 °C. Напряжения питания полезно сравнить с номиналами: ядро ПЛИС и процессора (`vccint`, `vccpint`, `vccbram`) — 1,0 В, вспомогательное питание (`vccaux`, `vccpaux`) — 1,8 В, память DDR3 (`vccoddr`) — 1,5 В. Отклонение больше чем на 5 % — повод проверить блок питания и кабель.

Сводку `board_state` снимают в начале каждого занятия со стендом. Версию прошивки и режим каналов переносят в паспорт стенда (таблица 1.7), измеренный предел потока — в паспорт программной среды (таблица 2.5).

**Контрольная точка**

**Итоговые вопросы к главе.**

1. Перечислите узлы приёмного тракта AD9361 от входа до цифрового интерфейса. Какие из них настраивает программа и какими атрибутами?
2. Почему $F_s = 2$ Мвыб/с нельзя установить без КИХ-фильтра, а $F_s = 0{,}6$ Мвыб/с можно установить только с ним?
3. Какой путь проходит запись атрибута `frequency` канала `RX_LO` от программы на ПК до микросхемы? Чем этот путь отличается от пути отсчётов?
4. В выводе `iio_info` у атрибута `dcxo_tune_coarse` стоит `ERROR: No such device`. Неисправна ли плата?
5. Плата только что включена, программы не запускались. Можно ли соединить TX1 и RX1 кабелем без аттенюатора? Почему?
6. Сколько данных в секунду даёт приём двумя каналами при $F_s = 2{,}5$ Мвыб/с? Пройдёт ли такой поток непрерывно по результатам `rx_benchmark`?
7. * Как записать без пропусков 0,1 с сигнала в полосе 25 МГц, если непрерывный поток ограничен 55 МБ/с?

Ответы для сверки: вход A, МШУ, смеситель с RX LO, усилитель и ФНЧ, АЦП, цифровые фильтры; программа задаёт `rf_port_select`, `hardwaregain` и `gain_control_mode`, `rf_bandwidth`, `frequency` канала `RX_LO` и $F_s$. Без КИХ-фильтра наибольший коэффициент децимации 12, и АЦП пришлось бы тактировать ниже 25 МГц; для 0,6 Мвыб/с нужен коэффициент 48, возможный только с КИХ-фильтром (×4). Запись ат-

рибута: libiio → бэкенд → iiod → файл sysfs → драйвер `ad9361-phy` → регистры по SPI; отсчёты идут по DMA через DDR3 и `/dev/iio:deviceN`. Нет: атрибуты подстройки кварца недоступны, потому что на плате внешний генератор. Нельзя: гетеродин TX включён, ослабление всего 10 дБ, и правило главы 1 действует с момента включения. $R_{\mathrm{IQ}} = 4 \cdot 2 \cdot 2{,}5 = 20$ МБ/с; проходит. Записать одним буфером: 0,1 с при $F_s = 30{,}72$ Мвыб/с — это 3 072 000 отсчётов, 12,3 МБ в памяти платы; внутри блока DMA пропусков нет.

# 4 I/Q-отсчёты и минимальный инструментарий ЦОС

В предыдущих главах SDR рассматривалась как совокупность аппаратных и программных узлов. Теперь мы временно отложим конкретную плату в сторону и перейдём к тому объекту, с которым непосредственно работает программа, — к последовательности комплексных отсчётов. Именно в этой форме один и тот же сигнал далее будет сохранён в файл, передан через ZeroMQ, обработан с помощью FFTW3 и, наконец, принят или передан средствами libiio.

Цель главы — не заменить отдельный курс цифровой обработки сигналов, а зафиксировать минимальный набор соглашений, без которого невозможно поставить воспроизводимый SDR-эксперимент. К концу главы читатель должен уметь:

- связать номер отсчёта с физическим временем и частотой;
- объяснить смысл компонент $I$ и $Q$ и знак частотного смещения;
- заранее предсказать результат дискретизации, частотного переноса и изменения частоты дискретизации;
- отличить формат данных в памяти программы, в файле и в транспортном интерфейсе SDR;
- правильно сопоставить бины ДПФ физическим частотам;
- сформировать паспорт контрольного IQ-файла и проверить его без использования SDR-платы.

**Важно**

В SDR один и тот же массив чисел можно интерпретировать совершенно по-разному. Без частоты дискретизации, порядка компонент, масштаба амплитуды и сведений о центральной частоте файл с отсчётами не является законченным результатом эксперимента. Поэтому далее математическая операция всегда рассматривается вместе с соглашением о данных, на которых она выполняется.

## 4.1 От времени к номеру отсчёта

Пусть непрерывный сигнал наблюдается через равные интервалы времени $T_s$. Получаем последовательность

$$x[n] = x(t)\big|_{t=t_0+nT_s}, \qquad T_s = \frac{1}{f_s}, \tag{4.1}$$

где $n = 0, 1, \ldots, N-1$ — номер отсчёта, $f_s$ — частота дискретизации, а $t_0$ — момент, соответствующий первому отсчёту. В чисто программном опыте начало времени часто условно принимают равным нулю. В аппаратном опыте $t_0$ может быть меткой времени SDR, временем операционной системы либо просто моментом запуска программы. Эти варианты неравноценны, и выбранный вариант необходимо указать в паспорте эксперимента.

Запись из $N$ отсчётов охватывает интервал

$$T_{\text{obs}} = NT_s = \frac{N}{f_s}. \tag{4.2}$$

При этом разница во времени между первым и последним отсчётами равна $(N-1)T_s$. Различие между $(N-1)T_s$ и $NT_s$ кажется формальным, но становится заметным при разметке стыков соседних блоков: следующий блок должен начаться в момент $t_0 + NT_s$, а не повторять последний отсчёт предыдущего; иначе на стыке блоков один отсчёт будет продублирован или затерт, а временная сетка окажется нарушена

### 4.1.1 Блок как структурная единица сигнала

Программы SDR почти никогда не обрабатывают бесконечную последовательность целиком. Поток делится на буферы, или блоки, по $N_b$ отсчётов. Размер блока определяет вычислительную нагрузку, задержку и частоту обращения к файловому, сетевому или аппаратному интерфейсу, но сам по себе не изменяет физическую частоту дискретизации. Например, при $f_s = 1$ МГц блок из 4096 отсчётов соответствует 4,096 мс сигнала независимо от того, сколько времени процессор фактически затратил на его обработку.

Полезно различать три времени:

- *сигнальное время* $N_b/f_s$, описывающее реальную длительность данных;
- *время вычисления*, затраченное программой на обработку блока;
- *сквозную задержку* от появления отсчёта на входе вычислительного модуля до появления соответствующего результата на его выходе.

Если время вычисления устойчиво превышает сигнальное время, программа не успевает работать в реальном времени: растёт очередь, переполняется буфер, а в аппаратном приёмнике возникает переполнение потока. Файл при этом может быть обработан корректно, только медленнее реального времени. Это одна из причин, почему программный эталон следует сначала проверять на файле с измерением времени вычисления и лишь затем подключать к SDR.

**Контрольная точка**

При $f_s = 2$ МГц программа получает блоки по $N_b = 10\,000$ комплексных отсчётов в формате `cf32_le_iq` — см. §4.7. Нумерация блоков начинается с нуля.

1. Определите сигнальную длительность блока, число блоков в секунду и требуемую скорость записи данных без учёта метаданных.
2. Запишите моменты времени первого и последнего отсчётов блока с номером $r$, если $t_0 = 0$. В какой момент должен начинаться блок $r + 1$?
3. Буфер вмещает 64 блока. На сколько секунд непрерывного сигнала и на какой объём данных он рассчитан?
4. * В каждом цикле из десяти блоков девять обрабатываются по 4 мс, а один — 12 мс. Определите среднее время обработки, выясните, растёт ли очередь неограниченно, и найдите максимальное накопленное отставание внутри установившегося цикла.

Ответы для сверки: 5 мс; 200 блоков/с; 16 Мбайт/с; для блока $r$ границы равны $rN_b/f_s$ и $(rN_b + N_b - 1)/f_s$, следующий блок начинается в $(r + 1)N_b/f_s$; буфер хранит 0,32 с, или 5,12 Мбайт. В задании * среднее время равно 4,8 мс, поэтому очередь не растёт неограниченно; максимальное отставание сразу после медленного блока равно 7 мс и устраняется за семь последующих быстрых блоков.

## 4.2 Комплексная базовая полоса

Радиосигнал на антенном разъёме представляет собой вещественное напряжение. Тем не менее хранить и обрабатывать его непосредственно на радиочастоте обычно невыгодно. Например, сигнал с полосой 10 МГц, расположенный около $f_0 = 868$ МГц, при обычной прямой дискретизации без использова-

ния полосового режима АЦП потребовал бы частоты дискретизации более $2 \cdot 873 = 1746$ МГц. После переноса того же сигнала в базовую полосу необходимая частота дискретизации определяется уже преимущественно шириной его полосы, а не значением несущей.

В непрерывных спектральных выкладках далее используется круговая частота $\Omega = 2\pi f$ и обозначение спектра $X(j\Omega)$. Соответственно, $\Omega_0 = 2\pi f_0$ — круговая частота несущей. В дискретных формулах и численных примерах сохраняется частота $f$ в герцах, поскольку именно в этих единицах задаются параметры SDR и NCO.

В аналоговом приёмнике сигнал можно перенести на ненулевую промежуточную частоту $f_{\mathrm{IF}} \neq 0$ либо непосредственно в окрестность нуля ($f_{\mathrm{IF}} = 0$). Второй вариант называют архитектурой нулевой промежуточной частоты, прямого преобразования или ZIF (Zero Intermediate Frequency). Он позволяет заменить часть полосовой фильтрации фильтрами нижних частот, однако требует сохранить не одну, а две квадратурные проекции принимаемого сигнала.

### 4.2.1 Почему одного смесителя недостаточно

Представим следующую ситуацию. Вы с другом решили организовать радиосвязь. У друга находится передатчик, а у вас — приёмник. Для текстовых сообщений используется амплитудная манипуляция последовательностью по закону азбуки Морзе, а для передачи звука — частотная модуляция.

Первоначально приёмник построен по супергетеродинной схеме с ненулевой промежуточной частотой. После преобразователя частоты в нём остаётся обычное низкочастотное гармоническое колебание, пригодное для аналогового амплитудного или частотного детектора. Затем вы решаете перейти к ZIF: полосовой фильтр на ПЧ заменяется фильтром нижних частот, а полезный сигнал переносится точно к нулю. После перестройки телеграфный канал неожиданно пропадает, а звуковой канал оказывается сильно искажённым. Проверка монтажа не обнаруживает неисправности, поэтому причину приходится искать в самом принципе работы одноканального смесителя.

Рассмотрим сначала амплитудно-манипулированный сигнал

$$s_{\mathrm{RF}}(t) = a(t)\cos(\Omega_0 t), \tag{4.3}$$

где огибающая $a(t)$ принимает уровни, соответствующие наличию и от-

сутствию телеграфной посылки. Предположим идеальные условия: частоты генераторов совпадают, их начальные фазы равны, а канал не вносит дополнительного фазового сдвига. Если передатчик использует косинус, а единственный смеситель приёмника — синус, то на выходе смесителя получаем

$$2s_{\mathrm{RF}}(t)\sin(\Omega_0 t) = a(t)\sin(2\Omega_0 t). \tag{4.4}$$

В произведении нет составляющей в базовой полосе. Высокочастотная составляющая подавляется фильтром нижних частот, поэтому

$$\mathrm{LPF}\{2s_{\mathrm{RF}}(t)\sin(\Omega_0 t)\} = 0. \tag{4.5}$$

Тот же результат можно получить в частотной области. Спектры косинусного и синусного опорных сигналов равны соответственно (рис. 4.1, а)

$$C(j\Omega) = \pi\left[\delta(\Omega-\Omega_0) + \delta(\Omega+\Omega_0)\right], \tag{4.6}$$

$$S(j\Omega) = \frac{\pi}{\mathrm{j}}\left[\delta(\Omega-\Omega_0) - \delta(\Omega+\Omega_0)\right]. \tag{4.7}$$

Умножение во времени соответствует свёртке спектров. Две копии спектра $A(j\Omega)$, перенесённые к нулевой частоте синусным смесителем, имеют противоположные знаки и взаимно компенсируются (рис. 4.1, б). Энергия входного сигнала при этом никуда не исчезает: после свёртки остаются составляющие около $\pm 2\Omega_0$, которые подавляются фильтром нижних частот, тогда как составляющая около нуля отсутствует (рис. 4.1, в). Таким образом, одноканальный приёмник измеряет только одну проекцию сигнала, и в выбранной геометрии эта проекция равна нулю.

Идеальное совпадение фаз в реальной линии связи маловероятно. Пусть $\varphi_{\mathrm{TX}}(t)$ включает фазу передатчика и фазовый сдвиг канала, а $\varphi_{\mathrm{LO}}(t)$ — фазу гетеродина приёмника. Обозначим их разность через

$$\Delta\varphi(t) = \varphi_{\mathrm{TX}}(t) - \varphi_{\mathrm{LO}}(t). \tag{4.8}$$

После умножения на косинусный опорный сигнал и фильтрации одноканальный приёмник выдаёт

$$y_I(t) = a(t)\cos(\Delta\varphi(t)). \tag{4.9}$$

При $\Delta\varphi = \pm\pi/2$ его выход снова обращается в нуль. Если частоты генераторов немного различаются, $\Delta\varphi(t)$ непрерывно изменяется, а амплитуда на выходе периодически ослабевает, меняет знак и снова возрастает. Следовательно, в реальной системе проблема проявляется не как постоянное исчезновение сигнала, а как фазовые замирания и искажения.

Для FM-сигнала ситуация несколько отличается. Его амплитуда почти постоянна, а информация заключена в законе изменения фазы:

$$s_{\mathrm{FM}}(t) = A\cos(\Omega_0 t + \varphi_m(t))\,. \tag{4.10}$$

Одна вещественная проекция не позволяет однозначно восстановить $\varphi_m(t)$ на всём интервале $2\pi$ и определить направление её изменения. Поэтому FM-сигнал в одноканальном ZIF-тракте не обязан исчезнуть полностью, но его корректная демодуляция в общем случае невозможна. Кроме того, обычные аналоговые AM- и FM-детекторы, рассчитанные на ненулевую ПЧ, нельзя без изменений поставить после ZIF-смесителя: на его выходе уже нет прежней гармонической несущей.

### 4.2.2 Квадратурный приём и передача

Чтобы исключить фазовую слепую зону (устранить фазовую неоднозначность), в приёмник вводят второй смеситель. На первый подаётся косинусный опорный сигнал, а на второй — опорный сигнал, сдвинутый на 90°, — синус. При принятом далее соглашении

$$I(t) = \mathrm{LPF}\{s_{\mathrm{RF}}(t)\cos(\Omega_0 t)\}\,, \tag{4.11}$$

$$Q(t) = \mathrm{LPF}\{s_{\mathrm{RF}}(t)\sin(\Omega_0 t)\}\,. \tag{4.12}$$

Для входного сигнала с амплитудой $a(t)$ получаются две ортогональные проекции:

$$I(t) = \frac{1}{2}a(t)\cos(\Delta\varphi(t))\,, \tag{4.13}$$

$$Q(t) = \frac{1}{2}a(t)\sin(\Delta\varphi(t))\,. \tag{4.14}$$

Если одна из них равна нулю, вся амплитуда наблюдается во второй.

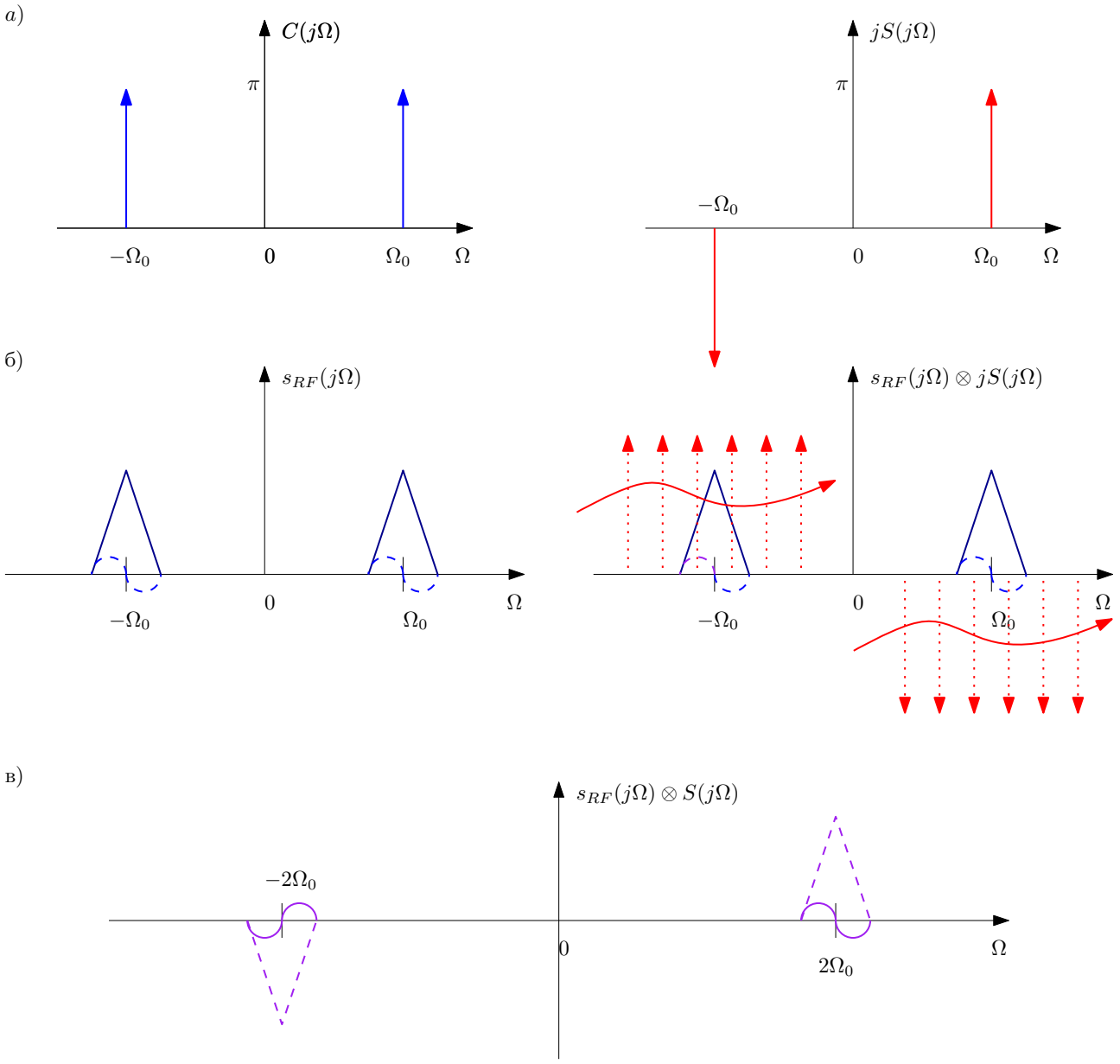



**Рис. 4.1:** Спектральное представление переноса косинусного сигнала синусным смесителем: а) спектры косинусного сигнала и синусного опорного сигнала (для наглядности второй показан в виде $jS(j\Omega)$); б) формирование сдвинутых копий спектра при свёртке; в) результирующие составляющие около $\pm 2\Omega_0$ при отсутствии составляющей в базовой полосе. Сплошной чертой обозначена вещественная часть спектра, пунктирной — мнимая.

Совместная обработка двух каналов позволяет восстановить амплитуду и фазу:

$$a(t) = 2\sqrt{I^2(t) + Q^2(t)}, \qquad \varphi(t) = \operatorname{atan2}(Q(t), I(t))\,. \tag{4.15}$$

Для AM информация извлекается преимущественно из $A(t)$, а для FM — из изменения $\varphi(t)$. В дискретном времени простейшая оценка мгновенного частотного отклонения имеет вид

$$\widehat{f}[n] = \frac{f_s}{2\pi} \arg(x[n]x^*[n-1])\,. \tag{4.16}$$

**Важно**

Второй квадратурный канал сначала появился как средство устранения фазовой неопределённости. Однако синус и косинус ортогональны, поэтому те же две ветви можно использовать для одновременной передачи двух независимых вещественных последовательностей $u(t)$ и $v(t)$:

$$s_{\mathrm{RF}}(t) = u(t)\cos(\Omega_0 t) + v(t)\sin(\Omega_0 t). \tag{4.17}$$

При точной частотной и фазовой синхронизации квадратурный демодулятор восстанавливает $I(t) = u(t)$ и $Q(t) = -v(t)$: перекрёстные произведения после умножения переносятся к $2\Omega_0$ и подавляются ФНЧ. Так одна и та же несущая и одна и та же полоса частот получают две независимые координаты. На этом принципе основаны квадратурное уплотнение, фазовая манипуляция и QAM.

Произвольные AM- и FM-сигналы нельзя без дополнительных условий считать двумя независимыми квадратурными каналами: FM является нелинейной фазовой модуляцией и в общем случае использует обе компоненты. Для квадратурного уплотнения в ветви $I$ и $Q$ подают непосредственно независимые модулирующие последовательности.

Структурные схемы квадратурных модулятора и демодулятора приведены на рис. 4.2

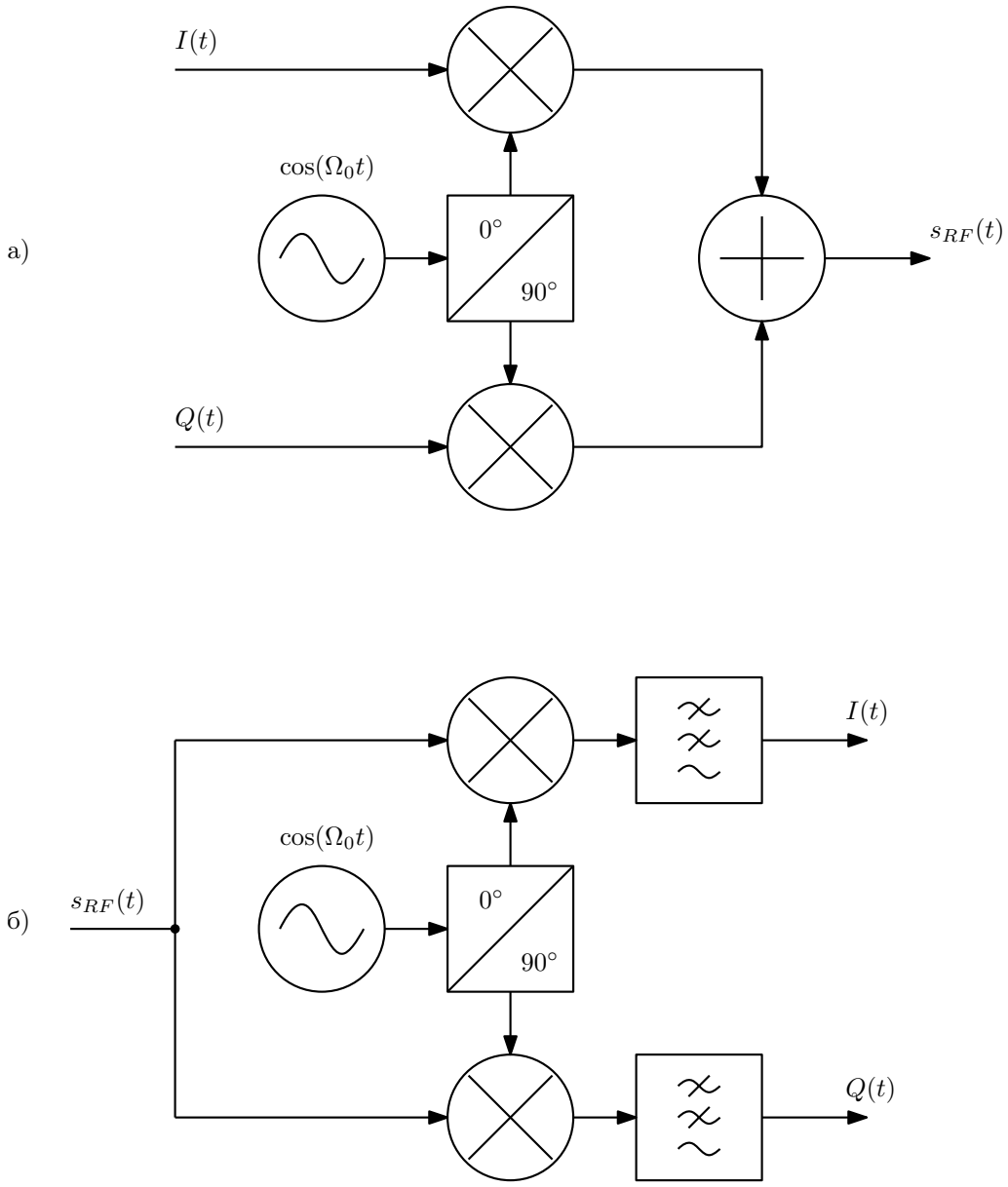



**Рис. 4.2:** Квадратурные модулятор и демодулятор: а) модулятор; б) демодулятор

### 4.2.3 Комплексная огибающая и аналитический сигнал

Две вещественные квадратурные компоненты удобно объединить в одно комплексное число

$$x(t) = I(t) + \mathrm{j}Q(t), \qquad \mathrm{j}^2 = -1. \tag{4.18}$$

В частотной области этому равенству соответствует

$$X(j\Omega) = I(j\Omega) + \mathrm{j}Q(j\Omega). \tag{4.19}$$

Следовательно, спектры $I$- и $Q$-компонент складываются не как две вещественные кривые: $I(j\Omega)$ образует действительную, а $\mathrm{j}Q(j\Omega)$ — мнимую часть комплексного спектра (рис. 4.3, а). Даже если модуль $|X(j\Omega)|$ симметричен относительно нуля, это ещё не означает, что $x(t)$ вещественен. Для вещественного сигнала должно выполняться более строгое условие $X(-j\Omega) = X^*(j\Omega)$.

Комплексная запись не означает существования мнимого напряжения в физической цепи. Она лишь объединяет две одновременно измеряемые вещественные координаты. При принятом соглашении действительный радиосигнал восстанавливается как

$$s_{\mathrm{RF}}(t) = \mathrm{Re}\{x(t)\exp(\mathrm{j}\Omega_0 t)\} = I(t)\cos(\Omega_0 t) - Q(t)\sin(\Omega_0 t). \tag{4.20}$$

Используя свойства частотного переноса, спектр радиосигнала можно записать в развёрнутом виде:

$$\begin{aligned} S_{\mathrm{RF}}(j\Omega) = {} & \frac{1}{2}\left[I(j(\Omega-\Omega_0)) + I(j(\Omega+\Omega_0))\right] \\ & + \frac{\mathrm{j}}{2}\left[Q(j(\Omega-\Omega_0)) - Q(j(\Omega+\Omega_0))\right]. \end{aligned} \tag{4.21}$$

Около $+\Omega_0$ находится сдвинутая копия $X(j(\Omega-\Omega_0))/2$, а около $-\Omega_0$ — её зеркально-сопряжённая копия. Раздельный перенос $I$- и $Q$-компонент и результат их объединения показаны соответственно на рис. 4.3, б и рис. 4.3, в. Рисунок является качественным, поэтому коэффициент $1/2$, возникающий при умножении на синус или косинус, на нём не отражён.

Для дискретной последовательности амплитуда и фаза вычисляются аналогично:

$$A[n] = |x[n]| = \sqrt{I^2[n] + Q^2[n]}, \qquad \varphi[n] = \operatorname{atan2}(Q[n], I[n])\,. \tag{4.22}$$

Функция atan2 учитывает знаки обеих компонент и правильно выбирает квадрант фазы. Обычная функция $\arctan(Q/I)$ этой информацией не располагает и, кроме того, не определена при $I = 0$.

Связь комплексного представления со спектром устанавливается с помощью аналитического сигнала. Для любого вещественного сигнала выполняется сопряжённая симметрия

$$S_{\mathrm{RF}}(-j\Omega) = S^{*}_{\mathrm{RF}}(j\Omega). \tag{4.23}$$

Следовательно, отрицательная половина спектра является избыточной зеркальной копией положительной. Аналитический сигнал определяют как

$$s_a(t) = s_{\mathrm{RF}}(t) + \mathrm{j}\widehat{s}_{\mathrm{RF}}(t), \tag{4.24}$$

где $\widehat{s}_{\mathrm{RF}}(t)$ — преобразование Гильберта исходного сигнала. Во временной области оно определяется главным значением интеграла

$$\widehat{s}_{\mathrm{RF}}(t) = \mathcal{H}\{s_{\mathrm{RF}}(t)\} = \frac{1}{\pi}\,\mathrm{p.\,v.}\int_{-\infty}^{\infty} \frac{s_{\mathrm{RF}}(\tau)}{t-\tau}\,\mathrm{d}\tau. \tag{4.25}$$

В частотной области преобразование Гильберта не изменяет модуль спектра, но сдвигает фазы положительных частот на $-\pi/2$, а отрицательных — на $+\pi/2$:

$$\widehat{S}_{\mathrm{RF}}(j\Omega) = -\mathrm{j}\ \mathrm{sgn}(\Omega) S_{\mathrm{RF}}(j\Omega). \tag{4.26}$$

Отсюда непосредственно следуют равенства

$$\mathrm{j}\widehat{S}_{\mathrm{RF}}(j\Omega) = \mathrm{sgn}(\Omega) S_{\mathrm{RF}}(j\Omega), \qquad S_a(j\Omega) = [1 + \mathrm{sgn}(\Omega)]\, S_{\mathrm{RF}}(j\Omega). \tag{4.27}$$

При сложении $S_{\mathrm{RF}}(j\Omega)$ и $\mathrm{j}\widehat{S}_{\mathrm{RF}}(j\Omega)$ отрицательные частоты взаимно уничтожаются, а положительные удваиваются (рис. 4.3, в). Поэтому спектр аналитического сигнала имеет вид

$$S_a(j\Omega) = \begin{cases} 2S_{\mathrm{RF}}(j\Omega), & \Omega > 0, \\ S_{\mathrm{RF}}(0), & \Omega = 0, \\ 0, & \Omega < 0. \end{cases} \tag{4.28}$$

Удаление отрицательной половины не приводит к потере информации: для вещественного сигнала она полностью определяется положительной. Перенос аналитического сигнала с $\Omega_0$ к нулю даёт комплексную огибающую

$$x(t) = s_a(t)\exp(-\mathrm{j}\Omega_0 t). \tag{4.29}$$

В частотной области прямой и обратный переносы записываются как

$$X(j\Omega) = S_a(j(\Omega + \Omega_0))\,, \qquad S_a(j\Omega) = X(j(\Omega - \Omega_0))\,. \tag{4.30}$$

Именно после этого переноса спектр общего комплексного сигнала уже не обязан быть симметричным относительно нуля. Участок радиоспектра ниже $\Omega_0$ занимает отрицательные частоты комплексной базовой полосы, а участок выше $\Omega_0$ — положительные. Аналитический сигнал является математической конструкцией; аппаратный квадратурный приёмник получает эквивалентную информацию с помощью двух смесителей. Полный переход от исходных $I$- и $Q$-компонент к аналитическому сигналу и обратно к комплексной огибающей показан на рис. 4.3, г.

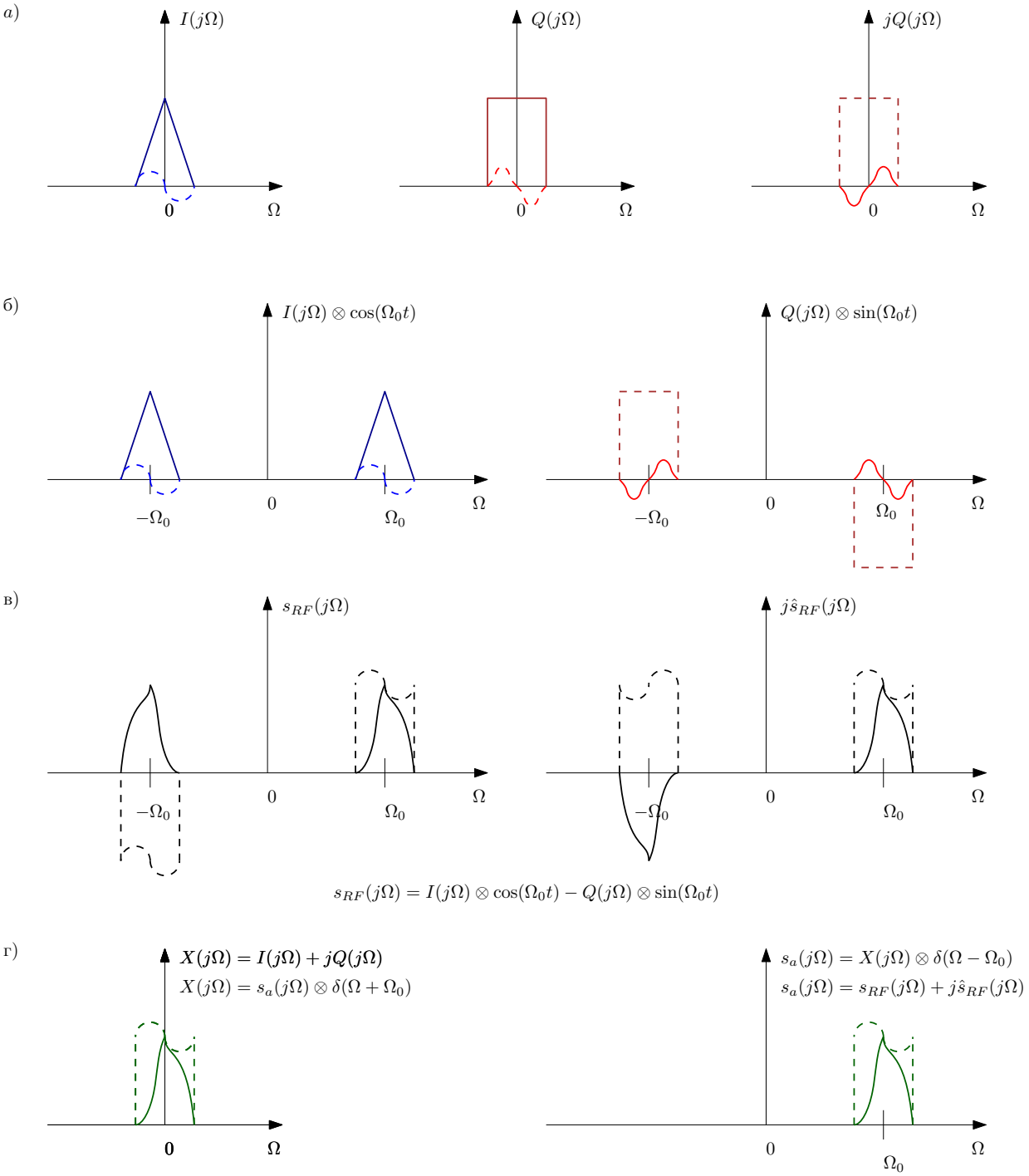



**Рис. 4.3:** Спектральное представление комплексной огибающей и аналитического сигнала: а) формирование $X(j\Omega) = I(j\Omega) + \mathrm{j}Q(j\Omega)$; б) перенос спектров квадратурных компонент к $\pm\Omega_0$; в) спектр вещественного радиосигнала и действие преобразования Гильберта; г) односторонний спектр аналитического сигнала и его перенос к нулевой частоте

### 4.2.4 Что означает нулевая частота

Нулевая частота комплексной базовой полосы соответствует не постоянному напряжению на антенне, а центральной частоте настройки SDR $f_0$. Компонента с частотой $f_c$ в массиве IQ-отсчётов соответствует радиочастоте

$$f_{\mathrm{RF}} = f_0 + f_c. \tag{4.31}$$

Например, при $f_0 = 100$ МГц линия $f_c = -1{,}2$ МГц соответствует 98,8 МГц, а линия $f_c = +1{,}2$ МГц — 101,2 МГц. Абсолютная центральная частота в самих значениях IQ-отсчётов не закодирована, поэтому $f_0$ необходимо сохранять вместе с записью.

В идеальном комплексном представлении при частоте дискретизации $f_s$ без неоднозначности описывается интервал

$$\begin{aligned} -\frac{\Omega_s}{2} \le \Omega < \frac{\Omega_s}{2}, \qquad \Omega_s = 2\pi f_s, \\ -\frac{f_s}{2} \le f < \frac{f_s}{2}. \end{aligned} \tag{4.32}$$

Полная ширина интервала равна $\Omega_s$ рад/с, или $f_s$ Гц. Так, комплексная запись с $f_s = 8$ МГц охватывает смещения от $-4$ до $+4$ МГц относительно $f_0$. Реально используемая полоса дополнительно ограничена аналоговыми фильтрами, полосой трансивера и переходными областями цифровых фильтров.

**Внимание**

Знак компоненты $Q$ зависит от соглашений конкретного трансивера и библиотеки. Формула (4.20) фиксирует соглашение этого методического пособия. При замене $Q[n]$ на $-Q[n]$ спектр отражается относительно нулевой частоты: верхняя и нижняя стороны относительно $f_0$ меняются местами.

### 4.2.5 Комплексный тон, NCO и цифровая настройка

Комплексный тон — одна из важнейших последовательностей в ЦОС:

$$x[n] = A \exp\left( \mathrm{j} \left( 2\pi \frac{f_c}{f_s} n + \varphi_0 \right) \right). \tag{4.33}$$

Его действительная и мнимая части равны

$$Re\{x[n]\} = A\cos\left(2\pi\frac{f_c}{f_s}n + \varphi_0\right), \tag{4.34}$$

$$Im\{x[n]\} = A\sin\left(2\pi\frac{f_c}{f_s}n + \varphi_0\right). \tag{4.35}$$

При $f_c > 0$ вектор $x[n]$ вращается на комплексной плоскости против часовой стрелки, а при $f_c < 0$ — по часовой стрелке. У одного действительного гармонического колебания различить эти направления нельзя: его спектр всегда содержит пару сопряжённых линий на $+f_c$ и $-f_c$.

Комплексный тон используется в SDR в двух основных ролях. Во-первых, это диагностический сигнал, с помощью которого проверяют знак частоты, порядок $I/Q$, масштаб амплитуды и расположение FFT-бинов. Во-вторых, комплексный тон с единичной амплитудой является сигналом численно управляемого генератора (NCO):

$$q[n] = \exp(\mathrm{j}\varphi_{\mathrm{NCO}}[n]), \qquad \varphi_{\mathrm{NCO}}[n] = \varphi_0 + 2\pi\frac{f_{\mathrm{NCO}}}{f_s}n. \tag{4.36}$$

NCO служит цифровым гетеродином. Умножение последовательности на комплексную экспоненту переносит весь её спектр без взаимного наложения положительных и отрицательных частот. При соглашениях этого раздела операция

$$y[n] = x[n]\exp\left(-\mathrm{j}2\pi\frac{f_{\mathrm{shift}}}{f_s}n\right) \tag{4.37}$$

перемещает компоненту с $f_c$ на $f_c - f_{\mathrm{shift}}$. Это непосредственно следует из сложения показателей комплексных экспонент:

$$\exp\left(\mathrm{j}2\pi\frac{f_c}{f_s}n\right)\exp\left(-\mathrm{j}2\pi\frac{f_{\mathrm{shift}}}{f_s}n\right) = \exp\left(\mathrm{j}2\pi\frac{f_c - f_{\mathrm{shift}}}{f_s}n\right). \tag{4.38}$$

Рассмотрим практический пример. Приёмник настроен на $f_0 = 100$ МГц и записывает комплексную полосу шириной 8 МГц, то есть $f_s = 8$ МГц. Интересующая FM-радиостанция занимает условную полосу 100 кГц и наблюдается в IQ-спектре со смещением $f_c = +1{,}2$ МГц. Её абсолютная центральная частота равна 101,2 МГц.

Обрабатывать все 8 МГц ради одной станции невыгодно. Поэтому цифровой тракт выполняет три последовательные операции:

1. NCO с $f_{\text{shift}} = +1{,}2$ МГц переносит выбранную станцию в окрестность нулевой частоты;
2. ФНЧ выделяет её полосу и подавляет остальные станции;
3. после ограничения полосы частота дискретизации уменьшается, например, с 8 МГц до 250 кГц децимацией в 32 раза.

Первый этап является цифровой настройкой на станцию внутри уже принятой широкой полосы. Для выбора другой станции не требуется менять аппаратную частоту $f_0$: достаточно изменить частоту NCO, если новая станция остаётся в пределах захваченного интервала $[-4; +4)$ МГц.

На практике фазу NCO удобно обновлять рекуррентно:

$$\varphi_{\text{NCO}}[n+1] = \varphi_{\text{NCO}}[n] + \Delta\varphi, \tag{4.39}$$

$$\Delta\varphi = 2\pi \frac{f_{\text{NCO}}}{f_s}. \tag{4.40}$$

Эквивалентно можно рекуррентно обновлять сам комплексный отсчёт генератора:

$$q[n+1] = q[n] \exp(\mathrm{j}\Delta\varphi). \tag{4.41}$$

Фазовое состояние NCO должно сохраняться между соседними блоками. Если в начале каждого буфера вновь задавать $\varphi_{\text{NCO}} = 0$, на границах возникнут разрывы фазы и паразитные спектральные составляющие. При длительной работе фазу периодически приводят к ограниченному интервалу, например $[-\pi; \pi)$, либо нормируют модуль рекуррентно вычисляемого $q[n]$.

**Внимание**

Частотный перенос не является фильтрацией. Умножение на NCO только сдвигает спектр, который в дискретном времени периодически продолжается с периодом $f_s$. Перед децимацией выбранную полосу необходимо выделить ФНЧ, иначе в неё наложатся внеполосные компоненты. Знак переноса следует проверять комплексным контрольным тоном: в разных библиотеках могут использоваться противоположные соглашения о знаке $Q$ и направлении частотной оси.

**Контрольная точка**

Приёмник настроен на $f_0 = 100$ МГц и формирует IQ-поток с $f_s = 4$ МГц:

$$x[n] = 0{,}6\exp(\mathrm{j}2\pi\,0{,}18n) + 0{,}25\exp\!\left(-\mathrm{j}2\pi\,0{,}31n + \mathrm{j}\frac{\pi}{4}\right).$$

1. Определите смещения обеих спектральных линий в герцах и их абсолютные радиочастоты.
2. Какой комплексной экспонентой нужно умножить последовательность, чтобы первую линию перенести к нулю? Где после этого окажется вторая линия?
3. Предскажите, как изменятся знаки частот при перестановке компонент $I$ и $Q$. Используйте равенство $Q[n] + \mathrm{j}I[n] = \mathrm{j}x^*[n]$.
4. * Начиная с $s_{\mathrm{RF}}(t) = I(t)\cos(\Omega_0 t) - Q(t)\sin(\Omega_0 t)$, получите выражение для полосы около $+\Omega_0$ и объясните, почему полоса около $-\Omega_0$ является её зеркально-сопряжённой копией.

Ответы для сверки: смещения равны $+720$ кГц и $-1{,}24$ МГц, абсолютные частоты — 100,72 и 98,76 МГц. Умножение на $\exp(-\mathrm{j}2\pi\,0{,}18n)$ переносит линии в 0 и $-1{,}96$ МГц. Перестановка $I/Q$ отражает частотную ось. Около $+\Omega_0$ получается $X(j(\Omega - \Omega_0))/2$; отрицательная полоса определяется условием вещественности $S_{\mathrm{RF}}(-j\Omega) = S^*_{\mathrm{RF}}(j\Omega)$.

## 4.3 Дискретизация и алиасинг

С математической точки зрения дискретизацию удобно представить в виде двух операций: умножения сигнала на периодическую последовательность импульсов и последующего преобразования взвешенных импульсов в последовательность чисел. Периодическая последовательность импульсов имеет вид

$$s(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT). \tag{4.42}$$

где $\delta(t)$ — $\delta$-функция Дирака. Модулируя сигнал $x(t)$ функцией $s(t)$, получаем

$$x_s(t) = x(t)s(t) = \sum_{n=-\infty}^{\infty} x(nT)\delta(t - nT). \tag{4.43}$$

Известно, что преобразование Фурье от периодической последовательности импульсов также является периодической последовательностью импульсов, т. е.

$$S(j\Omega) = \frac{2\pi}{T} \sum_{k=-\infty}^{\infty} \delta(\Omega - k\Omega_s). \tag{4.44}$$

По свойству преобразования Фурье получаем

$$X_s(j\Omega) = \frac{1}{2\pi} X(j\Omega) \bigotimes S(j\Omega), \tag{4.45}$$

где $\bigotimes$ обозначает свёртку по непрерывной переменной. Тогда

$$X_s(j\Omega) = \frac{1}{T} \sum_{k=-\infty}^{\infty} X(j(\Omega - k\Omega_s)) \,. \tag{4.46}$$

После дискретизации спектр становится периодическим с периодом $f_s$ (или $\Omega_s$ в зависимости от того, какую частоту мы взяли для построения графиков спектра) — см. рис. 4.4, а, б, в.

Любые две непрерывные частоты, отличающиеся на целое число частот дискретизации, порождают одну и ту же дискретную последовательность:

$$\exp\left(\mathrm{j}2\pi \frac{f_c + m f_s}{f_s} n\right) = \exp\left(\mathrm{j}2\pi \frac{f_c}{f_s} n\right), \qquad m \in \mathbb{Z}. \tag{4.47}$$

Частоту любого комплексного тона можно привести к основному интервалу (4.32), вычитая или прибавляя $f_s$. В случае если условие Котельникова–Найквиста не выполняется, наблюдается эффект наложения спектров. Например, тон $0{,}60 f_s$ после дискретизации наблюдается как тон $-0{,}40 f_s$, а тон $-0{,}75 f_s$ — как $+0{,}25 f_s$. Такое наложение спектральных копий называется алиасингом — рис. 4.4, г.

### 4.3.1 Почему одной настройки $f_s$ недостаточно

Соблюдение условия Котельникова–Найквиста не защищает АЦП от внеполосных сигналов. До дискретизации спектр должен быть ограничен аналоговым фильтром, иначе мощная помеха из соседней спектральной копии (побочных зон Найквиста) попадёт в рабочую полосу уже необратимо.

После АЦП дополнительные цифровые фильтры позволяют сузить полосу перед децимацией, но не могут отделить друг от друга компоненты,

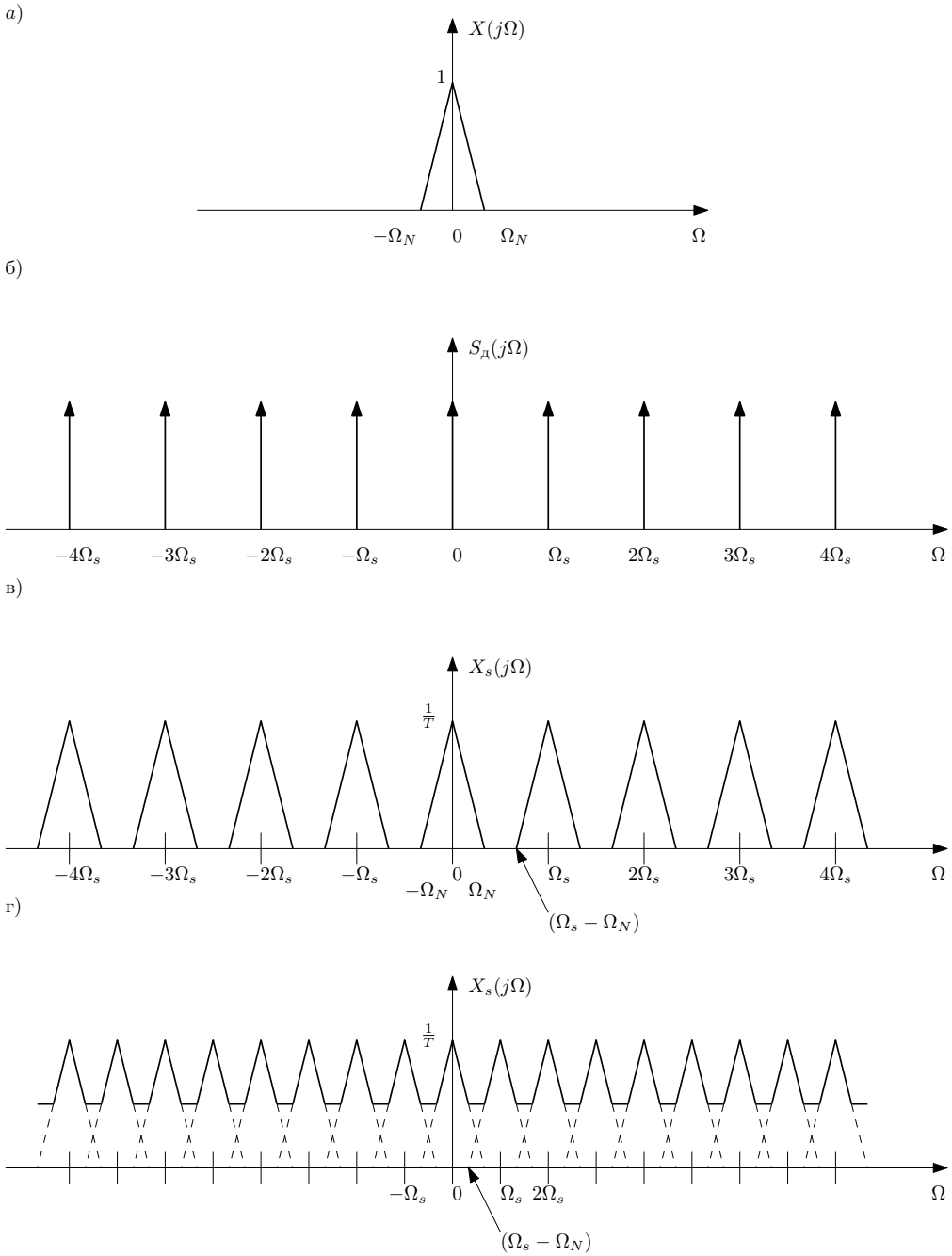



**Рис. 4.4:** Спектральное представление дискретизации: а) спектр исходного полосоограниченного сигнала; б) спектр последовательности дискретизирующих импульсов; в) спектральные копии не перекрываются при $\Omega_s > 2\Omega_N$; г) перекрытие копий и возникновение алиасинга при $\Omega_s < 2\Omega_N$

которые уже наложились при первичной дискретизации.

**Важно**

В SDR следует отдельно фиксировать как минимум:

- центральную частоту настройки $f_0$;
- частоту комплексных отсчётов на выходе драйвера $f_s$;
- ожидаемую занимаемую полосу сигнала;
- полосу аналогового тракта или фильтра, если она настраивается отдельно;
- коэффициенты последующей программной децимации или интерполяции.

Эти величины связаны, но не взаимозаменяемы. Увеличение $f_s$ не расширяет автоматически аналоговую полосу платы, а широкая аналоговая полоса не означает, что компьютер способен принять такой поток без потерь.

На рис. 4.5 показано экспериментальное проявление алиасинга при намеренно расширенной полосе аналогового фильтра. Этот рисунок хорошо демонстрирует реальное проявление алиасинга: в эксперименте полоса антиалайзингового фильтра намеренно выбрана намного шире первой зоны Найквиста ($f_s = 8$ МГц), и мы видим, что сигналы FM-станций, лежащих в соседних зонах Найквиста, «завернулись» в первую зону. Так, сигнал LOVE-радио, который расположен на частоте 105,3 МГц, вследствие алиасинга отобразился на $105{,}3 + 8 = 113{,}3$ МГц. Положение линии на таком экране само по себе не доказывает отсутствие алиасинга: для интерпретации необходимо знать центральную частоту, $f_s$ и характеристики аналоговых и цифровых фильтров.

**Внимание**

Алиасинг не всегда выглядит как очевидное искажение. Узкая спектральная линия может остаться узкой и устойчивой, но появиться на другой частоте. Поэтому проверка должна включать не только форму спектра, но и заранее вычисленное ожидаемое положение каждой контрольной компоненты.

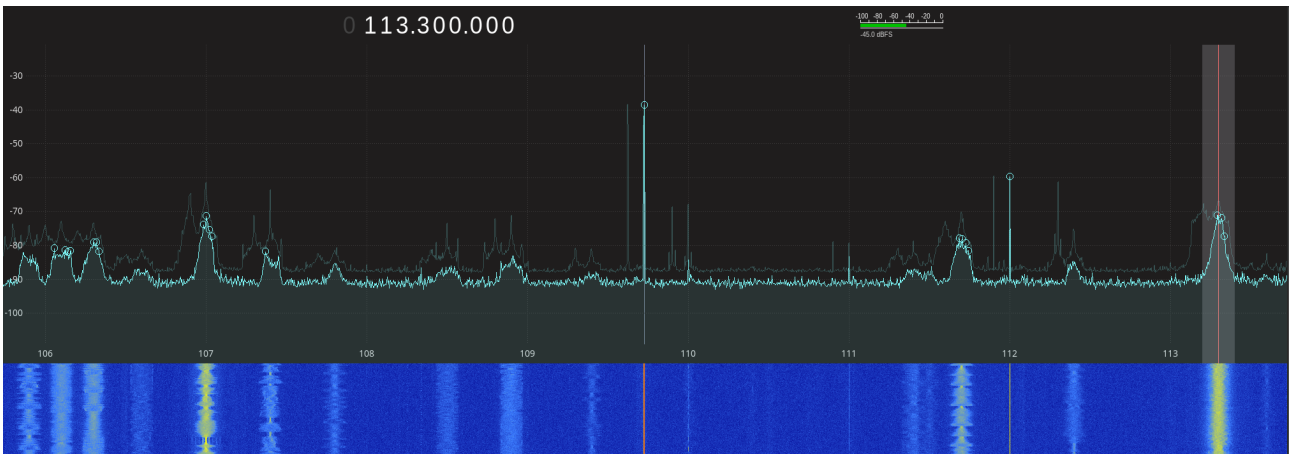



**Рис. 4.5:** Пример алиасинга при обзоре спектра и водопада в SDR++. Наблюдаемое изображение является результатом аналоговой фильтрации, дискретизации, частотного переноса и последующей цифровой обработки

### Контрольная точка

Комплексный АЦП работает с частотой $f_s = 8$ МГц. До дискретизации во входной полосе присутствуют тоны со смещениями $+1{,}1$, $+5{,}4$, $-6{,}3$ и $+12{,}2$ МГц относительно частоты настройки.

1. Приведите все четыре частоты к интервалу $[-4; 4)$ МГц.
2. Для каждой наблюдаемой линии запишите семейство аналоговых частот, которые после дискретизации невозможно различить только по её положению.
3. Для полезной полосы $[-1{,}5; +1{,}5]$ МГц задайте требования к полосе пропускания и началу полосы подавления аналогового антиалиасингового фильтра.
4. * Помеха со смещением $+13{,}7$ МГц имеет уровень $-5$ dBFS. Фильтр ослабляет её на 45 дБ. Найдите наблюдаемую частоту и уровень после алиасинга. Какое ослабление необходимо, чтобы помеха оказалась не выше $-75$ dBFS?

Ответы для сверки: $+1{,}1$, $-2{,}6$, $+1{,}7$ и $-3{,}8$ МГц. Все частоты, различающиеся на $mf_s$, образуют неразличимые пары. Фильтр должен без искажений пропускать полезную полосу и обеспечить требуемое подавление до 4 МГц. В задаче * помеха появится на $-2{,}3$ МГц с уровнем $-50$ dBFS; для уровня не выше $-75$ dBFS требуется не менее 70 дБ ослабления.

## 4.4 Децимация и интерполяция

**Важно**

Многоскоростная обработка изменяет частоту дискретизации сигнала при сохранении его физической временной шкалы. Такое преобразование нельзя свести только к удалению или добавлению элементов массива: оно обязательно включает фильтрацию, предотвращающую наложение или появление спектральных копий.

### 4.4.1 Децимация

Децимацией с целочисленным коэффициентом $D$ называется уменьшение частоты дискретизации в $D$ раз. Операция состоит из двух этапов: предварительной антиалиасинговой фильтрации и последующего прореживания последовательности. Если $h_D[n]$ — импульсная характеристика фильтра, то

$$v[n] = \sum_{\ell=-\infty}^{\infty} h_D[\ell]x[n-\ell], \tag{4.48}$$

$$y[m] = v[mD], \qquad f_{s,\text{out}} = \frac{f_{s,\text{in}}}{D}. \tag{4.49}$$

Прореживание само по себе не удаляет высокочастотные составляющие. Его действие в частотной области описывается выражением

$$Y\left(e^{\mathrm{j}\omega}\right) = \frac{1}{D}\sum_{r=0}^{D-1} V\left(e^{\mathrm{j}(\omega+2\pi r)/D}\right). \tag{4.50}$$

Следовательно, после прореживания складываются $D$ сжатых копий исходного спектра. Чтобы они не перекрывались, фильтр должен подавить составляющие вне нового интервала Найквиста $[-f_{s,\text{out}}/2, f_{s,\text{out}}/2)$. Компоненты, не подавленные до децимации, необратимо накладываются на полезную полосу.

Например, при переходе с 4 до 1 Мвыб/с коэффициент $D = 4$, а новая граница Найквиста равна 500 кГц. Если полезный комплексный сигнал занимает ±400 кГц, фильтр может иметь полосу пропускания до 400 кГц и должен достаточно подавить помехи к 500 кГц. Интервал между этими частотами является переходной полосой, от ширины которой существенно зависит

длина фильтра. Последовательность операций при децимации показана на рисунке 4.6.

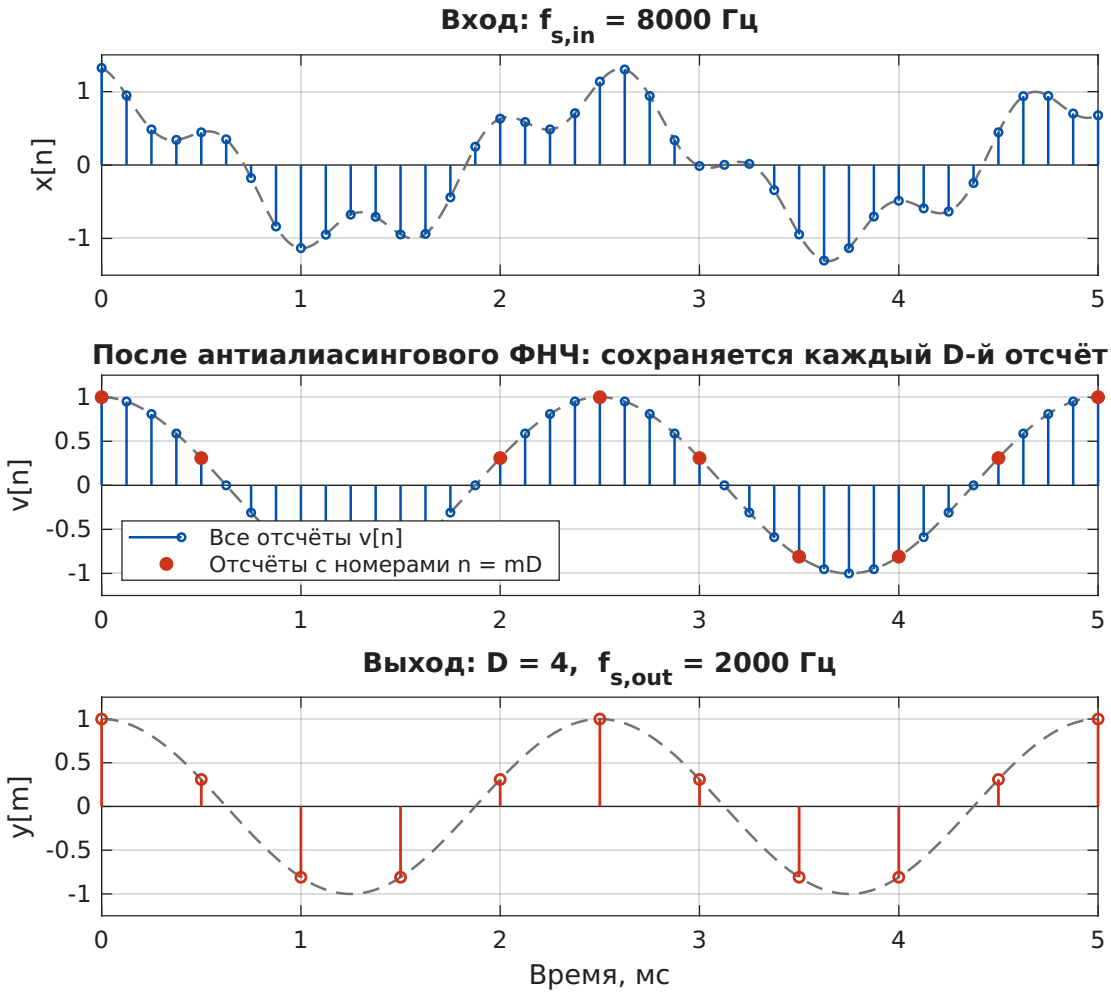



**Рис. 4.6:** Децимация с коэффициентом $D = 4$. После антиалиасинговой фильтрации из последовательности $v[n]$ сохраняется каждый четвёртый отсчёт; выходная последовательность $y[m]$ описывает тот же сигнал на более редкой временной сетке

### 4.4.2 Интерполяция

Интерполяцией называется вычисление значений сигнала на более плотной временной сетке в соответствии с выбранной моделью восстановления. Линейные, кубические и сплайновые методы интерполяции используют различные кусочно-полиномиальные модели. В задачах многоскоростной ЦОС обычно предполагается, что исходный сигнал ограничен по полосе; тогда интерполяцию реализуют с помощью повышающего дискретизатора и интерполяционного фильтра.

При увеличении частоты дискретизации в $L$ раз сначала формируется промежуточная последовательность

$$u[n] = \begin{cases} x[n/L], & n \equiv 0 \pmod{L}, \\ 0, & \text{иначе}, \end{cases} \qquad f_{s,\text{out}} = L f_{s,\text{in}}. \tag{4.51}$$

Вставленные нули не являются оценками сигнала в промежуточные моменты времени. Это вспомогательное представление, удобное для дискретной реализации. Итоговые отсчёты вычисляет интерполяционный фильтр $h_I[n]$:

$$y[n] = \sum_{k=-\infty}^{\infty} u[k] h_I[n-k] = \sum_{m=-\infty}^{\infty} x[m] h_I[n-mL]. \tag{4.52}$$

Для идеального полосоограниченного восстановления

$$h_I[n] = \operatorname{sinc}\!\left(\frac{n}{L}\right), \qquad \operatorname{sinc}(x) = \frac{\sin(\pi x)}{\pi x}, \tag{4.53}$$

а частотная характеристика фильтра имеет коэффициент усиления $L$ в полосе $|\omega| \le \pi/L$. На практике бесконечную импульсную характеристику заменяют КИХ-фильтром конечной длины. Вставка нулей создаёт $L$ спектральных копий, поскольку

$$U\!\left(e^{\mathrm{j}\omega}\right) = X\!\left(e^{\mathrm{j}L\omega}\right), \tag{4.54}$$

а интерполяционный фильтр подавляет копии вне основной полосы. Кубические и сплайновые методы также можно трактовать как использование другого ядра интерполяции, однако они сами по себе не обеспечивают заданного подавления спектральных образов. Поэтому их применение в радиотехническом тракте требует отдельной оценки частотной характеристики. Последовательность повышения частоты дискретизации показана на рисунке 4.7.

Рациональное преобразование частоты $L/M$ строится как интерполяция в $L$ раз, фильтрация и децимация в $M$ раз. В эффективной полифазной реализации промежуточная последовательность с полной повышенной частотой явно не формируется, но математический результат остаётся тем же.

### 4.4.3 Ловушка нормированной частоты

Физическая частота $f$, измеряемая в герцах, и нормированная частота — разные величины. В этом методическом пособии используются частота в циклах на отсчёт $\nu$ и угловая частота в радианах на отсчёт $\omega$:

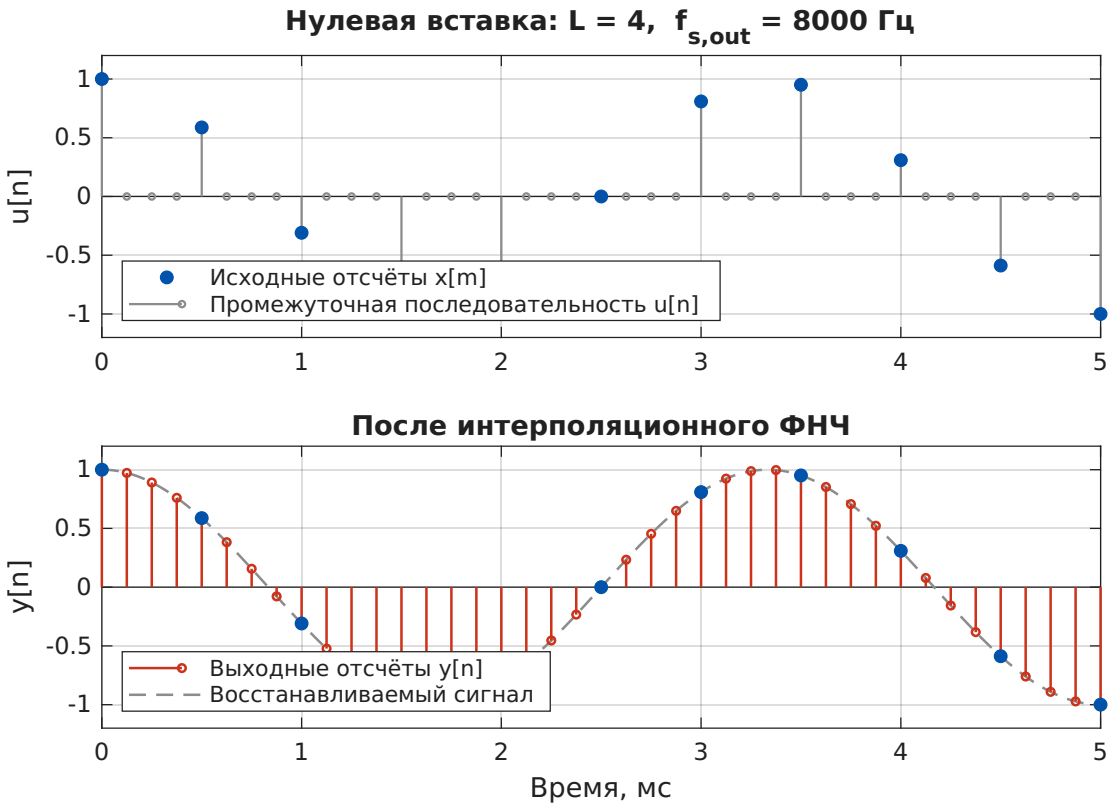



**Рис. 4.7:** Интерполяция с коэффициентом $L = 4$. Нулевая вставка формирует промежуточную последовательность $u[n]$, после чего интерполяционный фильтр вычисляет отсчёты $y[n]$ на более плотной временной сетке

$$\nu = \frac{f}{f_s}, \qquad \omega = 2\pi\nu = 2\pi\frac{f}{f_s}. \tag{4.55}$$

Встречается также нормировка относительно частоты Найквиста $\widehat{f} = f/(f_s/2) = 2\nu$. Поэтому численное значение нормированной частоты имеет смысл только вместе с явно указанным соглашением.

При корректном преобразовании частоты дискретизации физическая частота составляющей не изменяется, но её нормированная частота изменяется по закону

$$\nu_{\text{out}} = \begin{cases} D\nu_{\text{in}}, & \text{при децимации в } D \text{ раз}, \\ \nu_{\text{in}}/L, & \text{при интерполяции в } L \text{ раз}. \end{cases} \tag{4.56}$$

**Внимание**

Первое соотношение справедливо для составляющих, сохранённых антиалиасинговым фильтром. Если новая граница Найквиста нарушена, частота отображается в основную полосу с алиасингом. По этой причине

коэффициенты цифровых фильтров, частоты среза и шаг фазового аккумулятора NCO нельзя переносить между участками тракта без пересчёта относительно действующего $f_s$.

**Контрольная точка**

Тракт должен преобразовать комплексный поток с $f_{s,\text{in}} = 9{,}6$ МГц в поток с $f_{s,\text{out}} = 600$ кГц. Полезный сигнал занимает полосу $[-180; +180]$ кГц, а внеполосный тон находится на $+1{,}1$ МГц.

1. Найдите общий коэффициент децимации и проверьте, помещается ли полезный сигнал в новую зону Найквиста.
2. Вычислите нормированные границы полезной полосы до и после преобразования как в циклах, так и в радианах на отсчёт.
3. Где появится тон $+1{,}1$ МГц, если выполнить прореживание без фильтрации? Задайте полосу пропускания и предельную границу переходной полосы требуемого ФНЧ.
4. * Для преобразования 2,4 МГц в 1 МГц найдите несократимые коэффициенты $L/M$, порядок операций и нормированные границы полосы пропускания 180 кГц и полосы подавления 500 кГц на промежуточной частоте дискретизации.

Ответы для сверки: $D = 16$, новая граница Найквиста равна 300 кГц; полезная граница изменяется с 0,01875 до 0,3 цикла на отсчёт, или с $0{,}0375\pi$ до $0{,}6\pi$ рад/отсчёт. Без фильтра тон появится на $-100$ кГц; ФНЧ должен пропускать $|f| \le 180$ кГц и обеспечить требуемое подавление не позднее $|f| = 300$ кГц. В задаче * $L/M = 5/12$: повышение частоты в пять раз, фильтрация и децимация в двенадцать раз. Промежуточная частота равна 12 МГц, а нормированные границы — 0,015 и $1/24$ цикла на отсчёт, то есть $0{,}03\pi$ и $\pi/12$ рад/отсчёт.

## 4.5 Дискретное преобразование Фурье

Дискретное преобразование Фурье (ДПФ) переводит конечный блок отсчётов в набор комплексных спектральных коэффициентов. В этом методическом пособии используется соглашение

$$X[k] = \sum_{n=0}^{N-1} x[n] \exp\left(-\mathrm{j}2\pi \frac{kn}{N}\right), \qquad k = 0, 1, \ldots, N-1. \tag{4.57}$$

**Важно**

Быстрое преобразование Фурье (БПФ, FFT) — это семейство алгоритмов эффективного вычисления ДПФ, а не отдельное физическое преобразование. Поэтому результат нужно интерпретировать через определение (4.57), а не через название конкретной функции библиотеки.

При частоте дискретизации $f_s$ расстояние между соседними частотными бинами равно

$$\Delta f = \frac{f_s}{N} = \frac{1}{T_{\text{obs}}}. \tag{4.58}$$

### 4.5.1 Утечка и оконные функции

Применение ДПФ к конечному блоку эквивалентно умножению исходной последовательности на оконную функцию. Рассмотрим гармонический сигнал

$$x[n] = A\cos\left(2\pi\frac{f_c}{f_s}n + \varphi_0\right). \tag{4.59}$$

Оконная последовательность записывается как $x_w[n] = x[n]w[n]$. Если $W(f)$ — преобразование Фурье окна, то по свойству частотного сдвига

$$X_w(f) = \frac{A}{2}e^{\mathrm{j}\varphi_0}W(f - f_c) + \frac{A}{2}e^{-\mathrm{j}\varphi_0}W(f + f_c). \tag{4.60}$$

Следовательно, спектр оконного сигнала состоит из двух копий спектра окна, сдвинутых к частотам $\pm f_c$. Если вид окна отдельно не указан, подразумевается прямоугольное окно

$$w_R[n] = \begin{cases} 1, & 0 \le n \le N-1, \\ 0, & \text{иначе}. \end{cases} \tag{4.61}$$

Дискретно-временное преобразование Фурье прямоугольного окна равно

$$W_R(f) = \sum_{n=0}^{N-1} \exp\left(-\mathrm{j}2\pi \frac{f}{f_s} n\right) = \exp\left(-\mathrm{j}\pi(N-1)\frac{f}{f_s}\right) \frac{\sin(\pi N f/f_s)}{\sin(\pi f/f_s)}. \tag{4.62}$$

После нормировки на число отсчётов его амплитудно-частотная характеристика принимает вид

$$\frac{|W_R(f)|}{N} = \left|\frac{\sin(\pi N f/f_s)}{N\sin(\pi f/f_s)}\right|. \tag{4.63}$$

Выражение (4.63) представляет собой нормированное ядро Дирихле, главный лепесток которого при больших $N$ приближается функцией sinc. Нули характеристики расположены при $f = mf_s/N = m\Delta f$, где $m \in \mathbb{Z}$ и $m \not\equiv 0 \pmod N$. Поэтому, когда частота гармоники совпадает с FFT-бином, остальные отсчёты ДПФ попадают в нули сдвинутой характеристики прямоугольного окна.

Таким образом, для прямоугольного окна возможны два случая:

$$\text{Спектральная утечка} = \begin{cases} \text{отсутствует}, & f_c = k\Delta f, \quad k \in \mathbb{Z}, \\ \text{присутствует}, & f_c/\Delta f \notin \mathbb{Z}. \end{cases} \tag{4.64}$$

Возможна немного иная трактовка: гармонический сигнал, частота которого совпадает с FFT-бином, является одной из базисных функций ДПФ. Если частота находится между бинами, сигнал представляется совокупностью нескольких базисных функций, что и наблюдается как спектральная утечка.

Эффект проиллюстрирован на рисунке 4.9.

ДПФ предполагает периодическое продолжение анализируемого блока. Разрыв на стыке соседних периодов при несовпадении частоты с FFT-бином показан на рисунке 4.8.

Для уменьшения уровня боковых лепестков применяют оконные функции. Они плавно уменьшают значения отсчётов вблизи границ анализируемого блока, ослабляя разрыв при его периодическом продолжении. При этом подавление боковых лепестков сопровождается расширением главного лепестка и, следовательно, снижением частотного разрешения. Поэтому выбор окна

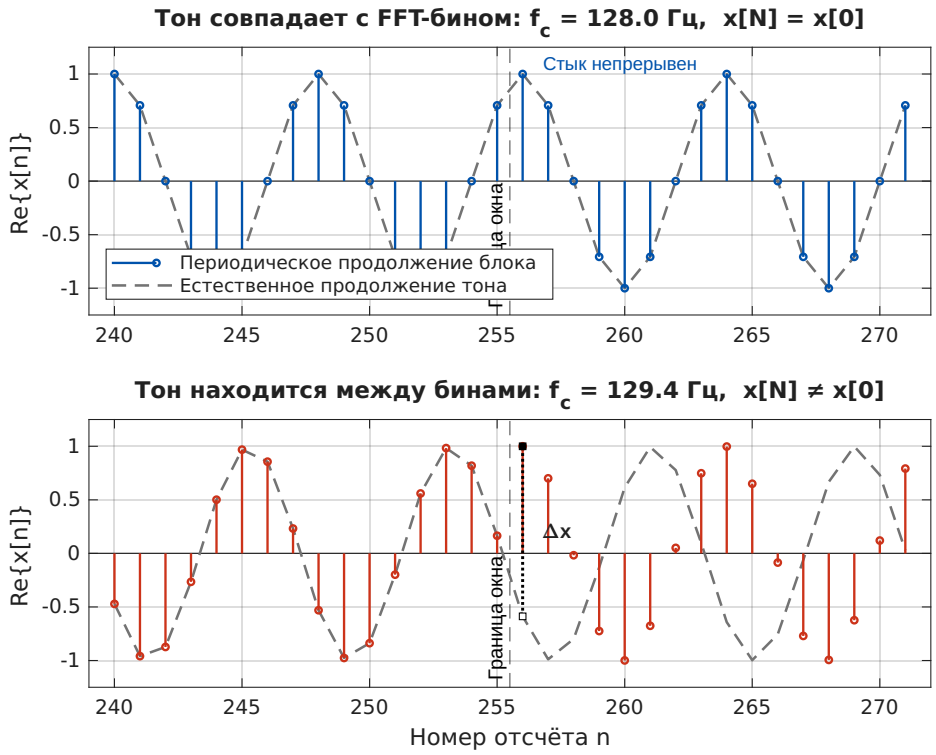



**Рис. 4.8:** Отсчёты комплексного тона около границы анализируемого блока. При совпадении частоты с FFT-бином периодическое продолжение остаётся непрерывным, а для частоты между бинами на стыке возникает разрыв

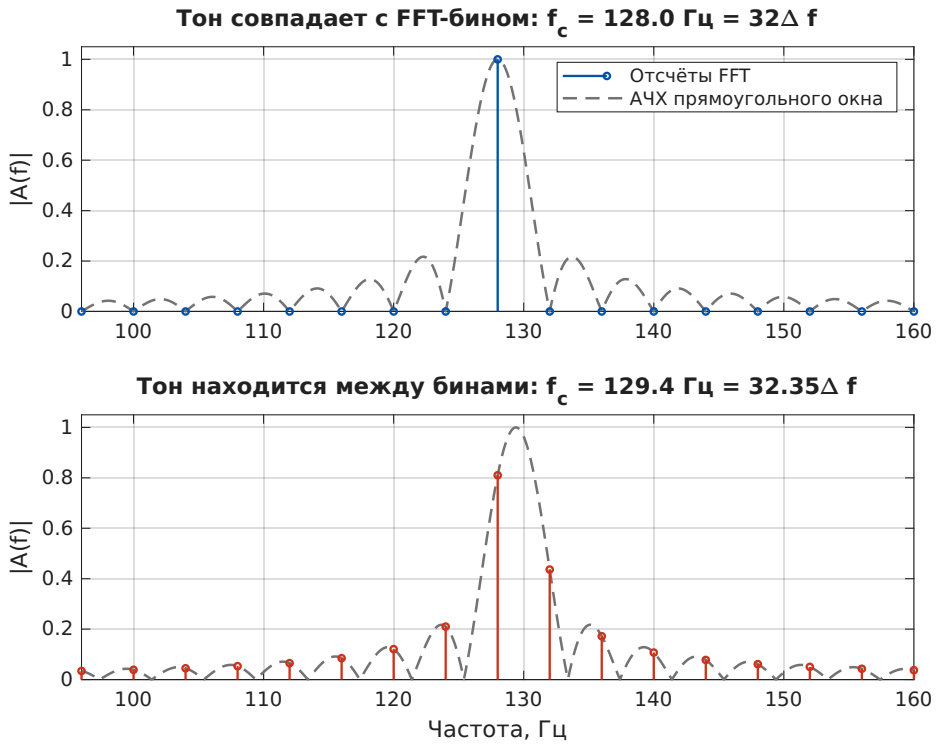



**Рис. 4.9:** Спектр комплексного тона при совпадении его частоты с FFT-бином и при расположении частоты между соседними бинами

всегда является компромиссом:

- прямоугольное окно обеспечивает наиболее узкий главный лепесток, но характеризуется высоким уровнем боковых лепестков;
- окно Ханна обеспечивает хороший компромисс между шириной главного лепестка и подавлением боковых лепестков;
- окна с более сильным подавлением боковых лепестков позволяют обнаруживать слабые спектральные компоненты рядом с мощными, но сильнее размывают близко расположенные линии.

### 4.5.2 FFTW3

FFTW3 (Fastest Fourier Transform in the West — «Самое быстрое преобразование Фурье на Западе») — это бесплатная библиотека на языке C для эффективного вычисления дискретного преобразования Фурье (DFT) и других связанных операций.

FFTW3 возвращает комплексный спектр в естественном порядке: нулевая частота находится в $X[0]$, положительные частоты — в первой части массива, а отрицательные — в конце. Для чётного $N$ физическую частоту удобно определять как

$$f[k] = \begin{cases} k\dfrac{f_s}{N}, & 0 \le k < \dfrac{N}{2}, \\ (k-N)\dfrac{f_s}{N}, & \dfrac{N}{2} \le k < N. \end{cases} \tag{4.65}$$

**Внимание**

Для строгого математического отображения обычно выполняют перестановку, известную как `fftshift`, и отображают частоты от $-f_s/2$ до $f_s/2$. Перестановка меняет только порядок элементов массива, но не вычисляет новое преобразование.

В отличие от спектра действительного сигнала, спектр общего комплексного IQ-сигнала не обязан быть зеркально-симметричным. Поэтому при работе с IQ нельзя безусловно отбрасывать отрицательную половину или удваивать положительную, как это иногда делают для одностороннего спектра действительных данных.

### 4.5.3 Нормировка FFTW3

FFTW3 вычисляет ненормированное преобразование: перед прямым ДПФ нет множителя $1/N$, и последовательное прямое и обратное преобразования возвращают исходный массив, умноженный на $N$ [17, раздел «What FFTW Really Computes»]. Следовательно, правило нормировки должно находиться в коде анализа, а не предполагаться неявно. Для оценки амплитуды комплексного тона удобно использовать

$$A[k] = \frac{|X[k]|}{N}. \tag{4.66}$$

Если применено окно $w[n]$, амплитуду дополнительно корректируют на его когерентный коэффициент усиления

$$G_c = \frac{1}{N}\sum_{n=0}^{N-1} w[n], \qquad A_w[k] = \frac{|X_w[k]|}{N G_c}. \tag{4.67}$$

Эта нормировка пригодна для амплитуды спектральной линии. Для оценки плотности мощности шума требуется другая нормировка, учитывающая энергию окна и ширину бина. Поэтому подпись «спектр, dB» без указания вычисляемой величины недостаточна.

**Контрольная точка**

Комплексная последовательность длиной $N = 4096$ получена при $f_s = 2{,}048$ МГц и содержит тоны с амплитудами 0,45 и 0,08 на частотах $+125$ и $-300{,}25$ кГц соответственно.

1. Найдите длительность блока и расстояние между соседними бинами ДПФ.
2. Укажите индекс первого тона в естественном порядке FFTW3. Между какими индексами находится второй тон и для какого из них возникнет спектральная утечка при прямоугольном окне?
3. Для окна с когерентным коэффициентом усиления $G_c = 0{,}5$ модуль ненормированного коэффициента первого тона равен 921,6. Восстановите его амплитуду и укажите используемую нормировку.
4. * Блок дополнен нулями до 16 384 отсчётов. Найдите новый шаг отображаемой частотной сетки и объясните, изменились ли длительность наблюдения и способность различать две близкие спектральные линии.

Ответы для сверки: $T_{\text{obs}} = 2$ мс, $\Delta f = 500$ Гц; первый тон находится в бине $k = 250$, второй — между $k = 3495$ и $k = 3496$, поэтому он даёт утечку. Амплитуда равна $921{,}6/(4096 \cdot 0{,}5) = 0{,}45$. После дополнения нулями шаг сетки равен 125 Гц, но длительность исходного наблюдения и физическое разрешение не изменяются: получается лишь более плотная интерполяция спектральной характеристики окна.

## 4.6 Амплитуда, мощность и децибелы

**Важно**

Логарифмический масштаб позволяет одновременно отображать сигналы, мощности которых различаются на несколько порядков. Это особенно важно при анализе радиочастотного спектра, где слабые составляющие могут находиться рядом с мощными сигналами и шумовым фоном.

### 4.6.1 Относительные уровни в децибелах

Децибел (дБ, dB) выражает отношение двух одноимённых величин и не имеет собственной опорной мощности. Для мощностей $P_1$ и $P_2$

$$L_P = 10 \log_{10}\left(\frac{P_2}{P_1}\right) \text{ дБ}, \qquad \frac{P_2}{P_1} = 10^{L_P/10}. \tag{4.68}$$

Для отношения амплитуд используется множитель 20:

$$L_A = 20 \log_{10}\left(\frac{A_2}{A_1}\right) \text{ дБ}, \qquad \frac{A_2}{A_1} = 10^{L_A/20}. \tag{4.69}$$

Поскольку мощность при неизменном сопротивлении пропорциональна квадрату амплитуды, записи $20 \log_{10} |X|$ и $10 \log_{10} |X|^2$ эквивалентны. Ошибка возникает, когда множитель выбирают по названию графика, а не по величине под логарифмом.

### 4.6.2 Абсолютные уровни мощности: дБВт и дБм

Для задания абсолютной мощности к обозначению децибела добавляют опорный уровень. Величина дБВт (dBW) отсчитывается относительно 1 Вт, а дБм (dBm) — относительно 1 мВт:

$$P_{\mathrm{dBW}} = 10\log_{10}\left(\frac{P}{1\ \text{Вт}}\right), \tag{4.70}$$

$$P_{\mathrm{dBm}} = 10\log_{10}\left(\frac{P}{1\ \text{мВт}}\right). \tag{4.71}$$

Поскольку 1 Вт равен 1000 мВт, перевод между этими шкалами выполняется добавлением или вычитанием 30 дБ:

$$P_{\mathrm{dBm}} = P_{\mathrm{dBW}} + 30, \qquad P_{\mathrm{dBW}} = P_{\mathrm{dBm}} - 30. \tag{4.72}$$

Обратный переход к линейной мощности имеет вид

$$P\,[\text{мВт}] = 10^{P_{\mathrm{dBm}}/10}, \qquad P\,[\text{Вт}] = 10^{(P_{\mathrm{dBm}}-30)/10} = 10^{P_{\mathrm{dBW}}/10}. \tag{4.73}$$

Разность двух абсолютных уровней является относительным уровнем в децибелах:

$$L_P\,[\text{дБ}] = P_2\,[\text{дБм}] - P_1\,[\text{дБм}], \qquad P_2\,[\text{дБм}] = P_1\,[\text{дБм}] + L_P\,[\text{дБ}]. \tag{4.74}$$

Например, усиление на 10 дБ переводит уровень $-20$ дБм в $-10$ дБм. Оно означает увеличение мощности в 10 раз, а не прибавление некоторой мощности в милливаттах.

**Таблица 4.1:** Соответствие линейной мощности, дБм и дБВт

| Мощность | Уровень, дБм | Уровень, дБВт |
|---|---|---|
| 1 пВт | $-90$ | $-120$ |
| 1 нВт | $-60$ | $-90$ |
| 1 мкВт | $-30$ | $-60$ |
| 0,1 мВт | $-10$ | $-40$ |
| 1 мВт | 0 | $-30$ |
| 10 мВт | 10 | $-20$ |
| 100 мВт | 20 | $-10$ |
| 1 Вт | 30 | 0 |
| 10 Вт | 40 | 10 |

**Внимание**

Обозначение dB не означает уровень относительно 1 Вт. Относительно 1 Вт задаётся уровень dBW, а относительно 1 мВт — dBm. Поэтому запись «мощность равна −20 dB» без указанной опоры или второй сравниваемой мощности неоднозначна.

### 4.6.3 Цифровой уровень и калибровка SDR

В SDR сигнал проходит через аналоговый радиочастотный тракт, усилители, смесители и АЦП. Поэтому числовая мощность IQ-отсчётов в общем случае не равна мощности на антенном входе. Для перевода цифрового уровня в дБм необходимо калибровать весь измерительный тракт: подать на вход сигнал известной мощности и определить соответствие между этой мощностью и уровнем относительно полной шкалы АЦП.

Для сравнения цифровых амплитуд часто используют dBFS — децибелы относительно принятой полной шкалы. Если $A_{\mathrm{FS}}$ определено как опорная амплитуда, то

$$L_{\mathrm{dBFS}} = 20 \log_{10} \left( \frac{A}{A_{\mathrm{FS}}} \right). \tag{4.75}$$

Не существует единственного соглашения о полной шкале комплексного числа. Например, можно ограничить каждую компоненту диапазоном $[-1, 1)$; тогда модуль точки $(I, Q)$ теоретически может приблизиться к $\sqrt{2}$. В другом проекте полной шкалой считают единичный модуль и дополнительно ограничивают $|x[n]| \leq 1$. В этом методическом пособии для генерируемых сигналов используется второе, более консервативное правило: сумма компонент тестового сигнала выбирается так, чтобы $|x[n]| < 1$. При чтении данных сторонней программы её соглашение нужно проверять отдельно.

**Контрольная точка**

На вход приёмного тракта подан сигнал мощностью −92 дБм. Последовательно действуют усиление 18 дБ, потери в кабеле 3 дБ, потери преобразования смесителя 6 дБ и усиление 12 дБ.

1. Определите мощность сигнала на выходе тракта в дБм, дБВт и ваттах.
2. Во сколько раз изменится мощность при ослаблении на 12 дБ? Во

сколько раз изменится амплитуда при неизменном сопротивлении?

3. Для комплексного отсчёта $I = Q = 0{,}5$ и правила $A_{\mathrm{FS}} = 1$ найдите модуль и уровень в dBFS.
4. * При прежней настройке усиления калибровочный сигнал $-50$ дБм давал $-20$ dBFS. Затем усиление приёмника увеличили на 8 дБ, и неизвестный сигнал дал $-37$ dBFS. Оцените его входную мощность, предполагая линейность тракта и неизменность остальных параметров.

Ответы для сверки: суммарный коэффициент равен $+21$ дБ, поэтому выходной уровень составляет $-71$ дБм, или $-101$ дБВт, что соответствует примерно $7{,}94 \cdot 10^{-11}$ Вт. Ослабление на 12 дБ уменьшает мощность в 15,85 раза, а амплитуду — в 3,98 раза. Для $I = Q = 0{,}5$ модуль равен $1/\sqrt{2}$, а уровень — примерно $-3{,}01$ dBFS. В задаче * входная мощность равна $-75$ дБм.

## 4.7 Форматы IQ

**Важно**

Комплексный отсчёт состоит из двух компонент, но способ их представления не является универсальным. Нужно различать по меньшей мере три уровня:

1. формат, с которым работает алгоритм в памяти компьютера;
2. формат потока между SDR и компьютером;
3. формат долговременного хранения в файле.

В libiio формат потока задаёт само устройство. Для каждой компоненты драйвер сообщает формат хранения: знак, число значащих бит, размер слова, сдвиг и порядок байт. Например, запись `le:S12/16>>0` означает 12-битное знаковое число в 16-битном слове little-endian без сдвига. Программа получает буфер именно в таком виде и сама переводит отсчёты в формат вычислений, например в `std::complex<float>`. У трансиверов AD936x отсчёты приёма — 12-битные числа в 16-битных словах, а при передаче 12 значащих бит должны занимать старшие разряды слова. Если включено несколько каналов, их компоненты в буфере чередуются, поэтому начало и шаг отсчётов канала получают функциями `iio_buffer_first()` и `iio_buffer_step()`, а не вычисляют вручную [18, 19].

Для обозначения форматов удобно использовать краткую запись из открытого стандарта метаданных SigMF (таблица 4.2): буква `c` — комплексный отсчёт, затем тип и разрядность компоненты, затем порядок байт [20].

**Таблица 4.2:** Часто используемые форматы комплексных отсчётов

| Формат | Тип компоненты | Смысл |
|---|---|---|
| `cf64_le` | `double` | комплексный отсчёт с плавающей точкой двойной точности |
| `cf32_le` | `float` | комплексный отсчёт с плавающей точкой одинарной точности; формат вычислений и учебных файлов |
| `ci16_le` | `int16_t` | комплексный знаковый 16-битный целочисленный отсчёт; типичный формат потока SDR, в том числе AD936x |
| `ci8` | `int8_t` | комплексный знаковый 8-битный целочисленный отсчёт |

Меньшее число бит в формате потока снижает требуемую пропускную способность транспортного интерфейса, но повышает шум квантования и уменьшает доступный динамический диапазон. Выбор формата поэтому является частью бюджета потока, а не только синтаксической настройкой программы.

### 4.7.1 Соглашение этого методического пособия для файлов

Расширение `.iq` само по себе ничего не говорит о содержимом файла. Чтобы все примеры части A были совместимы, вводится канонический учебный формат `cf32_le_iq`:

- каждая компонента хранится как 32-битное число IEEE 754 типа `float`;
- байты записываются в порядке little-endian;
- компоненты чередуются как $I[0], Q[0], I[1], Q[1], \ldots$;
- канонический файл содержит один канал;
- отсчёты нормированы в программном масштабе, а выбранное значение полной шкалы явно указывается в метаданных;
- рядом хранится текстовый sidecar-файл с метаданными.

Название `cf32_le_iq` является соглашением данного проекта, а не общеотраслевым стандартом. Оно намеренно содержит тип компоненты, порядок байт и порядок компонент. Для $N$ одноканальных отсчётов ожидаемый размер такого файла равен

$$S_{\text{file}} = 2N \ \text{sizeof}(\texttt{float}) = 8N \quad \text{байт}. \tag{4.76}$$

Если фактический размер не совпадает с (4.76), обработку следует остановить до выяснения причины. Типичные причины — заголовок внутри файла, другой тип данных, неполный последний отсчёт или многоканальная запись.

### 4.7.2 Минимальные метаданные

Sidecar-файл должен позволять интерпретировать массив без устных пояснений автора. Минимальный набор приведён в таблице 4.3.

**Таблица 4.3:** Минимальный паспорт IQ-файла

| Поле | Что необходимо зафиксировать |
|---|---|
| Формат | тип компоненты, порядок байт и порядок $I/Q$ |
| Геометрия | число каналов, число отсчётов на канал, способ чередования каналов |
| Время | $f_s$, момент или правило начала записи, наличие пропусков |
| Частота | $f_0$ либо явное указание, что сигнал создан в базовой полосе |
| Масштаб | правило нормировки, усиление приёмника и единица амплитуды |
| Происхождение | генератор или устройство, версия программы, tag/commit |
| Целостность | размер файла и контрольная сумма, например SHA-256 |

**Внимание**

Значения `fc32` в памяти программы не следует автоматически считать вольтами или dBm. Это безразмерные числа после неизвестного количества аналоговых и цифровых усилений и преобразований. Перевод к физической мощности требует отдельной калибровки всего измерительного тракта.

### 4.7.3 Контрольный IQ-файл

Чтобы следующие главы проверяли не разные случайные сигналы, а один известный эталон, зафиксируем контрольную последовательность. Она специально содержит положительную и отрицательную частоты, постоянную составляющую и ненулевые начальные фазы. Благодаря этому один тест выявляет перепутанные $I/Q$, неверный знак частоты, неправильную нормировку FFT и ошибку формата файла.

Параметры эталона:

$$f_s = 1\,000\,000 \text{ Гц}, \tag{4.77}$$

$$N = 65\,536, \tag{4.78}$$

$$x[n] = 0{,}65 \exp\left(\mathrm{j}\left(2\pi \frac{125\,000}{f_s} n + 0{,}3\right)\right) + 0{,}20 \exp\left(\mathrm{j}\left(2\pi \frac{-312\,500}{f_s} n - 0{,}8\right)\right) + (0{,}03 + \mathrm{j}0{,}02). \tag{4.79}$$

Обе частоты выбраны кратными $f_s/N$, поэтому при прямоугольном окне линии точно попадают в бины и не имеют спектральной утечки. Длительность записи равна 65,536 мс, шаг FFT полного файла — приблизительно 15,2588 Гц, а размер файла `cf32_le_iq` — 524 288 байт.

До написания генератора можно вычислить ожидаемые инварианты:

- среднее значение равно $0{,}03 + \mathrm{j}0{,}02$;
- спектральные линии находятся на $+125$ и $-312{,}5$ кГц;
- отношение амплитуд линий равно $20 \log_{10}(0{,}65/0{,}20) \approx 10{,}24$ дБ;
- средняя мощность $N^{-1} \sum |x[n]|^2$ равна $0{,}65^2 + 0{,}20^2 + 0{,}03^2 + 0{,}02^2 = 0{,}4638$;
- первый отсчёт приблизительно равен $x[0] \approx 0{,}79031 + \mathrm{j}0{,}06862$;
- максимальный модуль остаётся меньше единицы.

Минимальный sidecar может выглядеть следующим образом:

**Листинг 4.1:** Метаданные контрольного IQ-файла

```
{
  "schema": "sdr-part-a-iq-v1",
  "data_file": "reference_iq.cf32",
  "format": "cf32_le_iq",
  "channels": 1,
  "sample_rate_hz": 1000000,
  "center_frequency_hz": null,
  "sample_count": 65536,
  "full_scale_rule": "abs(sample) < 1",
  "generator": "reference_iq",
  "git_revision": "TO_BE_FILLED",
  "sha256": "TO_BE_FILLED"
}
```

Поле `center_frequency_hz` имеет значение `null`, поскольку эталон создан непосредственно в комплексной базовой полосе и не относится к конкретной несущей. После прохождения через аппаратный тракт это поле должно содержать фактическую настройку SDR.

### 4.7.4 Порядок проверки

Контрольный файл считается корректным только после следующих независимых проверок:

1. размер файла совпадает с (4.76);
2. первые несколько отсчётов совпадают с прямым вычислением (4.79) в пределах точности `float`;
3. среднее значение и средняя мощность совпадают с ожидаемыми;
4. FFT показывает две линии с правильными знаками частот и отношением амплитуд;
5. после записи и повторного чтения массив совпадает с исходным побитно;
6. sidecar содержит фактическое число отсчётов, размер и SHA-256 файла.

**Контрольная точка**

**Итоговая сверка главы.** Все ответы сначала получают расчётом, не запуская генератор и не используя SDR-плату. Исходной считается последовательность (4.79) в формате `cf32_le_iq`.

1. Рассчитайте длительность записи, размер файла, шаг ДПФ и индексы трёх спектральных компонент в естественном порядке FFTW3.
2. Независимо вычислите среднее значение, среднюю мощность, отношение уровней двух тонов и первый комплексный отсчёт. Эти величины должны войти в отчёт анализатора как проверяемые инварианты.
3. Предскажите результат двух ошибок формата: перестановки $I/Q$ и чтения `cf32` как `sc16`. Укажите, какие проверки обнаружат каждую ошибку до визуального анализа спектра.
4. Последовательность умножают на $\exp(-\mathrm{j}2\pi\,125\,000n/f_s)$, фильтруют идеальным ФНЧ с полосой $\pm 20$ кГц и децимируют в 20 раз. Предскажите частоты всех компонент после NCO, состав сигнала после ФНЧ, выходную частоту дискретизации и ожидаемую среднюю мощность без учёта переходного процесса фильтра.
5. Составьте минимальный протокол приёмки файла: какие поля sidecar, численные инварианты и признаки целостности необходимо сравнить до передачи файла следующему участнику эксперимента?
6. * Получен файл размером 524 280 байт при заявленных $N = 65\,536$; SHA-256 не совпадает, а спектральные линии расположены на $-125$ и $+312{,}5$ кГц. Укажите все обнаруживаемые противоречия, наибо-

лее вероятные причины и порядок дальнейших проверок, не изменяя исходный файл.

Ориентиры для сверки: длительность 65,536 мс, размер 524 288 байт, $\Delta f \approx 15{,}2588$ Гц; индексы линий равны 8192, 45056 и 0. После NCO частоты составят 0, $-437{,}5$ и $-125$ кГц; после ФНЧ останется постоянная комплексная компонента $0{,}65\exp(\mathrm{j}0{,}3)$, после децимации $f_s = 50$ кГц, а средняя мощность равна 0,4225. Перестановка $I/Q$ отражает знаки частот, а неверный тип данных нарушает размер элемента, значения первых отсчётов и численные инварианты. В задаче * файл короче ровно на один комплексный отсчёт; несовпадение контрольной суммы подтверждает нарушение целостности, а отражение спектра указывает на перестановку $I/Q$ или комплексное сопряжение. Проверку следует выполнять на копии: сначала сохранить исходный файл и его хеш, затем сверить размер и метаданные, исследовать первые отсчёты и только после этого проверять порядок компонент и спектр.

# 5 Первый приём и анализ спектра

В главе 4 сигнал создавала программа, и все его параметры были известны заранее. Теперь источником отсчётов становится QPSK SDR Lab: мы примем вещательный FM-диапазон 88–108 МГц, запишем IQ-фрагмент вместе с метаданными, построим временную форму, спектр и водопад и, наконец, демодулируем одну станцию до звука. Передатчик в этой главе не используется.

FM-вещание выбрано как контрольный сигнал не случайно: станции работают круглосуточно, их частоты известны, а мощности хватает даже на антенну, которая рассчитана на другой диапазон. Кроме того, в главе 4 уже выведен частотный дискриминатор (4.16), и здесь он заработает на реальном сигнале.

После выполнения главы читатель должен уметь:

- собрать и запустить минимальный C++-приёмник на libiio;
- согласовать центральную частоту, частоту дискретизации, аналоговую полосу и усиление с задачей;
- читать временную форму, спектр и водопад одного и того же потока;
- распознавать слабый сигнал, перегрузку АЦП и центральную составляющую;
- сохранять запись с метаданными и демодулировать FM-станцию.

**Важно**

Красивый водопад ещё не является измерением. Каждое наблюдение на графике нужно связать с конкретным параметром приёмника: частотой настройки, частотой дискретизации, полосой или усилением. Если этого сделать нельзя, картинку невозможно ни повторить, ни объяснить.

## 5.1 От супергетеродина к SDR

Чтобы увидеть, что именно делает программно-конфигурируемый приёмник, сравним его с классическим супергетеродинным приёмником вещательного

FM (рис. 5.1, а). В нём каждая операция выполнена отдельным аналоговым узлом. Входной фильтр и усилитель радиочастоты (УРЧ) выделяют диапазон и усиливают слабый сигнал. Смеситель с гетеродином переносит выбранную станцию на постоянную промежуточную частоту (ПЧ), обычно 10,7 МГц. Фильтр и усилитель ПЧ (УПЧ) отделяют станцию от соседних, частотный детектор превращает изменения частоты в звук, а усилитель низкой частоты (УНЧ) передаёт его громкоговорителю. Чтобы принимать другой вид модуляции, пришлось бы заменить детектор, то есть изменить схему.

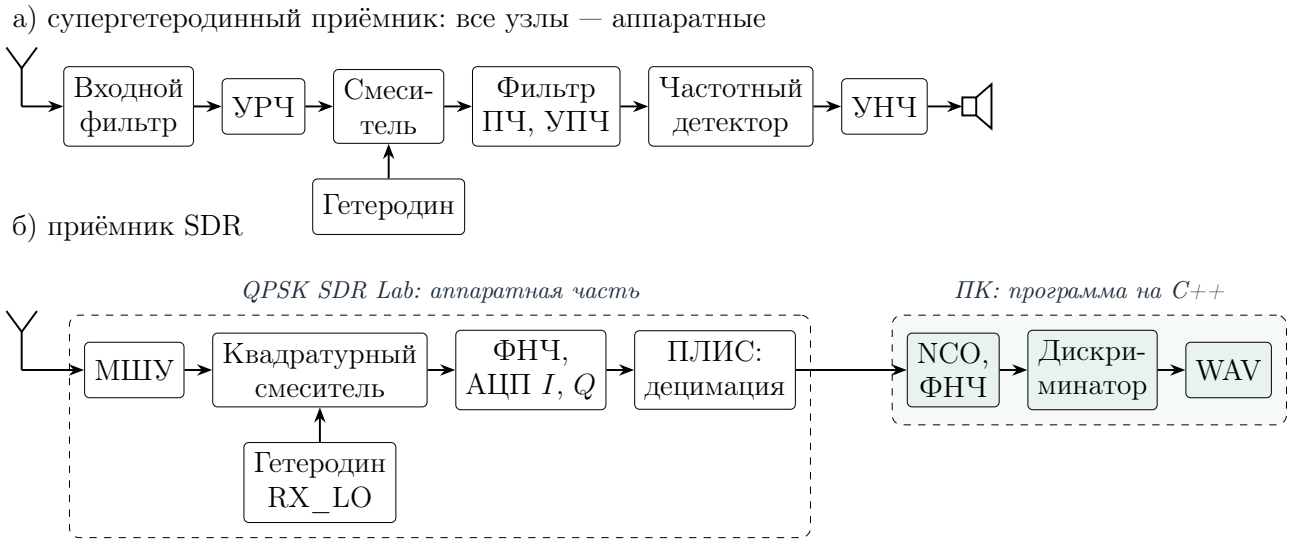



**Рис. 5.1:** Классический и программно-конфигурируемый приёмники FM. Зелёным отмечены операции, которые в SDR выполняет программа `fm_demod`.

В приёмнике SDR (рис. 5.1, б) аппаратными остаются только те узлы, которые программой не заменить: антенна, малошумящий усилитель (МШУ), смеситель с гетеродином и АЦП. Смеситель квадратурный: он переносит выбранный участок эфира сразу к нулевой частоте и выдаёт две составляющие, $I$ и $Q$ (раздел 4.2). Вместо одной станции на выходе получается полоса шириной $F_s$ со всеми станциями, которые в неё попали. Дальше всё делает программа. Выбор станции — это NCO, фильтр ПЧ — цифровой ФНЧ, частотный детектор — формула дискриминатора, а громкоговоритель заменяет файл WAV. Чтобы принять AM, достаточно другой программы, а схема платы не меняется. Именно в этом и состоит идея программно-конфигурируемого радио.

## 5.2 Подготовка: библиотеки и сборка

Для спектрального анализа используется библиотека FFTW3, знакомая по главе 4, а для построения графиков и водопада — OpenCV. Обе ставятся штатным менеджером пакетов. В RED OS:

```
dnf info fftw-devel opencv-devel
sudo dnf install fftw-devel opencv-devel
```

В Debian/Ubuntu:

```
sudo apt install libfftw3-dev libopencv-dev
```

Установку проверяют командой `pkg-config --modversion fftw3f opencv4`: она должна напечатать две версии. Учебные программы этой главы собираются в отдельном каталоге с теми же локальными libiio и libad9361-iio, что и в главе 2:

```
cmake -S . -B build/ch05 -DCMAKE_BUILD_TYPE=Release \
  -DLAB_WITH_IIO=ON -DLAB_IIO_PREFIX="$PWD/build/iio-install" \
  -DLAB_WITH_DSP=ON
cmake --build build/ch05 --parallel
ctest --test-dir build/ch05 --output-on-failure
mkdir -p build/ch05/results
```

Ожидается двенадцать тестов. К девяти тестам главы 2 добавлены три новых: синтетический тон на частоте бина ДПФ должен дать в спектре уровень $20\log_{10}0{,}5 \approx -6{,}02$ dBFS, синтетический FM-сигнал с тоном 1 кГц должен демодулироваться в звук с той же частотой, а программа анализа должна отказаться читать несуществующий файл. Все три теста работают без платы. Программы главы перечислены в таблице 5.1.

**Таблица 5.1:** Программы главы

| Программа | Вход | Результат |
|---|---|---|
| `rx_capture` | плата QPSK SDR Lab | непрерывная запись `.cf32` и метаданные `.cf32.json` |
| `spectrum_view` | запись `.cf32` | временная форма, спектр и водопад в PNG, список сильных линий |
| `fm_demod` | запись `.cf32` | звук станции в WAV, проверка стереопилота |

К разъёму RX1 подключают любую комплектную антенну. Она рассчитана на 1800 и 4680 МГц, поэтому на 100 МГц работает плохо, но мощности городских FM-станций хватает с запасом. Разъёмы TX остаются свободными.

## 5.3 Минимальный C++ RX-тракт

Приём через libiio складывается из двух устройств. Управляющее устройство `ad9361-phy` хранит параметры радиотракта: канал `altvoltage0` управляет гетеродином приёмника (RX_LO), а канал `voltage0` — полосой и усилением первого приёмника. Потоковое устройство `cf-ad9361-lpc` отдаёт отсчёты: его каналы `voltage0` и `voltage1` — компоненты $I$ и $Q$ первого радиоканала (раздел 2.7). Частоту дискретизации задаёт libad9361-iio: функция `ad9361_set_bb_rate()` сама рассчитывает и загружает FIR-фильтр, без которого трансивер не работает ниже 2,08 Мвыб/с. Сама libad9361-iio отсчётов не передаёт: она лишь записывает нужные атрибуты через libiio.

Ключевая часть программы `rx_capture` без проверок ошибок:

```
iio_device* phy = iio_context_find_device(ctx, "ad9361-phy");
iio_device* rx  = iio_context_find_device(ctx, "cf-ad9361-lpc");
iio_channel* lo  = iio_device_find_channel(phy, "altvoltage0", true); // RX_LO
iio_channel* ctl = iio_device_find_channel(phy, "voltage0", false);   // RX1

ad9361_set_bb_rate(phy, 1000000);                       // F_s и FIR-фильтр
iio_channel_attr_write_longlong(ctl, "rf_bandwidth", 1000000);
iio_channel_attr_write_longlong(lo, "frequency", 98350000);
iio_channel_attr_write(ctl, "gain_control_mode", "manual");
iio_channel_attr_write_double(ctl, "hardwaregain", 40);

iio_channel* ci = iio_device_find_channel(rx, "voltage0", false);     // I
iio_channel* cq = iio_device_find_channel(rx, "voltage1", false);     // Q
iio_channel_enable(ci);
iio_channel_enable(cq);
iio_buffer* buf = iio_device_create_buffer(rx, n, false);
iio_buffer_refill(buf);              // первая порция отбрасывается
iio_buffer_refill(buf);              // непрерывная запись из n отсчётов

auto* pi = (char*)iio_buffer_first(buf, ci);
auto* pq = (char*)iio_buffer_first(buf, cq);
for (; pi < (char*)iio_buffer_end(buf);
     pi += iio_buffer_step(buf), pq += iio_buffer_step(buf)) {
    int16_t i, q;
    iio_channel_convert(ci, &i, pi);  // знак и сдвиг по формату драйвера
    iio_channel_convert(cq, &q, pq);
    x.emplace_back(i / 2048.0f, q / 2048.0f);  // полная шкала = 1
}
```

Порядок действий отражает путь сигнала из главы 1: сначала часто-

та дискретизации и цифровые фильтры, затем аналоговая полоса и гетеродин, затем усиление и только после этого поток. Запись выполняется одним буфером на все $n$ отсчётов, поэтому внутри неё нет разрывов. Первая порция данных отбрасывается: она могла начаться до того, как устройство перестроилось. Отсчёты АЦП имеют 12 значащих бит, поэтому делением на 2048 они приводятся к полной шкале, равной единице, как в главе 4. Функция `iio_channel_convert()` учитывает формат `le:S12/16>>0`, о котором сообщает драйвер (раздел 4.7).

Запишем три секунды вблизи станции 98,1 МГц:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/ch05/rx_capture --uri ip:192.168.2.1 --freq 98.35e6 \
  --rate 1e6 --gain 40 --samples 3000000 \
  --out build/ch05/results/fm98.cf32
```

На стенде авторов программа вывела:

```
f0 = 98349998 Hz (requested 98350000)
Fs = 1000000 S/s, RF bandwidth = 1000000 Hz
gain: manual, 40 dB
3000000 samples, RMS -29.5 dBFS, clipped 0 (0.00 %)
```

Запрошенная и фактическая центральные частоты различаются на 2 Гц: синтезатор перестраивается с шагом около 2,4 Гц и выбирает ближайшее достижимое значение. Поэтому в метаданные записывается прочитанное из устройства значение, а не запрошенное.

**Контрольная точка**

1. Какие атрибуты относятся к устройству `ad9361-phy`, а какие — к потоковому `cf-ad9361-lpc`? Что изменится в программе для приёма по RX2?
2. Сколько байт передаёт плата за одну запись из $3 \cdot 10^6$ отсчётов, если каждая компонента $I$, $Q$ занимает 16 бит? Сколько займёт файл `.cf32`?
3. Почему порядок *настроить, затем создать буфер* лучше, чем обратный?

Ответы для сверки: для RX2 используются канал `voltage1` в `ad9361-phy` и каналы `voltage2`, `voltage3` потокового устройства. По сети передаётся $3 \cdot 10^6 \cdot 4 = 12$ Мбайт, файл занимает $3 \cdot 10^6 \cdot 8 = 24$ Мбайт: вместо двух 16-битных целых хранятся два 32-битных числа с плавающей точкой. Поток, созданный до настройки, содержит отсчёты переходного процесса

перестройки.

## 5.4 Четыре частотных параметра

В приёмнике одновременно действуют четыре частотных параметра (таблица 5.2). Их нельзя подменять друг другом: каждый задаётся в своём месте тракта и проявляется по-своему.

**Таблица 5.2:** Частотные параметры приёмника

| Параметр | Где задаётся | Что определяет |
|---|---|---|
| Центральная частота $f_0$ | гетеродин RX_LO | какой участок эфира переносится к нулевой частоте |
| Частота дискретизации $F_s$ | libad9361-iio | ширину наблюдаемого интервала $[f_0 - F_s/2;\ f_0 + F_s/2)$ и поток данных |
| Аналоговая полоса $W$ | `rf_bandwidth` | какие сигналы доходят до АЦП и складываются в его нагрузке |
| Занимаемая полоса $B$ | передатчик станции | сколько спектра нужно сохранить; для вещательного FM около 200 кГц |

Компонента с частотой $f_c$ в записи соответствует радиочастоте $f_0 + f_c$ (формула (4.31)). Обычно выбирают $B < W \le F_s$: полоса аналогового фильтра не шире наблюдаемого интервала, иначе сигналы за его краями наложатся на рабочую полосу (раздел 4.3). В примере выше $f_0 = 98{,}35$ МГц, а станция 98,1 МГц оказалась на смещении $f_c = -250$ кГц. Настраиваться точно на станцию неудобно: в нуле частот находятся утечка гетеродина и постоянная составляющая АЦП, и они легли бы прямо на сигнал.

Аналоговая полоса влияет не только на то, что видно на спектре, но и на перегрузку. АЦП складывает *все* сигналы, прошедшие фильтр. На стенде авторов на $f_0 = 106$ МГц при усилении 71 дБ и $W = 1$ МГц не было ни одного отсчёта в насыщении, а при $W = F_s = 4$ МГц и том же усилении в насыщении оказались 23% отсчётов: в полосу попали ещё несколько мощных станций.

**Контрольная точка**

1. Для $f_0 = 98$ МГц и $F_s = 4$ Мвыб/с найдите границы наблюдаемого интервала. Попадёт ли в него станция 100,0 МГц?
2. Почему при $F_s = 1$ Мвыб/с для приёма одной FM-станции нельзя выставить $W = 4$ МГц, не опасаясь последствий?

3. * Какую $f_0$ выбрать, чтобы при $F_s = 1$ Мвыб/с одновременно записать станции 98,1 и 98,6 МГц и при этом не поместить ни одну из них в ноль частот?

Ответы для сверки: 96–100 МГц; станция 100,0 МГц лежит точно на краю $f_0 + F_s/2$, половина её полосы оказывается за пределами интервала, а у края ещё и спадает АЧХ фильтров. Сигналы шириной до 4 МГц пройдут фильтр и наложатся на наблюдаемый интервал шириной 1 МГц, а также увеличат нагрузку АЦП. Например, $f_0 = 98{,}35$ МГц: станции окажутся на $-250$ и $+250$ кГц, каждая шириной около 200 кГц.

## 5.5 Усиление и АРУ

Усиление приёмника задаётся двумя способами. В режиме `manual` значение `hardwaregain` остаётся постоянным, и записи разных опытов можно сравнивать между собой. Режимы `slow_attack`, `fast_attack` и `hybrid` включают АРУ: трансивер сам подстраивает усиление под уровень сигнала. Это удобно для прослушивания, но в опыте уровни на графиках перестают быть сопоставимыми. В учебных опытах используется ручной режим; `rx_capture` принимает число в децибелах или название режима АРУ.

Признаки неправильного усиления видны на временной форме сразу (рис. 5.2). При усилении 0 дБ полезный сигнал занимает единицы младших разрядов АЦП: на графике видны лишь ступеньки $\pm 1/2048$, а в спектре сильнее всех оказывается центральная составляющая. При 71 дБ и широкой полосе вершины сигнала срезаны на уровне полной шкалы. Такая перегрузка необратима: срезанную часть сигнала программно не восстановить, а нелинейность порождает в спектре ложные составляющие.

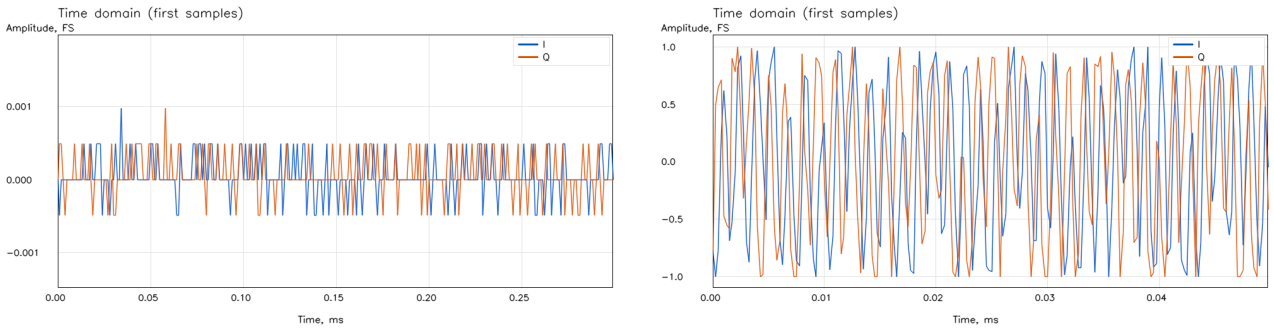



**Рис. 5.2:** Временная форма при недостаточном усилении (0 дБ, слева) и при перегрузке АЦП (71 дБ, $W = 4$ МГц, справа). Записи стенда авторов.

**Таблица 5.3:** Признаки выбора усиления на стенде авторов

| Режим | Наблюдение | Признак |
|---|---|---|
| Мало усиления | 0 дБ: −66 dBFS | ступеньки АЦП во временной форме; сильнейшая линия — в нуле частот |
| Нормально | 40 дБ: −30 dBFS | плавная форма, ни одного отсчёта в насыщении, станции выше шума |
| Перегрузка | 71 дБ, $W = 4$ МГц: −0,5 dBFS | срезанные вершины, 23% отсчётов в насыщении, предупреждение программы |

Практическое правило: усиление поднимают, пока шум приёмника не станет заметно выше ступенек АЦП, и останавливаются задолго до первых отсчётов в насыщении. Программа `rx_capture` печатает среднеквадратичный уровень в dBFS и число отсчётов в насыщении и предупреждает о перегрузке, если их больше 0,1%.

Уровень в dBFS — цифровой уровень относительно полной шкалы АЦП, а не мощность на разъёме (раздел 4.6). Увеличение усиления на 10 дБ поднимает все уровни на графике на 10 дБ, хотя мощность в эфире не изменилась. Перевести dBFS в дБм можно только после калибровки тракта известным источником.

## 5.6 Время, спектр и водопад

Программа `spectrum_view` строит по записи три согласованных представления и печатает самые сильные спектральные линии:

```
./build/ch05/spectrum_view build/ch05/results/fm4m.cf32 --nfft 4096
```

Для записи шириной 4 МГц около 98 МГц на стенде авторов:

```
2000000 samples, Fs = 4000000 S/s, duration 0.500 s
N = 4096, df = Fs/N = 976.6 Hz, block 1.024 ms, averaged 64 blocks
Strongest lines (offset from f0, level):
      +98632.8 Hz   -48.63 dBFS   (RF 98.098631 MHz)
```

Спектр вычисляется по блокам из $N$ отсчётов с окном Ханна. После перестановки `fftshift` бин с номером $k$ соответствует частоте

$$f_k = f_0 + \left(k - \frac{N}{2}\right)\frac{F_s}{N}, \qquad \Delta f = \frac{F_s}{N}, \qquad T_{\text{blk}} = \frac{N}{F_s}, \tag{5.1}$$

где $\Delta f$ — шаг частотной сетки, а $T_{\text{blk}}$ — длительность одного блока (формула (4.65)). Уровни нормированы на когерентный коэффициент окна

(формула (4.67)), поэтому комплексный тон с амплитудой $A$ даёт линию $20\log_{10} A$ dBFS; именно это проверяет один из тестов. Спектр шумоподобного сигнала от блока к блоку сильно флуктуирует, поэтому спектры $N_{\text{avg}}$ соседних блоков усредняются: чем больше $N_{\text{avg}}$, тем глаже кривая, но тем хуже видны быстрые изменения во времени.

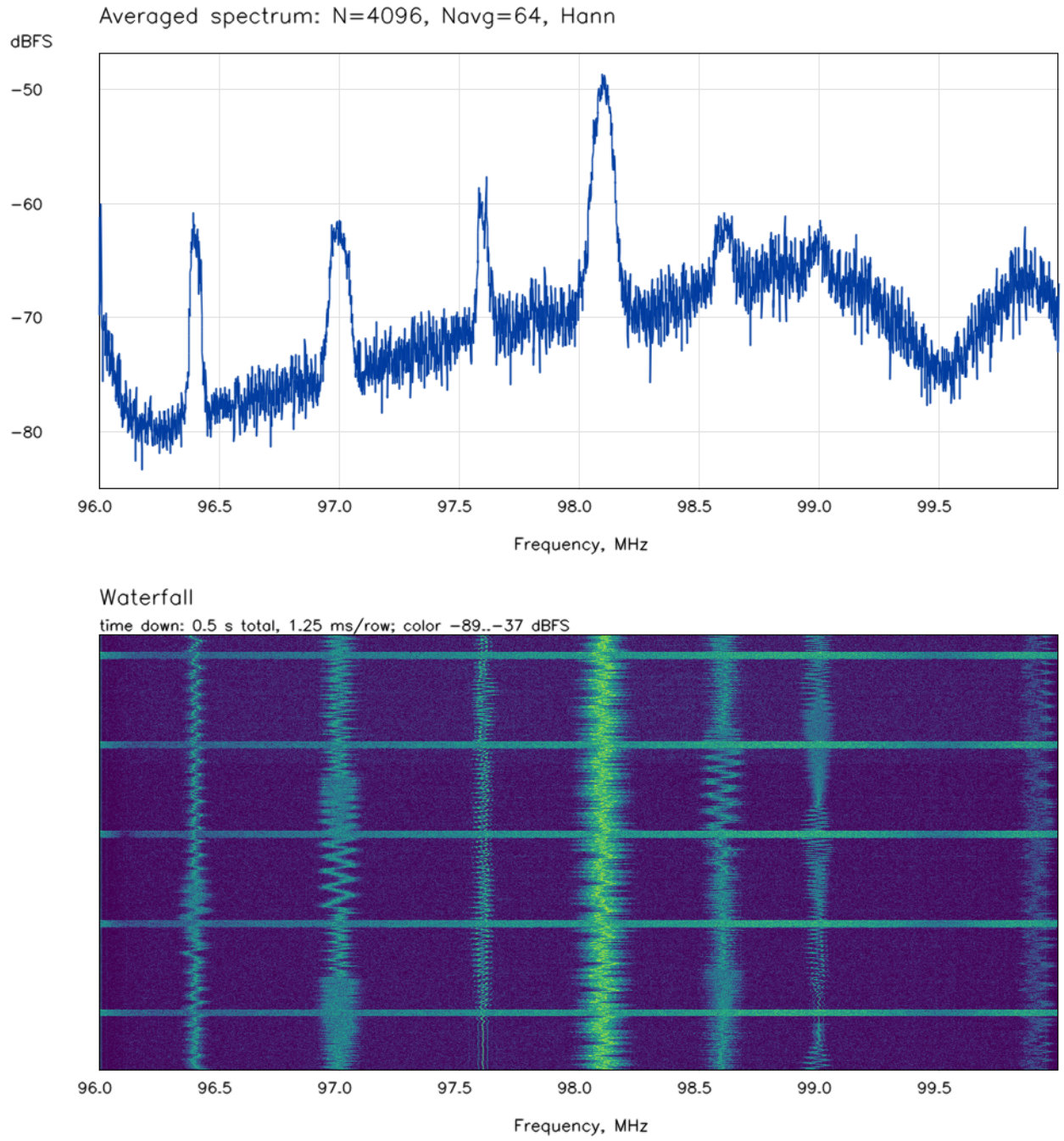



**Рис. 5.3:** Усреднённый спектр и водопад участка 96–100 МГц: $F_s = 4$ Мвыб/с, $N = 4096$, усиление 40 дБ. Запись стенда авторов.

**Рисунки главы в полном размере: удобно рассмотреть спектр и водопад на экране телефона**
`https://github.com/MCHelectronics/LETI_MIR_accelerator/blob/main/docs/img/ch05`

На рис. 5.3 видны станции на 96,4; 97,0; 97,6; 98,1; 98,6 и 99,0 МГц. Водопад — те же спектры отдельных блоков, выстроенные по времени сверху вниз; цвет показывает уровень. Каждая станция выглядит вертикальной полосой, а её извилистость — это частотная модуляция: мгновенная частота следует за звуком. Три представления отвечают на разные вопросы: временная форма — о размахе сигнала и насыщении, спектр — о частотном составе и уровнях, водопад — о том, как состав меняется во времени.

**Контрольная точка**

1. Найдите $\Delta f$ и $T_{\text{blk}}$ для $F_s = 1$ Мвыб/с и $N = 4096$. Хватит ли такого разрешения, чтобы различить две линии, отстоящие на 500 Гц?
2. Как изменятся спектр и водопад, если увеличить $N$ до 16 384 при той же записи?
3. На водопаде рис. 5.3 видны горизонтальные полосы, которые захватывают весь диапазон. Определите по рисунку их период.
4. * Тот же период получен при $F_s = 1$ и 4 Мвыб/с, то есть он постоянен во времени, а не в числе отсчётов. Что это говорит об источнике полос? Проверьте гипотезу: период 102,4 мс совпадает со стандартным интервалом маяков Wi-Fi (100 TU). Как подтвердить или опровергнуть её на своём стенде?

Ответы для сверки: $\Delta f \approx 244$ Гц и $T_{\text{blk}} \approx 4{,}1$ мс; линии на расстоянии 500 Гц разделяются, хотя и всего двумя бинами. При $N = 16\,384$ шаг сетки уменьшится вчетверо, а каждая строка водопада станет вчетверо длиннее по времени. Период полос около 0,1 с. Раз он не зависит от $F_s$, полосы вызваны внешней помехой, а не разрывами в записи. Гипотезу проверяют, меняя одно условие за раз: выключают ближайшие Wi-Fi-устройства, переносят антенну, меняют $f_0$.

## 5.7 Запись с метаданными

Каждая запись сопровождается файлом метаданных (раздел 4.7, таблица 4.3). Программа `rx_capture` заносит в него фактически установленные значения, прочитанные из устройства:


```
{
  "format": "cf32_le_iq",
  "sample_rate_hz": 1000000,
  "center_frequency_hz": 98349998,
  "sample_count": 3000000,
  "rf_bandwidth_hz": 1000000,
  "rx_channel": 1,
  "gain_control_mode": "manual",
  "hardwaregain_db": 40,
  "full_scale": "1.0 = 2048 ADC counts",
  "rms_dbfs": -29.456758991271009,
  "clipped_samples": 0,
  "contiguous": "yes: single buffer, no gaps",
  "uri": "ip:192.168.70.152",
  "utc_start_approx": "2026-10-12T00:12:05Z",
  "program": "rx_capture"
}
```


По этим полям запись можно обработать без устных пояснений: `spectrum_view` берёт из них $F_s$ и $f_0$ для подписи оси частот, а `fm_demod` — частоту дискретизации. Программы отказываются читать файл, размер которого не совпадает с числом отсчётов из метаданных, и не перезаписывают существующие записи. Время начала отмечено как приблизительное: это время компьютера в момент сохранения, а не метка времени платы.

## 5.8 Демодуляция FM

Программа `fm_demod` собирает из инструментов главы 4 простейший FM-приёмник:

1. NCO переносит выбранную станцию со смещения $f_c$ в ноль частот (формула (4.37));
2. ФНЧ с частотой среза 100 кГц оставляет только эту станцию, после чего частота дискретизации понижается до 250 квыб/с (раздел 4.4);
3. дискриминатор (4.16) вычисляет мгновенную частоту; деление на девиацию вещательного FM 75 кГц приводит результат к шкале $\pm 1$;
4. цепь коррекции предыскажений с постоянной времени $\tau = 50$ мкс ослаб-

ляет высокие звуковые частоты, которые передатчик заранее подчеркнул:

$$y[n] = (1 - a)\,x[n] + a\,y[n-1], \qquad a = \exp\!\left(-\frac{1}{F_s \tau}\right); \tag{5.2}$$

5. ФНЧ 15 кГц и понижение частоты дискретизации до 50 квыб/с дают звук, который сохраняется в 16-битный WAV.

Станция 98,1 МГц в записи `fm98.cf32` находится на смещении −250 кГц:

```
./build/ch05/fm_demod build/ch05/results/fm98.cf32 \
  build/ch05/results/fm98.wav --offset -250000
```

Вывод на стенде авторов:

```
Fs 1000000 -> 250000 (x1/4) -> audio 50000 S/s (x1/5), 3.00 s
Mean frequency offset of the station: +265 Hz
Stereo pilot 19 kHz: 21.2 dB above the composite noise (broadcast FM detected)
```

Полученный файл открывается любым проигрывателем, например командой `aplay build/ch05/results/fm98.wav`. Объективная проверка не требует слуха: стереофоническое вещание передаёт пилот-тон 19 кГц, и программа ищет его в выходе дискриминатора. Пилот на 21 дБ выше шума доказывает, что демодулирована настоящая FM-станция. Средний сдвиг частоты — грубая оценка расхождения частоты станции и гетеродина приёмника; от записи к записи он меняется на десятки герц, поскольку зависит и от содержания передачи.

**Контрольная точка**

1. Демодулируйте другую станцию из той же записи. Какое смещение `--offset` нужно указать для 98,6 МГц и почему для этого не требуется новая запись?
2. Что услышите, если ошибиться со знаком смещения? Как это объяснить по спектру?
3. * Повторите демодуляцию с `--deemph 75`. Чем звук будет отличаться и в какой стране такая постоянная времени стандартна?

Ответы для сверки: +250 кГц; запись при $F_s = 1$ Мвыб/с уже содержит интервал 97,85–98,85 МГц. При неверном знаке в ноль частот переносится участок без станции или соседняя станция, и вместо звука получится шум. С $\tau = 75$ мкс (стандарт США) высокие частоты будут ослаблены сильнее, и звук станет глуше.

## 5.9 Программный тракт без платы

Все программы анализа работают с файлами, поэтому записанный однажды фрагмент можно обрабатывать сколько угодно раз без платы, а синтетические сигналы с известными параметрами — проверять в автоматических тестах. В этом методическом пособии файл служит и программным эталоном: в аппаратном опыте с передачей сигнал, принятый по кабелю, сравнивается с тем же сигналом, записанным в файл до передачи.

Обмен потоком между независимыми процессами, например через ZeroMQ, в обязательный маршрут не входит. Желающие могут повторить конвейер этой главы в GNU Radio: в нём есть готовые блоки ZeroMQ и FM-демодулятора, а метаданные записи позволяют настроить их на те же параметры.

**Контрольная точка**

**Итоговые вопросы к главе.**

1. Перечислите параметры, которые нужно записать, чтобы другой участник получил по вашему файлу тот же спектр и тот же звук.
2. На спектре видна узкая линия точно в нуле частот, а при изменении $f_0$ она остаётся в нуле. Что это и как её обойти?
3. Средний уровень записи $-3$ dBFS, `rx_capture` сообщает 5% отсчётов в насыщении. Предложите два независимых способа это исправить.
4. Почему уровень станции в dBFS нельзя записать в отчёт как её мощность в дБм?
5. * Какая максимальная $F_s$ возможна при непрерывной записи двух каналов по USB 2.0 с практической скоростью около 35 Мбайт/с?

Ответы для сверки: $f_0$, $F_s$, $W$, режим и значение усиления, канал, формат и полная шкала, параметры анализа $N$ и $N_{\text{avg}}$, смещение станции. Это утечка гетеродина и постоянная составляющая; приёмник настраивают со смещением относительно сигнала. Уменьшить усиление или сузить аналоговую полосу $W$, чтобы мощные соседние сигналы не доходили до АЦП. dBFS отсчитывается от полной шкалы АЦП и зависит от усиления; для перевода в дБм нужна калибровка. Два канала по 4 байта на отсчёт дают $8F_s$ байт/с, откуда $F_s \approx 35/8 \approx 4{,}4$ Мвыб/с.

# 6 Аппаратный тракт TX→RX и AM/FM как диагностические сигналы

В главе 5 плата только принимала сигналы, параметры которых мы не задавали. Теперь передатчик и приёмник соединяются в один тракт: программа создаёт сигнал с известными параметрами, передатчик QPSK SDR Lab излучает его в кабель с аттенюатором, а приёмник той же платы записывает. Результат сравнивается с программным эталоном — тем же сигналом, записанным в файл до передачи. Тон, AM и FM здесь служат не средством связи, а диагностическими сигналами: по каждому из них хорошо видна своя часть искажений тракта.

После выполнения главы читатель должен уметь:

- объяснить, какие узлы передатчика SDR аппаратные, а какие программные;
- собрать кабельный тракт по допуску главы 1 и передать тон, AM и FM;
- предсказать спектр каждого сигнала и сравнить его с принятым;
- измерить глубину AM по спектру и проверить линейность тракта лестницей уровней;
- объяснять отличия принятого сигнала от эталона причинами в тракте.

**Внимание**

Передача выполняется только после решения *GO* по разделу 1.3: кабельный тракт TX1 → DC-блокер → аттенюатор → RX1 собран при выключенном передатчике, ослабление подтверждено. Антенны к разъёмам TX и RX, участвующим в опыте, не подключаются. Прямое соединение TX→RX без аттенюатора запрещено.

## 6.1 От классического передатчика к SDR

В классическом FM-передатчике (рис. 6.1, а) звуковой сигнал после усилителя управляет частотой генератора (ГУН — генератор, управляемый напряжением), синтезатор переносит его на рабочую частоту, а усилитель мощности (УМ) отдаёт сигнал в антенну. Вид модуляции задан схемой: для AM понадобился бы другой модулятор.

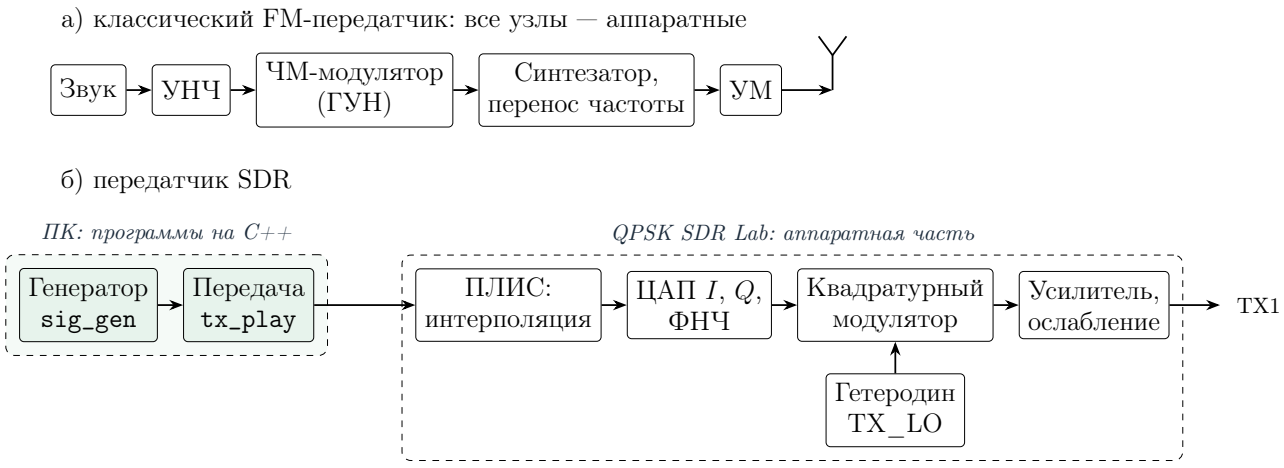



**Рис. 6.1:** Классический и программно-конфигурируемый передатчики. В SDR вид модуляции задаётся числами $I[n]$, $Q[n]$, которые вычисляет программа.

В передатчике SDR (рис. 6.1, б) модуляцию выполняет программа: она вычисляет комплексные отсчёты $x[n] = I[n] + \mathrm{j}Q[n]$ модулированного сигнала в базовой полосе. ПЛИС повышает частоту дискретизации, ЦАП превращает $I$ и $Q$ в напряжения, а квадратурный модулятор переносит их на частоту гетеродина TX_LO по формуле (4.20). Аппаратными остаются ЦАП, модулятор, гетеродин и усилитель. Тон, AM и FM в этой главе отличаются только формулой в генераторе, а плата и программа передачи одни и те же.

## 6.2 Схема опыта и допуск

Опыт собирается по рис. 6.2. Генератор записывает сигнал в файл; программа `tx_play` передаёт этот файл через TX1, а `rx_capture` записывает то, что пришло на RX1. Тот же исходный файл без передачи анализируется программой `spectrum_view` и служит программным эталоном.

Бюджет такого тракта уже рассчитан в главе 1. При верхней границе мощности передатчика $P_{\mathrm{TX,max}} = +10$ дБм и комплектном аттенюаторе 30 дБ с минимальным ослаблением 29 дБ наихудший уровень на входе RX составляет $-19$ дБм. Это ниже предела $+2{,}5$ дБм с рабочим запасом 20 дБ

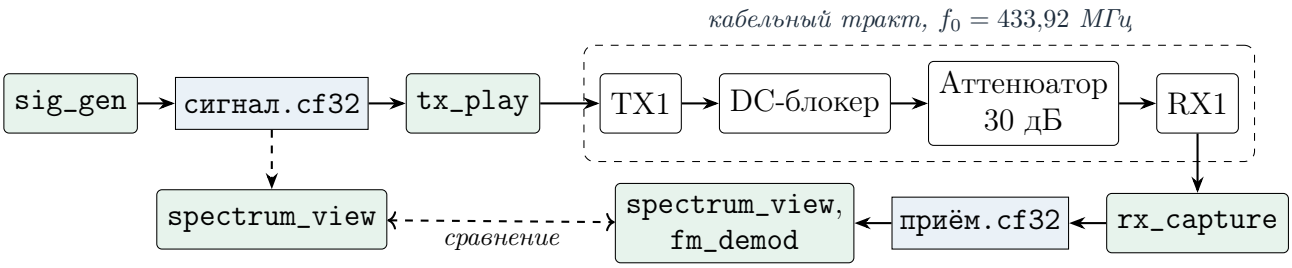



**Рис. 6.2:** Схема опыта TX→RX на одной плате. Пунктир — путь программного эталона без передачи.

(условие (1.5)) даже при нулевом ослаблении передатчика, поэтому все шаги опыта, включая лестницу уровней, безопасны для входа. Решение *GO* и фактические номиналы элементов записывают в паспорт стенда.

Центральная частота опыта — $f_0 = 433{,}92$ МГц, середина диапазона 433,05–434,79 МГц, в котором разрешена работа маломощных устройств. Тракт закрытый, но слабая утечка через корпус и разъёмы неизбежна, и пусть она попадает в такую полосу. Передатчик и приёмник работают на одной и той же $f_0$.

1. Собрать пассивную цепь TX1 → DC-блокер → аттенюатор 30 дБ → RX1 при выключенном передатчике; антенны с этих разъёмов снять.
2. Проверить решение *GO* в паспорте стенда.
3. Запускать передачу только программой `tx_play` с ключом `--go` и начинать с большого ослабления.

## 6.3 Тестовые сигналы и циклический буфер

Программа `sig_gen` создаёт три сигнала с частотой дискретизации $F_s$ на смещении $f_c$ относительно $f_0$ (таблица 6.1).

**Таблица 6.1:** Тестовые сигналы главы

| Сигнал | Отсчёты | Ожидаемый спектр |
|---|---|---|
| Тон | $x[n] = A\,e^{\mathrm{j}2\pi f_c n T_s}$ | одна линия на $f_c$ с уровнем $20\log_{10} A$ dBFS |
| AM | $x[n] = A\big[1 + \mu\cos(2\pi f_m n T_s)\big]e^{\mathrm{j}2\pi f_c n T_s}$ | несущая на $f_c$ и две боковые на $f_c \pm f_m$, ниже несущей на $20\log_{10}(2/\mu)$ дБ |
| FM | $x[n] = A\,e^{\mathrm{j}(2\pi f_c n T_s + \beta\sin(2\pi f_m n T_s))}$ | полоса шириной около $2(\Delta f + f_m)$ вокруг $f_c$ |

Здесь $\mu$ — глубина AM, $\Delta f$ — девиация частоты FM, а $\beta = \Delta f / f_m$ — индекс частотной модуляции. По умолчанию $F_s = 1$ Мвыб/с, $f_c = 100$ кГц,

$A = 0{,}5$, $f_m = 1$ кГц, $\mu = 0{,}5$, $\Delta f = 75$ кГц, длина записи $N = 100\,000$ отсчётов (0,1 с):

```
mkdir -p build/ch06/results
./build/ch06/sig_gen --type tone --out build/ch06/results/tone.cf32
./build/ch06/sig_gen --type am --mod-freq 10e3 --out build/ch06/results/am.cf32
./build/ch06/sig_gen --type fm --out build/ch06/results/fm.cf32
```

Вывод для AM и FM:

```
am: 100000 samples, 0.1000 s, offset +100000 Hz, peak 0.750 FS
AM: sidebands at +-10000 Hz, 12.04 dB below the carrier
fm: 100000 samples, 0.1000 s, offset +100000 Hz, peak 0.500 FS
FM: beta = 75.0, Carson bandwidth 152000 Hz
```

Генератор проверяет, что сигнал помещается в полосу $\pm F_s/2$ и что пик не превышает полную шкалу. Для AM пик равен $A(1+\mu)$, поэтому глубину больше единицы программа не примет: огибающая перешла бы через ноль, и сигнал перестал бы быть AM в обычном смысле.

Передача идёт из *циклического* буфера: плата раз за разом повторяет записанные $N$ отсчётов, пока программа не остановит передачу. Чтобы на стыке повторов не было скачка фазы, в записи должно укладываться целое число периодов каждой частоты:

$$\frac{f_c N}{F_s} \in \mathbb{Z}, \qquad \frac{f_m N}{F_s} \in \mathbb{Z}. \tag{6.1}$$

Для значений по умолчанию это 10 000 периодов несущей и 100 периодов модулирующего тона. Если условие нарушено, например при $f_c = 100\,003$ Гц, `sig_gen` предупреждает: скачок фазы каждые 0,1 с даст в спектре лишние составляющие, которых нет в эталоне.

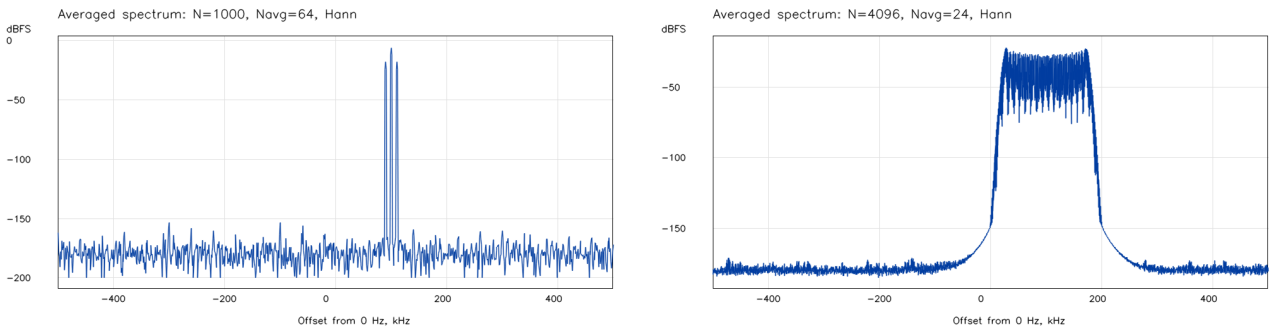



**Рис. 6.3:** Программный эталон: спектры AM ($\mu = 0{,}5$, $f_m = 10$ кГц, $N = 1000$, слева) и FM ($\Delta f = 75$ кГц, $f_m = 1$ кГц, $N = 4096$, справа), построенные `spectrum_view` по файлам генератора.

## 6.4 Программа передачи

Программа `tx_play` — зеркальное отражение `rx_capture`. Её ключевая часть без проверок ошибок:

```
iio_device* tx = iio_context_find_device(ctx, "cf-ad9361-dds-core-lpc");
iio_channel* ctl = iio_device_find_channel(phy, "voltage0", true);  // TX1
iio_channel* lo  = iio_device_find_channel(phy, "altvoltage1", true); // TX_LO

iio_channel_attr_write_double(ctl, "hardwaregain", -89.75);  // сначала тишина
ad9361_set_bb_rate(phy, rate);
iio_channel_attr_write_longlong(lo, "frequency", 433920000);

iio_channel_enable(ci);                                      // voltage0: I
iio_channel_enable(cq);                                      // voltage1: Q
iio_buffer* buf = iio_device_create_buffer(tx, n, true);     // циклический
for (...)  // 12 бит ЦАП в старших разрядах 16-битного слова
    i16 = lround(x.real() * 2047) * 16;
iio_buffer_push(buf);                                        // плата повторяет буфер

iio_channel_attr_write_double(ctl, "hardwaregain", gain);  // только теперь уровень
...                                                          // ждём T секунд
iio_channel_attr_write_double(ctl, "hardwaregain", -89.75);
iio_buffer_destroy(buf);
iio_channel_attr_write_bool(lo, "powerdown", true);          // гетеродин выключен
```

Порядок действий подчинён безопасности. Первым делом выставляется наибольшее ослабление $-89{,}75$ дБ, и только после загрузки буфера открывается заданный уровень. При завершении — по таймеру, по Ctrl+C или из-за ошибки — ослабление возвращается к максимуму, буфер уничтожается, а гетеродин передатчика выключается. Без ключа `--go` программа не подключается к плате вовсе. Это проверяет один из автоматических тестов.

При передаче отсчёты преобразуются обратно в формат ЦАП. АЦП приёмника отдавал 12-битные числа в младших разрядах 16-битного слова, а ЦАП передатчика ожидает 12 значащих бит в *старших* разрядах. Поэтому значение $x \cdot 2047$ дополнительно умножается на $16 = 2^4$, то есть сдвигается на четыре разряда влево (раздел 4.7).

Сборка программ главы и тесты:

```
cmake -S . -B build/ch06 -DCMAKE_BUILD_TYPE=Release \
```

```
  -DLAB_WITH_IIO=ON -DLAB_IIO_PREFIX="$PWD/build/iio-install" \
  -DLAB_WITH_DSP=ON -DLAB_WITH_TX=ON
cmake --build build/ch06 --parallel
ctest --test-dir build/ch06 --output-on-failure
```

Ожидается пятнадцать тестов: к двенадцати тестам главы 5 добавлены проверка уровней несущей и боковых полос AM, прохождение FM через демодулятор и отказ `tx_play` работать без `--go`.

Передача и приём запускаются в двух окнах терминала. В первом:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/ch06/tx_play --uri ip:192.168.2.1 --freq 433.92e6 \
  --in build/ch06/results/tone.cf32 --tx-gain -40 --seconds 60 --go
```

Во втором, пока идёт передача:

```
LD_LIBRARY_PATH="$PWD/build/iio-install/lib" \
  ./build/ch06/rx_capture --uri ip:192.168.2.1 --freq 433.92e6 \
  --rate 1e6 --gain 20 --samples 1000000 \
  --out build/ch06/results/rx_tone.cf32
./build/ch06/spectrum_view build/ch06/results/rx_tone.cf32 --nfft 1000
```

Если уровень на приёме слишком мал или велик, меняют `--tx-gain` и `--gain`, но не схему тракта.

## 6.5 Комплексный тон

Тон — самый простой и самый информативный сигнал: всё, что в принятом спектре отличается от одной линии на +100 кГц, внесено трактом. Ожидается следующее.

- *Основная линия на +100 кГц.* Если она оказалась на −100 кГц, где-то переставлены $I$ и $Q$ или изменён знак $Q$ (раздел 4.2.5).
- *Линия в нуле частот*: утечка гетеродина передатчика и постоянная составляющая приёмника. По спецификации трансивера утечка несущей около −50 дБн, то есть на 50 дБ ниже полезного сигнала.
- *Зеркальная линия на −100 кГц*: следствие неточной квадратуры, когда ветви $I$ и $Q$ немного различаются по амплитуде и фазе. Её уровень показывает качество калибровки квадратурных цепей.
- *Частотный сдвиг*: на одной плате передатчик и приёмник тактируются от общего опорного генератора, поэтому линия стоит на +100 кГц с точностью до шага синтезатора. При двух разных платах их опорные генераторы расходятся до ±0,5 ppm каждый, и на 433,92 МГц сдвиг может достигать сотен герц.

**Контрольная точка**

1. Принятая основная линия имеет уровень −30 dBFS, линия в нуле — −78 dBFS, зеркальная — −72 dBFS. Найдите подавление утечки гетеродина и зеркального канала в децибелах.
2. Каким окажется принятый спектр, если в `tx_play` по ошибке записать в канал $I$ значение $Q$ и наоборот?
3. * Оцените наибольший частотный сдвиг между двумя платами с опорой ±0,5 ppm на 433,92 МГц. Хватит ли шага ДПФ при $F_s = 1$ Мвыб/с и $N = 1000$, чтобы его увидеть?

Ответы для сверки: 48 и 42 дБ. Спектр зеркально отразится: основная линия окажется на −100 кГц. Сдвиг до $2 \cdot 0{,}5 \cdot 10^{-6} \cdot 433{,}92 \cdot 10^{6} \approx 434$ Гц; шаг сетки 1 кГц для этого груб, нужно $N = 10^4$ и больше ($\Delta f = 100$ Гц).

## 6.6 AM как тест амплитудного тракта

У AM-сигнала информация заключена в огибающей $A\big[1 + \mu\cos(2\pi f_m t)\big]$, поэтому он чувствителен ко всему, что искажает амплитуду: нелинейности усилителей, перегрузке АЦП, неравномерности АЧХ. В спектре однотональной AM несущая сопровождается двумя боковыми линиями на $f_c \pm f_m$ (рис. 6.3, слева). Отношение уровней несущей $L_c$ и боковой линии $L_{sb}$ в децибелах однозначно задаёт глубину модуляции:

$$L_c - L_{sb} = 20\log_{10}\frac{2}{\mu}, \qquad \mu = 2 \cdot 10^{-(L_c - L_{sb})/20}. \tag{6.2}$$

Для $\mu = 0{,}5$ боковые линии на 12,04 дБ ниже несущей: в эталоне −6,02 и −18,06 dBFS. Измеренная по принятому спектру глубина должна совпасть с заданной. Если при увеличении усиления разность $L_c - L_{sb}$ меняется, а рядом появляются линии на $f_c \pm 2f_m$, тракт работает нелинейно: вершины огибающей сжимаются. Чтобы линии AM попали точно на бины ДПФ, спектр строят с `--nfft 1000`: при $F_s = 1$ Мвыб/с шаг сетки равен 1 кГц, и все частоты 100, 90 и 110 кГц оказываются на бинах.

**Контрольная точка**

1. В принятом спектре несущая −24,3 dBFS, боковые линии −36,4 и −36,2 dBFS. Найдите глубину модуляции.
2. Почему при $N = 4096$ эталон показывает несущую −6,93 dBFS вме-

сто −6,02?

3. * Почему `sig_gen` не позволяет задать $\mu = 1{,}2$ при $A = 0{,}5$, а при $A = 0{,}4$ тоже откажет?

Ответы для сверки: $L_c - L_{sb} \approx 12{,}0$ дБ, $\mu \approx 0{,}50$. При $N = 4096$ шаг сетки 244 Гц, и частота 100 кГц попадает между бинами; потеря 0,9 дБ — это просадка окна Ханна между бинами (глава 4). Генератор ограничивает глубину значением $\mu \leq 1$: при $\mu > 1$ огибающая проходит через ноль и меняет знак; при $A = 0{,}4$ пик 0,88 ещё помещается в шкалу, но глубина по-прежнему больше единицы.

## 6.7 FM как тест фазочастотного тракта

У FM-сигнала амплитуда постоянна, а информация заключена в мгновенной фазе $\varphi(t) = 2\pi f_c t + \beta \sin(2\pi f_m t)$. Поэтому FM почти нечувствителен к нелинейности амплитуды, зато чувствителен к фазовым и частотным искажениям: шумам гетеродина, неравномерности фазовой характеристики фильтров, ошибкам частоты дискретизации. Мгновенная частота отклоняется от $f_c$ на $\pm\Delta f$; при $\Delta f \gg f_m$ занимаемая полоса хорошо оценивается правилом Карсона:

$$B \approx 2(\Delta f + f_m). \tag{6.3}$$

Для $\Delta f = 75$ кГц и $f_m = 1$ кГц получается 152 кГц: в эталоне (рис. 6.3, справа) сигнал занимает примерно 24–176 кГц. Принятый FM-сигнал демодулируют программой главы 5:

```
./build/ch06/fm_demod build/ch06/results/rx_fm.cf32 \
  build/ch06/results/rx_fm.wav --offset 100000
```

В файле должен звучать чистый тон 1 кГц. Пилот-тона 19 кГц здесь нет: программа сообщит, что вещательный сигнал не обнаружен, и это правильно — сигнал создан нами, а не радиостанцией.

**Контрольная точка**

1. Найдите ширину полосы по Карсону для $\Delta f = 5$ кГц и $f_m = 1$ кГц. Сколько процентов полосы $F_s = 1$ Мвыб/с она займёт?
2. Почему принятая FM-запись после ослабления в тракте на 30 дБ звучит так же громко, как эталон?

3. * Если в принятом FM появились медленные «плавания» тона, на какой узел тракта это указывает в первую очередь?

Ответы для сверки: 12 кГц, 1,2%. Дискриминатор измеряет частоту, а не амплитуду, и `fm_demod` нормирует громкость. Медленные изменения частоты указывают на гетеродин или опорный генератор: для одной платы маловероятно, для двух — ожидаемо.

## 6.8 Лестница уровней

Линейность тракта проверяют, меняя за один шаг только один параметр. Сначала при постоянном усилении приёмника, например 20 дБ, передают тон с ослаблением `--tx-gain` от −60 до 0 дБ с шагом 10 дБ и записывают среднеквадратичный уровень, который печатает `rx_capture`. Затем при постоянном `--tx-gain` так же меняют усиление приёмника. Результаты сводят в таблицу 6.2.

**Таблица 6.2:** Лестница уровней: заполняется на стенде

| `--tx-gain`, дБ | −60 | −50 | −40 | −30 | −20 | −10 | 0 |
|---|---|---|---|---|---|---|---|
| RMS, dBFS | | | | | | | |
| Приращение, дБ | — | | | | | | |
| В насыщении, % | | | | | | | |

В линейной области каждые 10 дБ ослабления меняют уровень на приёме ровно на 10 дБ. Где приращение стало меньше, начинается сжатие — в усилителе передатчика или в приёмнике; где появились отсчёты в насыщении, перегружен АЦП. Отдельная лестница по усилению приёмника показывает, в каком из двух узлов сжатие. Абсолютные уровни в dBFS здесь не важны: проверяется наклон 1 дБ на 1 дБ. Бюджет главы 1 гарантирует, что ни одна ступень не опасна для входа RX.

## 6.9 Сравнение с программным эталоном

Программный эталон и принятый сигнал никогда не совпадут полностью. Задача сравнения — объяснить каждое различие причиной в тракте (таблица 6.3).

**Таблица 6.3:** Что сравнивать и чем объяснять различия

| Различие | Причина |
|---|---|
| Абсолютный уровень | ослабление тракта, `--tx-gain` и `--gain`; калибровки в дБм нет |
| Линия в нуле частот | утечка гетеродина TX и постоянная составляющая RX |
| Зеркальная линия | неточность квадратурных цепей передатчика и приёмника |
| Шумовой уровень | шум приёмника и передатчика; в эталоне шума нет |
| Частотный сдвиг | при одной плате нет; при двух — расхождение опорных генераторов |
| Спад на краях полосы | аналоговые и цифровые фильтры трансивера |
| Лишние линии около сигнала | нецелое число периодов в циклическом буфере или нелинейность тракта |

Удобно сравнивать списки линий, которые печатает `spectrum_view` для эталона и для принятой записи с одинаковыми $N$ и $N_{\text{avg}}$: частоты сигнала должны совпасть, а разности уровней внутри сигнала — например между несущей и боковыми AM — сохраниться.

**Контрольная точка**

**Итоговые вопросы к главе.**

1. Какие действия обязательны до первого запуска `tx_play` и где они описаны?
2. Почему передача идёт из циклического буфера и какое условие должно выполняться для длины записи?
3. По принятому спектру AM измерена глубина 0,42 при заданной 0,5. Предложите две гипотезы и способ проверить каждую.
4. Что останется неизменным в программах, если вместо FM передавать другой вид модуляции, и что придётся изменить?
5. * Как, не меняя схему, отличить сжатие в передатчике от сжатия в приёмнике?

Ответы для сверки: сборка пассивного тракта при выключенном передатчике и решение *GO* по бюджету главы 1. Циклический буфер повторяется платой без участия программы; в нём должно укладываться целое число периодов всех частот (6.1). Гипотезы: нелинейность тракта (проверяется лестницей уровней) или неравномерность АЧХ на $f_c \pm f_m$ (проверяется переносом сигнала на другое смещение). Меняется только формула в генераторе и демодулятор; `tx_play`, `rx_capture` и плата остаются прежними. Сделать две лестницы: если наклон нарушается при росте `--tx-gain`, но не при росте `--gain`, сжимается передатчик, и на-

оборот.

# Список использованных источников


[1] SoapyPlutoSDR: модуль SoapySDR для трансиверов AD936x : исходный код. — URL: https://github.com/pothosware/SoapyPlutoSDR (дата обращения: 14.10.2026).

[2] Analog Devices. AD9361, AD9364 and AD9363 Linux device driver : документация драйвера IIO. — URL: https://developer.analog.com/docs/linux/drivers/iio-transceiver/ad9361.html (дата обращения: 14.10.2026).

[3] Analog Devices. AD9363 RF Agile Transceiver : data sheet. — URL: https://www.analog.com/media/en/technical-documentation/data-sheets/AD9363.pdf (дата обращения: 14.10.2026).

[4] Analog Devices EngineerZone. AD9361: What is the maximum RF signal power the input can handle? — URL: https://ez.analog.com/rf/wide-band-rf-transceivers/w/documents/25932/ad9361-what-is-the-maximum-rf-signal-power-the-input (дата обращения: 14.10.2026).

[5] Диаграмма направленности // Википедия : свободная энциклопедия. — URL: https://ru.wikipedia.org/?curid=131310 (дата обращения: 14.10.2026).

[6] Поляризация волн // Википедия : свободная энциклопедия. — URL: https://ru.wikipedia.org/?curid=23716 (дата обращения: 14.10.2026).

[7] РЕД СОФТ. РЕД ОС 8: управление пакетами. — URL: https://redos.red-soft.ru/base/redos-8_0/8_0-base-consept/8_0-sys-dnf/8_0-gen-info-dnf/8_0-manage-package/ (дата обращения: 14.10.2026).

[8] Analog Devices. libiio: библиотека доступа к устройствам IIO : исходный код и документация, версия 0.26. — URL: https://github.com/analogdevicesinc/libiio (дата обращения: 14.10.2026).

[9] Analog Devices. libad9361-iio: надстройка libiio для трансиверов AD936x : исходный код, версия 0.3. — URL: https://github.com/analogdevicesinc/libad9361-iio (дата обращения: 14.10.2026).

[10] Analog Devices Wiki. Customizing the device: config.txt. — URL: https://wiki.analog.com/university/tools/pluto/users/customizing (дата обращения: 14.10.2026).

[11] QPSK SDR Lab: основные характеристики / СПбГЭТУ «ЛЭТИ», кафедра радиотехнических систем. — СПб., 2026. — 1 с.

[12] Analog Devices. AD9361 RF Agile Transceiver : data sheet. Rev. F. — Norwood, MA, 2016. — 36 p. — URL: https://www.analog.com/media/en/technical-documentation/data-sheets/AD9361.pdf (дата обращения: 14.10.2026).

[13] Analog Devices. Драйвер AD9361 для ядра Linux : исходный код (файлы drivers/iio/adc/ad9361.c и ad9361_regs.h). — URL: https://github.com/analogdevicesinc/linux/blob/main/drivers/iio/adc/ad9361_regs.h (дата обращения: 14.10.2026).

[14] Analog Devices Wiki. AXI_AD9361: HDL-ядро интерфейса AD9361 для ПЛИС. — URL: https://wiki.analog.com/resources/fpga/docs/axi_ad9361 (дата обращения: 14.10.2026).

[15] The Linux Kernel documentation. Industrial I/O. — URL: https://docs.kernel.org/driver-api/iio/index.html (дата обращения: 14.10.2026).

[16] Analog Devices. libiio v0.26: пример iio_adi_xflow_check.c — проверка переполнения и опустошения буферов. — URL: https://github.com/analogdevicesinc/libiio/blob/v0.26/examples/iio_adi_xflow_check.c (дата обращения: 14.10.2026).

[17] Frigo M., Johnson S. G. FFTW 3.3.11 Manual. — URL: https://fftw.org/fftw3.pdf (дата обращения: 14.10.2026).

[18] Analog Devices. libiio v0.26: описание API (файл iio.h). — URL: https://github.com/analogdevicesinc/libiio/blob/v0.26/iio.h (дата обращения: 14.10.2026).

[19] Analog Devices. libiio v0.26: пример ad9361-iiostream.c — работа с буферами AD936x. — URL: https://github.com/analogdevicesinc/libiio/blob/v0.26/examples/ad9361-iiostream.c (дата обращения: 14.10.2026).

[20] SigMF — Signal Metadata Format : спецификация. — URL: https://github.com/sigmf/SigMF (дата обращения: 14.10.2026).